

NoFeeSwap™ [Draft]

Yellow Paper*

June 23, 2026

Abstract

NoFeeSwap is an Automated Market Making (AMM) protocol with several benefits compared to other Decentralized Exchanges (DEXs). In particular, it offers zero spread between the buy and sell marginal prices, while, at the same time, guarantees liquidity growth for Liquidity Providers (LPs) after each swap. This enables higher trading activity, even after small fluctuations of the market price, and, consequently, higher profitability for LPs. Various theoretical and empirical arguments are provided to demonstrate superior profitability for LPs compared to other platforms. The protocol has inherent compounding for liquidity growth, while still offering LPs the ability to concentrate liquidity within their desired ranges. A wide variety of innovative customizations ensure the utmost flexibility in liquidity distribution for LPs. This document explains concepts that are central to the protocol framework and provides proofs for the efficacy and security of the protocol. The description of smart contract methods and the point-based incentive program are covered in this document.

*Website: www.nofeeswap.org; Contact: contact@nofeeswap.org; US Patent Pending; Copyright 2025, NoFeeSwap LLC - All rights reserved.

1 Introduction

NoFeeSwap is a customizable Automated Market Making (AMM) protocol where traders do not pay fees and there is an inherent zero spread between the buy and sell marginal prices.

The NoFeeSwap platform consists of liquidity pools. Each pool is dedicated to swapping between a pair of tokens, x and y . Liquidity providers (LPs) contribute to token reserves in order to grow their liquidity ownership. Furthermore, traders use the available reserves to swap one token for the other.

NoFeeSwap offers LPs the ability to concentrate their liquidity within the desired price ranges. To facilitate that, each liquidity pool is partitioned into a sequence of price intervals. LPs make a profit from the swaps resulting from price fluctuations within their selected range.

Consider a liquidity pool dedicated to swaps between tokens x and y . Let $\mathbb{Z}_+ \triangleq \{1, 2, \dots\}$ denote a sequence of swaps where each $t \in \mathbb{Z}_+$ represents the pool's state before the t^{th} swap.¹ Denote the set of non-negative real numbers by $\mathbb{R}_+$. For each $t \in \mathbb{Z}_+$, let the pair $(x_t, y_t) \in \mathbb{R}_+^2$ represent the reserve amounts for tokens x and y before the t^{th} trade.² A trader, through the t^{th} trade, moves the reserve values to the point $(x_{t+1}, y_{t+1}) \in \mathbb{R}_+^2$ by exchanging $\delta x = x_{t+1} - x_t$ amount of token x and $\delta y = y_{t+1} - y_t$ amount of token y , where the exchange amounts, δx and δy , are subject to a mathematical formula which is referred to as the *curve*.³ The remainder of Section 1 discusses the following questions.

How can one build a protocol that follows the market price in an automated fashion without external input from the LPs? Section 1.2 provides such a framework after the more introductory formulation of Section 1.1. How can one make the platform profitable for LPs using the buy/sell spread? Section 1.3 explains the addition of a fee and its drawbacks. Why is it helpful to avoid the buy/sell spread? Section 1.3 briefly discusses this as well. How can one avoid the buy/sell spread? The NoFeeSwap protocol is an answer.

Section 2 introduces a variety of tools and techniques. In Section 3, the NoFeeSwap protocol is discussed in detail, including references to the implementations. Section 4 discusses an operator for interacting with the protocol. A point-based incentive system is built as a hook and is discussed in Section 5.

See NoFeeSwap's litepaper [1] for discussions on the advantages and the

¹ $t = 1$ represents the initial state of the pool.

²The terms *trade* and *swap* are used interchangeably in this document.

³The difference amount is positive for the token that is taken by the protocol and is negative for the other token that is given by the protocol.

desired features of the protocol for various market participants and community members.

1.1 Constant Price Liquidity Pool

Consider a simple protocol which is governed by a constant price $\check{p}$. In other words, traders are allowed to exchange any arbitrary amount of token y with token x at price $\check{p}$, subject to availability. More precisely, a trader can move the liquidity reserve values from $(x_t, y_t) \in \mathbb{R}_+^2$ to $(x_{t+1}, y_{t+1}) \in \mathbb{R}_+^2$, if and only if:

$$\check{p} x_{t+1} + y_{t+1} = \check{p} x_t + y_t. \quad (1)$$

In other words, $\check{p} = \frac{y_t - y_{t+1}}{x_{t+1} - x_t}$. Given the market price of $\check{p}_t$, a rational arbitrageur / trader is incentivized to solve the following problem in order to maximize their profits:

$$\begin{aligned} & \underset{x_{t+1}, y_{t+1} \geq 0}{\text{minimize}} && \check{p}_t (x_{t+1} - x_t) + (y_{t+1} - y_t) \end{aligned} \quad (2a)$$

$$\text{subject to} \quad \check{p} x_{t+1} + y_{t+1} = \check{p} x_t + y_t. \quad (2b)$$

In simple terms, the objective is to determine the optimal values for x_{t+1} and y_{t+1} to minimize the loss (2a) (maximize the profit) subject to (2b). Observe that the objective function represents the total loss of the trader in terms of token y , which is supposed to be a negative number because zero loss is attainable through $(x_{t+1}, y_{t+1}) = (x_t, y_t)$ ⁴. It is straightforward to verify that the answer to problem (2) is:

$$x_{t+1} = \begin{cases} 0 & \check{p} < \check{p}_t \\ x_t + \check{p}^{-1} y_t & \check{p} > \check{p}_t \end{cases}, \quad y_{t+1} = \begin{cases} \check{p} x_t + y_t & \check{p} < \check{p}_t \\ 0 & \check{p} > \check{p}_t \end{cases}. \quad (3)$$

In other words, the token sold below the market price is depleted from the reserve.⁵ This is illustrated in Figure 1.

This rudimentary trading protocol does not follow the market price and is not profitable for LPs. Section 1.2 deals with the first issue and Section 1.3 with the latter.

⁴which corresponds to a zero swap

⁵If $\check{p}_t = \check{p}$, the trader's profit is zero.

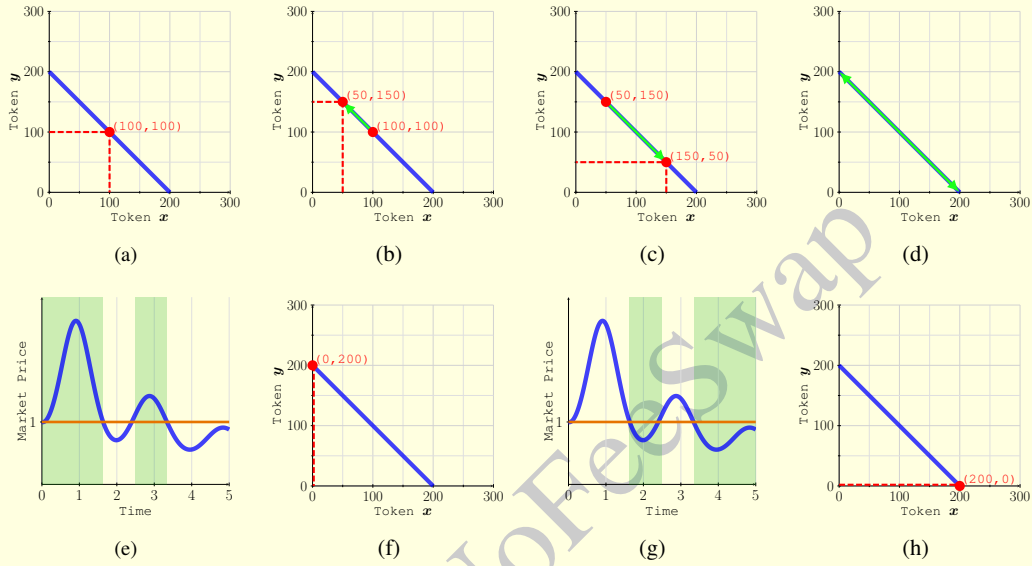


Figure 1: A constant price liquidity pool: (a) Consider 100 units of reserve for the tokens x and y with a constant price of 1; (b) A trader can move reserve values from the point $(100, 100)$ to the point $(50, 150)$ by trading 50 units of token y in exchange for 50 units of token x ; (c) Another trader can then move the reserve values from point $(50, 150)$ to point $(150, 50)$ by trading 100 units of token x in exchange for 100 units of token y ; (d) Traders are allowed to move the pool's reserve values to any point that belongs to the highlighted segment; (e) During the time when the market price exceeds 1 (the highlighted green area), rational traders are incentivized to buy the pool's entire reserve of token x ; (f) In this case, traders move the pool's reserve values to the leftmost corner of the segment, i.e., the point $(0, 200)$; (g) During the time when the market price is below 1 (the highlighted green area), rational traders are incentivized to buy the whole reserve of the y token in the pool; (h) In this case, traders move the reserve values to the right corner of the segment, i.e., the point $(200, 0)$.

1.2 Following the Market Price

In this section, a variety of AMM frameworks are discussed that follow the market price without requiring any input from the LPs. To this end, consider a *strictly concave* and *differentiable* function $c : \mathbb{R}_+^2 \rightarrow \mathbb{R}$, where a trader is permitted to move the reserve values from (x_t, y_t) to (x_{t+1}, y_{t+1}) , if and only if:

$$c(x_{t+1}, y_{t+1}) = c(x_t, y_t). \quad (4)$$

Observe that the previously discussed constant price liquidity pool corresponds to $c(x, y) = \check{p}x + y$. However, the additional requirement of strict concavity incentivizes rational traders to synchronize the pool's marginal price with the market price. To verify this, let us calculate the marginal price of this protocol for an infinitesimal trade. If $x_{t+1} - x_t \rightarrow 0$ and $y_{t+1} - y_t \rightarrow 0$, then according to Taylor series expansion:

$$\begin{aligned} c(x_t, y_t) &= c(x_{t+1}, y_{t+1}) \\ &\approx c(x_t, y_t) + c^{(1,0)}(x_t, y_t)(x_{t+1} - x_t) + c^{(0,1)}(x_t, y_t)(y_{t+1} - y_t), \end{aligned} \quad (5)$$

which results in the marginal price of:

$$\frac{y_t - y_{t+1}}{x_{t+1} - x_t} = \frac{c^{(1,0)}(x_t, y_t)}{c^{(0,1)}(x_t, y_t)}. \quad (6)$$

Now, given the market price of $\check{p}_t$, consider a rational trader that intends to maximize their profit. This trader solves the following:

$$\begin{aligned} &\underset{x_{t+1}, y_{t+1} \geq 0}{\text{minimize}} && \check{p}_t (x_{t+1} - x_t) + (y_{t+1} - y_t) \end{aligned} \quad (7a)$$

$$\text{subject to} \quad c(x_{t+1}, y_{t+1}) = c(x_t, y_t). \quad (7b)$$

Due to continuity of c , one may relax constraint (7b) to an inequality, without affecting the optimal solution:⁶

$$\begin{aligned} &\underset{x_{t+1}, y_{t+1} \geq 0}{\text{minimize}} && \check{p}_t (x_{t+1} - x_t) + (y_{t+1} - y_t) \end{aligned} \quad (8a)$$

$$\text{subject to} \quad c(x_{t+1}, y_{t+1}) \geq c(x_t, y_t). \quad (8b)$$

⁶If (x_{t+1}^*, y_{t+1}^*) is an optimal solution and $c(x_{t+1}^*, y_{t+1}^*) > c(x_t, y_t)$, then for a sufficiently small $\varepsilon > 0$, $(x_{t+1}^* - \varepsilon, y_{t+1}^* - \varepsilon)$ would be feasible with a lower cost value, which contradicts the optimality of (x_{t+1}^*, y_{t+1}^*) . Hence, $c(x_{t+1}^*, y_{t+1}^*) = c(x_t, y_t)$ holds for every optimal pair (x_{t+1}^*, y_{t+1}^*) , and the transformation of equality into inequality is lossless.

The optimal solution satisfies the following stationarity conditions:

$$\check{p}_t + \lambda^* c^{(1,0)}(x_{t+1}^*, y_{t+1}^*) = 0, \quad (9)$$

$$1 + \lambda^* c^{(0,1)}(x_{t+1}^*, y_{t+1}^*) = 0, \quad (10)$$

where λ^* is an optimal Lagrange multiplier and (x_{t+1}^*, y_{t+1}^*) is an optimal primal solution. Hence, canceling λ^* concludes:

$$\frac{c^{(1,0)}(x_{t+1}^*, y_{t+1}^*)}{c^{(0,1)}(x_{t+1}^*, y_{t+1}^*)} = \check{p}_t, \quad (11)$$

which means that, as a result of the profit-motivated trader's action, the pool's updated marginal price, $\frac{c^{(1,0)}(x_{t+1}, y_{t+1})}{c^{(0,1)}(x_{t+1}, y_{t+1})}$, is now synchronized with the market price, $\check{p}_t$.

A classical example of such a structure [2] is the constant product market making framework for which one may take the strictly concave invariant $c(x, y) = \log_e(x) + \log_e(y)$.⁷ This framework is illustrated in Figure 2.

1.3 Trading Fees and Profitability for LPs

In this section, we focus on the profitability of the protocol for LPs. A popular approach⁸ is to impose a fee ratio $0 \leq f < 1$ by allowing transactions that satisfy:

$$\begin{cases} c(x_t + (1-f)(x_{t+1} - x_t), y_{t+1}) = c(x_t, y_t) & \text{if } x_{t+1} > x_t, y_{t+1} < y_t, \\ c(x_{t+1}, y_t + (1-f)(y_{t+1} - y_t)) = c(x_t, y_t) & \text{if } x_{t+1} < x_t, y_{t+1} > y_t. \end{cases} \quad (12)$$

In other words, when the token x is traded in exchange for the token y , the pool confiscates an f portion of the incoming amount $x_{t+1} - x_t$ as a fee. Similarly, when the token y is traded in exchange for the token x , the pool confiscates an f portion of the incoming amount $y_{t+1} - y_t$ as a fee.

In this case, effectively, there are two distinct marginal prices, one for the case of the pool buying token x and one for the case of the pool buying token y . In order to derive the first one, we resort to Taylor expansion again:

$$\begin{aligned} 0 &= c(x_t + (1-f)(x_{t+1} - x_t), y_{t+1}) - c(x_t, y_t) \\ &\approx (1-f)c^{(1,0)}(x_t, y_t)(x_{t+1} - x_t) + c^{(0,1)}(x_t, y_t)(y_{t+1} - y_t), \end{aligned} \quad (13)$$

⁷In the context of cryptocurrency, the first mention of decentralized market making is by Ethereum's Vitalik Buterin. Martin Koppelmann of Gnosis later expanded on this idea and proposed a constant product AMM pricing scheme.

⁸and indeed the only approach before the NoFeeSwap protocol

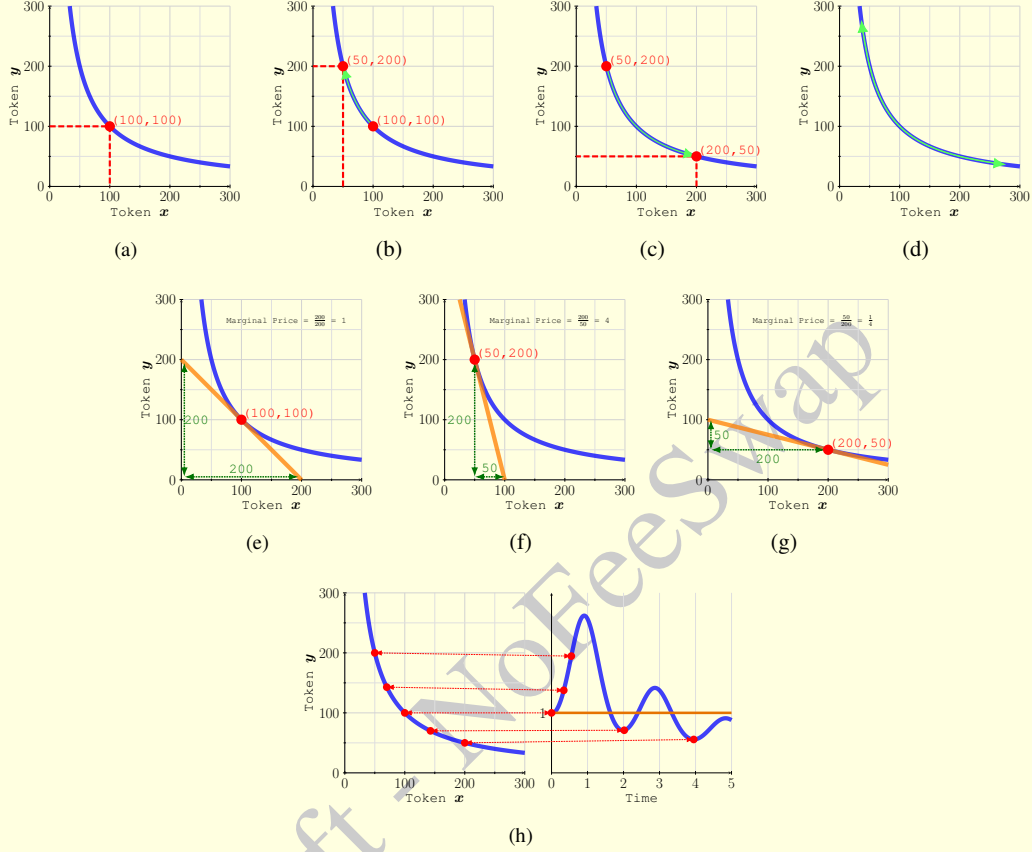


Figure 2: A constant product liquidity pool: (a) Consider 100 units of reserve for both tokens x and y ; (b) A trader can move the reserve values from point (100, 100) to point (50, 200) by trading 100 units of token y in exchange for 50 units of token x ; (c) Another trader can then move the reserve values from point (50, 200) to point (200, 50) by trading 150 units of token x in exchange for 150 units of token y ; (d) Traders are allowed to move the pool's reserve values to any point that belongs to the curve $xy = 10000$; (e), (f) and (g) illustrate the resulting marginal price changes as a result of the reserve movements; (h) The primary benefit of this protocol is that rational traders are always incentivized to adjust the pool's marginal price in accordance with the market price. As the token prices fluctuate (right-side), the pool's marginal price adjusts accordingly (left-side).

which leads to the marginal price of:

$$\frac{y_t - y_{t+1}}{x_{t+1} - x_t} = (1 - f) \frac{c^{(1,0)}(x_t, y_t)}{c^{(0,1)}(x_t, y_t)}. \quad (14)$$

Similarly, when the pool sells token x , the marginal price is derived as:

$$\begin{aligned} 0 &= c(x_{t+1}, y_t + (1 - f)(y_{t+1} - y_t)) - c(x_t, y_t) \\ &\approx c^{(1,0)}(x_t, y_t)(x_{t+1} - x_t) + (1 - f)c^{(0,1)}(x_t, y_t)(y_{t+1} - y_t), \end{aligned} \quad (15)$$

which leads to the marginal price of:

$$\frac{y_t - y_{t+1}}{x_{t+1} - x_t} = \frac{1}{1 - f} \frac{c^{(1,0)}(x_t, y_t)}{c^{(0,1)}(x_t, y_t)}. \quad (16)$$

As demonstrated in Figure 3, the marginal prices of buy and sell are different here. Their gap is referred to as the *spread*. This results in a *liquidity growth* after each swap, which leads to profitability for LPs.

Remark 1. *Note that the buy and sell marginal prices are different, i.e., the curves are not differentiable. Also, the curves before and after the update are not tangent to each other.*

Despite the profitability of LPs, a drawback of incorporating fees is that small market price fluctuations may not induce arbitrage trades when the market price remains inside the spread, which negatively impacts profits for LPs. More precisely,

- Fluctuations in the market price incentivize arbitrageurs to perform trades and synchronize pool prices with the market price.
- Trades result in liquidity growth and generate profit for LPs.
- Small price fluctuations are much more likely and more frequent than major price changes.
- If the market price falls within a pool's marginal spread interval, there would be no incentive for the arbitrageurs to trade in that pool.

Due to the above arguments, the existence of fees decreases arbitrage activities and the corresponding profits for LPs. Because of this, fee ratios are expected to be small in AMM pools. The second undesired consequence of fees is the

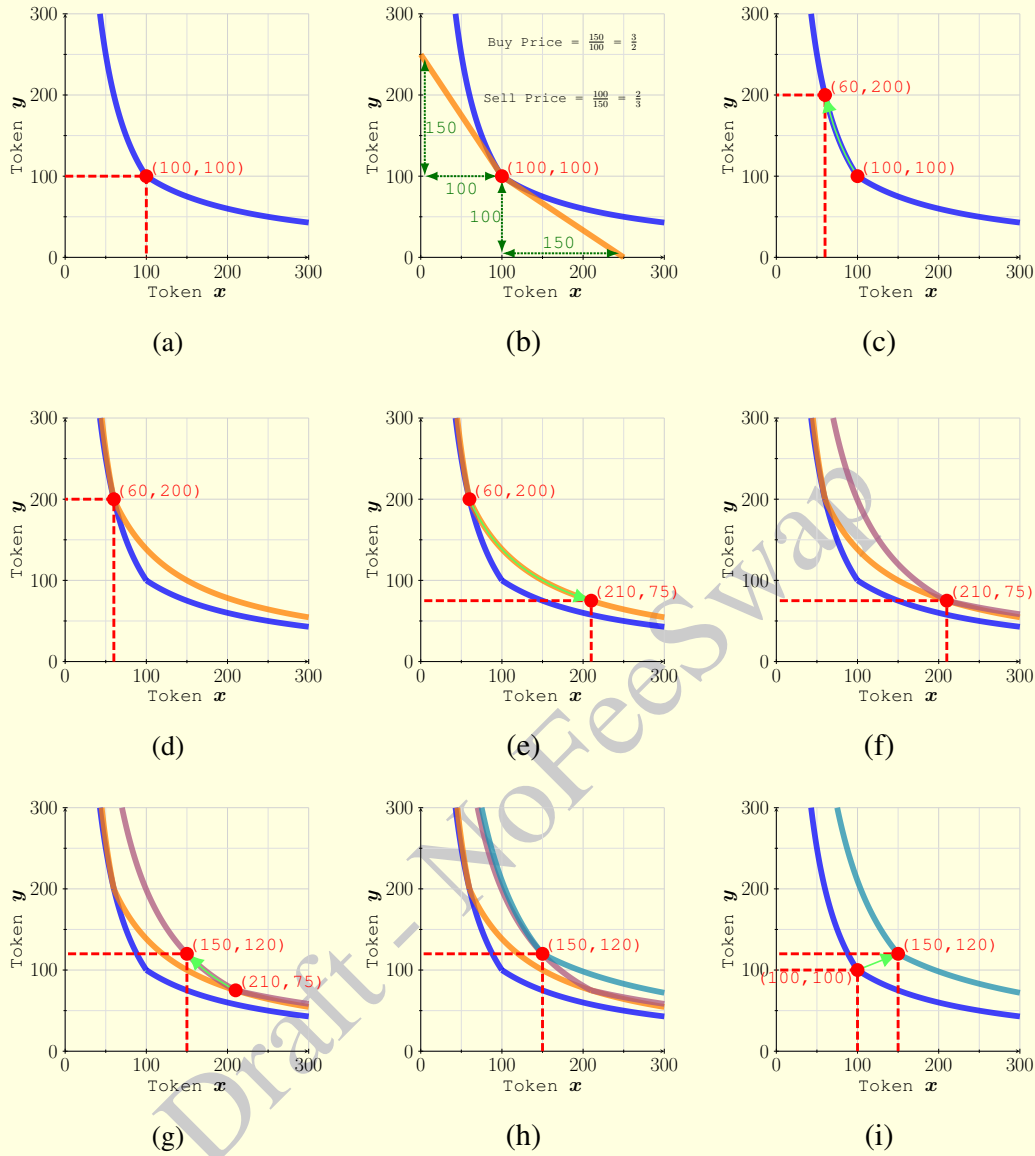


Figure 3: A constant product liquidity pool with an exaggerated $\frac{1}{3}$ fee ratio. The fee ratio typically does not exceed 1%. However, for visualization purposes, a high fee ratio is considered here: (a) Consider 100 units of reserve for each of the tokens x and y ; (b) Due to the presence of fees, the resulting curve is not smooth, which results in a gap between the buy and sell marginal prices; (c) A trader can move the reserve values from point $(100, 100)$ to point $(60, 200)$ by trading 100 units of token y in exchange for 40 units of token x ; (d) The curve then evolves into a new one; (e) Another trader can then move the reserve values from point $(60, 200)$ to point $(210, 75)$ by trading 150 units of token x in exchange for 125 units of token y ; (f) The curve then evolves into a new one; (g) Another trader may then move the reserve values from the point $(210, 75)$ to the point $(150, 120)$ by trading 45 units of token y in exchange for 60 units of token x ; (h) The curve then evolves into a new one; (i) This is profitable for LPs as the position of the pool changed from the initial point $(100, 100)$ to the point $(150, 120)$ with an overall profit of 50 token x and 20 token y .

corresponding price staleness. Last but not least, many traders, and particularly retail traders, do not consider trading in protocols/pools with significant fees altogether.

Given the above arguments, a question arises as to whether one can design an LP-profitable AMM platform with zero marginal spread, resulting in superior trade activity and profitability for LPs. Addressing this question is one of the core objectives of the NoFeeSwap protocol.

1.4 Concentrated Liquidity and Price Intervals

As explained before, a trade is tantamount to moving the price. An arbitrageur is incentivized to synchronize the price of a liquidity pool with the market price to gain profit. The higher the amount of liquidity in a pool, the larger the trade required for a fixed change in the market price in terms of tokens x and y . Additionally, the amount of profit from a liquidity growth for each LP is proportional to their liquidity.

One of the main drawbacks of protocols without concentrated liquidity is that the deposited liquidity is distributed over all price values within the range $(0, \infty)$. This is an inherent inefficiency that is the focus of platforms such as Uniswap v3 [3] and Curve.fi [4]. For instance, consider two stable tokens, whose values, by definition, do not fluctuate significantly. The price of one token with respect to the other one is expected to remain within a small vicinity of a fixed value. A question arises as to whether an LP can concentrate their liquidity in that particular vicinity as opposed to the entire range $(0, \infty)$, to gain more profit. To this end, Uniswap v3 modified the constant product scheme by employing virtual reserves, with the drawback of losing the growth compounding feature.

NoFeeSwap offers concentrated liquidity by partitioning the price horizon $(0, \infty)$ into custom intervals and allowing LPs to concentrate their liquidity to any set of intervals of their choosing. This is further illustrated in the next section, where various examples of liquidity distribution are explained.⁹

2 Zero Spread Market Making

In this section, a class of LP-profitable AMM protocols with zero marginal spread is constructed. For any arbitrary $t \in \mathbb{Z}_+$, consider a curve whose positive-reserve

⁹In the previous examples, there was no concentrated liquidity.

part is represented by a *strictly concave* and *differentiable* function $c_t : \mathbb{R}_{>0}^2 \rightarrow \mathbb{R}$. A trader is permitted to move the reserve values from (x_t, y_t) to (x_{t+1}, y_{t+1}) if and only if:

$$c_t(x_{t+1}, y_{t+1}) = c_t(x_t, y_t). \quad (17)$$

After this trade, a new function $c_{t+1} : \mathbb{R}_{>0}^2 \rightarrow \mathbb{R}$ is generated that satisfies the following:

$$c_{t+1}(x_{t+1}, y_{t+1}) = c_t(x_{t+1}, y_{t+1}), \quad (18)$$

$$\frac{c_{t+1}^{(1,0)}(x_{t+1}, y_{t+1})}{c_{t+1}^{(0,1)}(x_{t+1}, y_{t+1})} = \frac{c_t^{(1,0)}(x_{t+1}, y_{t+1})}{c_t^{(0,1)}(x_{t+1}, y_{t+1})}. \quad (19)$$

This procedure is illustrated in Figure 4. According to the differentiability of each function c_t , the marginal prices of buy and sell are identical. Furthermore, requirement (18) ensures the *conservation* of tokens, i.e., the two curves intersect at (x_{t+1}, y_{t+1}) . Equation (19) ensures that the two curves are *tangent* to each other at (x_{t+1}, y_{t+1}) . Therefore, the marginal price does not change in the transition from one curve to the next. As in Remark 1, this contrasts the approach in Section 1.3.

In this document, we intend to answer several questions: How is it possible and sustainable (in terms of complexity) to evolve these curves? What explicit strategy should be pursued for curve updates? What curves and curve updates are safe from the LPs' perspective?

In the rest of this section, additional tools are built before answering these questions in Section 3.

2.1 Coin Reserves as Functions of Price

It is instructive to view each trade as a price movement. If $\check{p} \in (0, \infty)$ represents the marginal price of the pool, define $\check{q} \triangleq \log_e(\check{p}) \in (-\infty, \infty)$. We work with the natural logarithm of the price since it simplifies the mathematical derivations due to the symmetry of the domain $(-\infty, \infty)$ and the logarithm is a common measure for price movements.

In the following, a parameterization of each curve with respect to $\check{q}$ is discussed. Let $e^{\check{q}_{\text{lower}}}$ and $e^{\check{q}_{\text{upper}}}$ represent the minimum and maximum price values for the current active liquidity interval that contains the current pool price. In other words,

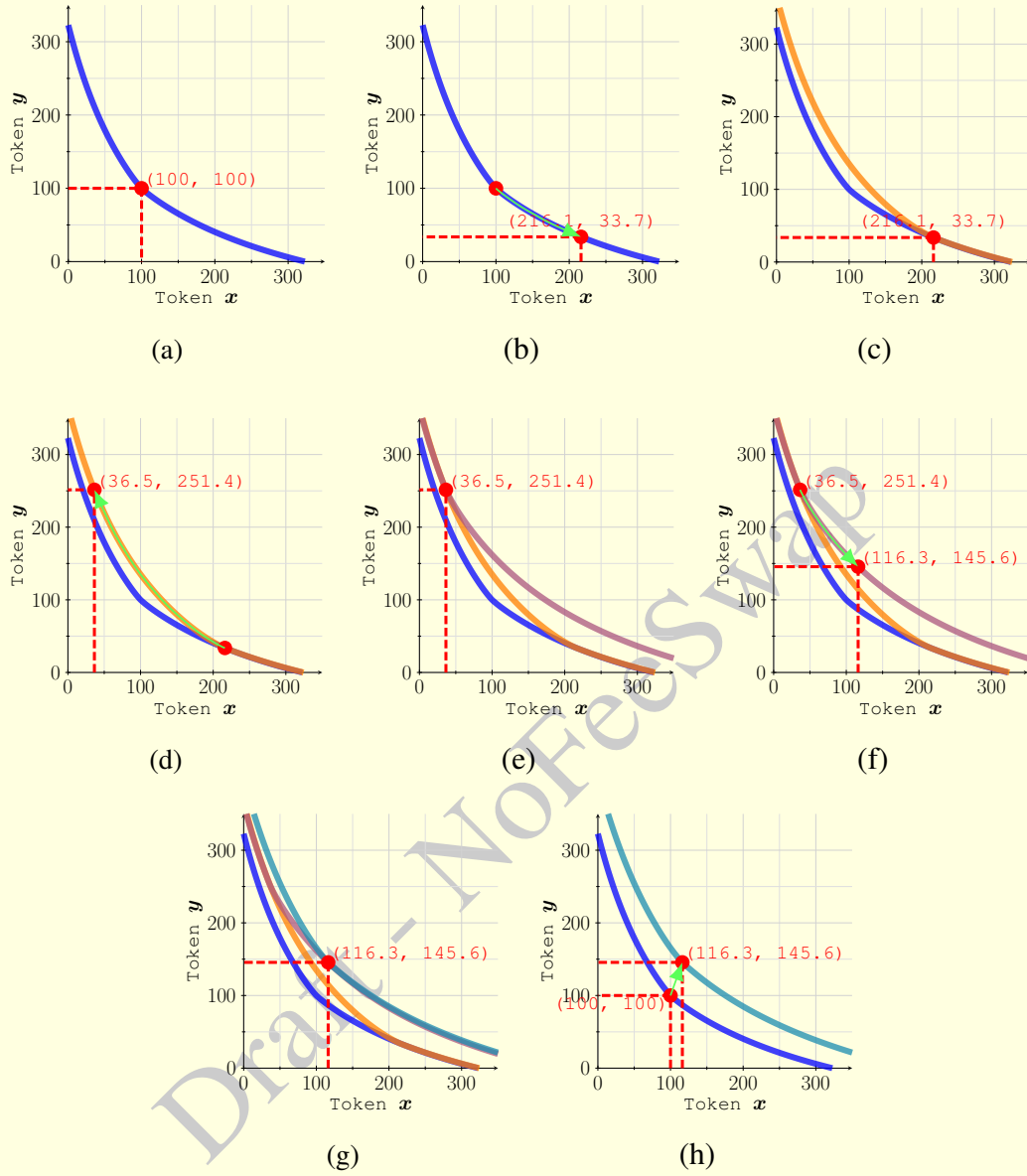


Figure 4: A zero spread market making pool with concentrated liquidity and an LP position within the price limits $p_{\min} = \frac{1}{4}$ and $p_{\max} = 4$: (a) Consider 100 units of reserve for each of the tokens x and y ; (b) A trader can move the reserve values from point (100, 100) to point (216.1, 33.7); (c) The curve then evolves into a new one, which is smooth and tangent to the initial curve; (d) Another trader can then move the reserve values from point (216.1, 33.7) to point (36.5, 251.4); (e) The curve then evolves into a new one; (f) Another trader can then move the reserve values from point (36.5, 251.4) to point (116.3, 145.6); (g) The curve then evolves into a new one; (h) This is profitable for LPs as the position of the pool changed from the initial point (100, 100) to the point (116.3, 145.6) with an overall profit of 16.3 token x and 45.6 token y .

$\check{p}_{\text{current}}$ satisfies $\check{q}_{\text{current}} = \log_e(\check{p}_{\text{current}}) \in [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}]$.¹⁰ For each $t \in \mathbb{Z}_+$, define $\mathbf{x}_t, \mathbf{y}_t : [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}] \rightarrow \mathbb{R}_+$ such that:

$$c_t(\mathbf{x}_t(\check{q}), \mathbf{y}_t(\check{q})) = c_t(x_t, y_t), \quad \forall \check{q} \in [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}], \quad (20a)$$

$$\frac{\mathbf{y}'_t(\check{q})}{\mathbf{x}'_t(\check{q})} = -e^{\check{q}}, \quad \forall \check{q} \in (\check{q}_{\text{lower}}, \check{q}_{\text{upper}}), \quad (20b)$$

where the values $x_t, y_t \in \mathbb{R}_+$ denote the amount of reserves in liquidity interval $[\check{q}_{\text{lower}}, \check{q}_{\text{upper}}]$ just before the t^{th} swap. $\mathbf{x}_t(\check{q})$ and $\mathbf{y}_t(\check{q})$ are parameterized reserve functions of token amounts and represent the pair amounts that satisfy (20a) for all log-prices in this liquidity interval. These two functions specify all potential reserve values in this liquidity interval after the t^{th} swap.

Equation (20a) is an immediate consequence of equation (17). Additionally, equation (20b) is the basic definition of the marginal price because:

$$\begin{aligned} \frac{\mathbf{y}'(\check{q})}{\mathbf{x}'(\check{q})} &= \frac{\frac{dy}{d\check{q}}}{\frac{dx}{d\check{q}}} = \frac{dy}{dx} = \frac{\text{small change in the reserve amount of token } \mathbf{y}}{\text{small change in the reserve amount of token } \mathbf{x}} \\ &= -\text{marginal price} = -\check{p} = -e^{\check{q}}. \end{aligned} \quad (21)$$

The above parameterization enables us to derive the token exchange amounts with respect to a change in the marginal price.

Now, a sequence of log-price values, $\{\check{q}_t \in \mathbb{R}\}_{t \in \mathbb{Z}_+}$, can be considered such that for each $t \in \mathbb{Z}_+$, the corresponding trade (t^{th} trade) involves a movement from the price $e^{\check{q}_t}$ to $e^{\check{q}_{t+1}}$. Additionally, for each $t \in \mathbb{Z}_+$:¹¹

$$x_t = \mathbf{x}_t(\check{q}_t), \quad y_t = \mathbf{y}_t(\check{q}_t), \quad (22a)$$

$$x_{t+1} = \mathbf{x}_t(\check{q}_{t+1}), \quad y_{t+1} = \mathbf{y}_t(\check{q}_{t+1}). \quad (22b)$$

Put simply, for each $\check{q} \in [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}]$, the pair $(\mathbf{x}_t(\check{q}), \mathbf{y}_t(\check{q}))$ denotes the resulting reserve values if the price moves from $e^{\check{q}_t}$ to $e^{\check{q}}$.

The system of differential equations (20) can be solved for a constant product market making framework by taking the strictly concave invariant $c_t(x, y) = \log_e(x) + \log_e(y)$. Its level sets satisfy:

$$\mathbf{x}_t(\check{q}) \mathbf{y}_t(\check{q}) = x_t y_t, \quad (23)$$

¹⁰When the pool price is on an interval boundary, one of the intervals on its left or right is active, and for moving to the other interval, a transition is needed. The transition is explained in Section 3.6. In Section 2, we focus only on the active liquidity interval.

¹¹As in equation (18), $\mathbf{x}_{t+1}(\check{q}_{t+1}) = \mathbf{x}_t(\check{q}_{t+1})$ and $\mathbf{y}_{t+1}(\check{q}_{t+1}) = \mathbf{y}_t(\check{q}_{t+1})$ hold as well, meaning that the curve update after each swap preserves the token amounts.

resulting in the following reserve functions:¹²

$$x_t(\check{q}) = \sqrt{x_t y_t} e^{-\frac{\check{q}}{2}}, \quad (26a)$$

$$y_t(\check{q}) = \sqrt{x_t y_t} e^{+\frac{\check{q}}{2}}. \quad (26b)$$

Example 1. If $\check{q}_{\text{lower}} = -\infty$, $\check{q}_{\text{upper}} = +\infty$, $x_t = y_t = 100$, and $c_t(x, y) = \log_e(x) + \log_e(y)$, then $x_t(\check{q}) = 100e^{-\frac{\check{q}}{2}}$ and $y_t(\check{q}) = 100e^{+\frac{\check{q}}{2}}$.

In the next section, the notion of liquidity distribution is introduced, which is central to the NoFeeSwap protocol.

2.2 Liquidity Distribution Function

Consider a liquidity interval with a sequence of curves as in (20a) where the positive-reserve part of each curve is represented by a strictly concave and differentiable function $c_t : \mathbb{R}_{>0}^2 \rightarrow \mathbb{R}$. Furthermore, assume that for every t , equations (18) and (19) are satisfied.

For every $t \in \mathbb{Z}_+$, consider a pair of parameterized reserve functions $x_t, y_t : [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}] \rightarrow \mathbb{R}_+$ with respect to the natural logarithm of the marginal price, $\check{q}$, which satisfies the system of equations (20). These functions are oriented so that x_t decreases and y_t increases with $\check{q}$ on the interior of the active liquidity interval.

For every $t \in \mathbb{Z}_+$, define the *liquidity distribution* function $\check{z}_t : [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}] \rightarrow \mathbb{R}_+$ as:

$$\check{z}_t(\check{q}) \triangleq \sqrt{-x'_t(\check{q})y'_t(\check{q})}, \quad (27)$$

¹²Another example is the Balancer platform [5], with the formula:

$$x_t^\alpha(\check{q})y_t^{1-\alpha}(\check{q}) = x_t^\alpha y_t^{1-\alpha}, \quad (24)$$

where the constant $\alpha \in (0, 1)$ is set by the pool creator. The reserve functions in this case are:

$$x_t(\check{q}) = x_t^\alpha y_t^{1-\alpha} \left(\frac{\alpha e^{\check{q}}}{1-\alpha} \right)^{1-\alpha}, \quad (25a)$$

$$y_t(\check{q}) = x_t^\alpha y_t^{1-\alpha} \left(\frac{\alpha e^{\check{q}}}{1-\alpha} \right)^{-\alpha}. \quad (25b)$$

which leads to:

$$x_t(\tilde{q}) = \int_{\tilde{q}}^{\tilde{q}_{\text{upper}}} e^{-\frac{h}{2}} \tilde{z}_t(h) dh, \quad (28a)$$

$$y_t(\tilde{q}) = \int_{\tilde{q}_{\text{lower}}}^{\tilde{q}} e^{+\frac{h}{2}} \tilde{z}_t(h) dh. \quad (28b)$$

These are the amounts of reserve tokens in the active liquidity interval. In order to prove equations (28a) and (28b), observe that:¹³¹⁴

$$\begin{aligned} \int_{\tilde{q}}^{\tilde{q}_{\text{upper}}} e^{-\frac{h}{2}} \tilde{z}_t(h) dh &= \int_{\tilde{q}}^{\tilde{q}_{\text{upper}}} \sqrt{-\frac{x'_t(h)}{y'_t(h)}} \sqrt{-x'_t(h)y'_t(h)} dh \\ &= \int_{\tilde{q}}^{\tilde{q}_{\text{upper}}} |x'_t(h)| dh \\ &= \int_{\tilde{q}}^{\tilde{q}_{\text{upper}}} -x'_t(h) dh = x_t(\tilde{q}) - x_t(\tilde{q}_{\text{upper}}) = x_t(\tilde{q}), \end{aligned} \quad (29a)$$

$$\begin{aligned} \int_{\tilde{q}_{\text{lower}}}^{\tilde{q}} e^{+\frac{h}{2}} \tilde{z}_t(h) dh &= \int_{\tilde{q}_{\text{lower}}}^{\tilde{q}} \sqrt{-\frac{y'_t(h)}{x'_t(h)}} \sqrt{-x'_t(h)y'_t(h)} dh \\ &= \int_{\tilde{q}_{\text{lower}}}^{\tilde{q}} |y'_t(h)| dh \\ &= \int_{\tilde{q}_{\text{lower}}}^{\tilde{q}} +y'_t(h) dh = y_t(\tilde{q}) - y_t(\tilde{q}_{\text{lower}}) = y_t(\tilde{q}). \end{aligned} \quad (29b)$$

Hence, given the interval bounds and current price, the pool behavior for the t^{th} swap is characterized by the liquidity distribution function $\tilde{z}_t$, i.e., the liquidity distribution function is sufficient to determine the token amounts for the t^{th} swap.

Remark 2. Below are the token exchange amounts through the t^{th} swap:

$$x_t(\tilde{q}_{t+1}) - x_t(\tilde{q}_t) = \int_{\tilde{q}_{t+1}}^{\tilde{q}_t} e^{-\frac{h}{2}} \tilde{z}_t(h) dh, \quad (30a)$$

$$y_t(\tilde{q}_{t+1}) - y_t(\tilde{q}_t) = \int_{\tilde{q}_t}^{\tilde{q}_{t+1}} e^{+\frac{h}{2}} \tilde{z}_t(h) dh. \quad (30b)$$

¹³When the pool price increases, $x_t(\tilde{q})$ decreases and $y_t(\tilde{q})$ increases, i.e., $|x'_t(h)| = -x'_t(h)$ and $|y'_t(h)| = +y'_t(h)$.

¹⁴ $x_t(\tilde{q}_{\text{upper}}) = y_t(\tilde{q}_{\text{lower}}) = 0$ holds as the concentrated liquidity ensures the depletion of one token when a liquidity interval's boundary is reached, i.e., the depletion of x on the upper boundary and y on the lower boundary.

Remark 3. Consider an infinitesimal log-price interval $[\check{q}, \check{q} + \delta\check{q}]$. We have:

$$x_t(\check{q} + \delta\check{q}) - x_t(\check{q}) = \int_{\check{q} + \delta\check{q}}^{\check{q}} e^{-\frac{h}{2}} \check{z}_t(h) dh \approx -\check{z}_t(\check{q}) \times \delta\check{q} \times e^{-\frac{\check{q}}{2}}, \quad (31a)$$

$$y_t(\check{q} + \delta\check{q}) - y_t(\check{q}) = \int_{\check{q}}^{\check{q} + \delta\check{q}} e^{+\frac{h}{2}} \check{z}_t(h) dh \approx +\check{z}_t(\check{q}) \times \delta\check{q} \times e^{+\frac{\check{q}}{2}}. \quad (31b)$$

Hence, $-\check{z}_t(\check{q}) \times \delta\check{q} \times e^{-\frac{\check{q}}{2}}$ amount of token x and $+\check{z}_t(\check{q}) \times \delta\check{q} \times e^{+\frac{\check{q}}{2}}$ amount of token y are exchanged by crossing $[\check{q}, \check{q} + \delta\check{q}]$. This means that $\check{z}_t(\check{q})$ dictates the volume of trade when crossing $[\check{q}, \check{q} + \delta\check{q}]$, which is why $\check{z}_t$ is considered the liquidity distribution function.

Remark 4. The liquidity amount available for trading is proportional to the value of the liquidity distribution function at each particular price.

Example 2 (Constant product DEX – without fee). Here, the parameterized reserve functions, x_t and y_t , as well as the liquidity distribution function, $\check{z}_t$, are derived for constant product DEXs, e.g., Uniswap v2 [6], in the hypothetical case of no fee. To this end, the following system of differential equations should be solved:

$$x_t(\check{q})y_t(\check{q}) = x_t y_t, \quad (32a)$$

$$\frac{y'_t(\check{q})}{x'_t(\check{q})} = -e^{\check{q}}, \quad (32b)$$

which leads to:¹⁵

$$x_t(\check{q}) = \sqrt{x_t y_t} e^{-\frac{\check{q}}{2}}, \quad (33a)$$

$$y_t(\check{q}) = \sqrt{x_t y_t} e^{+\frac{\check{q}}{2}}, \quad (33b)$$

$$\check{z}_t(\check{q}) = \frac{1}{2} \sqrt{x_t y_t}. \quad (33c)$$

However, this structure does not lead to any profit for the LPs and is not practical from the LPs' incentive perspective.

Example 3 (Constant product DEX – with fee). Here, the parameterized reserve functions x_t and y_t , as well as the liquidity distribution function, $\check{z}_t$, are derived

¹⁵The liquidity in Uniswap v2 is distributed across the entire $(-\infty, +\infty)$ price horizon as there is no concentrated liquidity. With concentrated liquidity, the equations would hold only locally, within the boundaries of the active liquidity interval where the current price of the pool is located.

and illustrated for constant product DEXs, e.g., Uniswap v2, in the presence of a fee. To this end, the following system of differential equations should be solved:¹⁶

$$x_t(\tilde{q})[y_t + (1 - f)(y_t(\tilde{q}) - y_t)] = x_t y_t, \quad \text{if } \tilde{q} \geq \tilde{q}_t, \quad (34a)$$

$$[x_t + (1 - f)(x_t(\tilde{q}) - x_t)]y_t(\tilde{q}) = x_t y_t, \quad \text{if } \tilde{q} \leq \tilde{q}_t, \quad (34b)$$

$$\frac{y'_t(\tilde{q})}{x'_t(\tilde{q})} = -e^{\tilde{q}}, \quad (34c)$$

which leads to:

$$x_t(\tilde{q}) = \begin{cases} \sqrt{\frac{x_t y_t}{1-f}} e^{-\frac{\tilde{q}}{2}} - \frac{f}{1-f} x_t & \tilde{q} \leq \log_e \left(\frac{(1-f)y_t}{x_t} \right) \\ x_t & \text{o.w.} \end{cases}, \quad (35a)$$

$$y_t(\tilde{q}) = \begin{cases} \sqrt{\frac{x_t y_t}{1-f}} e^{+\frac{\tilde{q}}{2}} & \tilde{q} \leq \log_e \left(\frac{(1-f)y_t}{x_t} \right) \\ y_t & \text{o.w.} \end{cases}, \quad (35b)$$

$$\tilde{z}_t(\tilde{q}) = \begin{cases} 0 & \log_e \left(\frac{(1-f)y_t}{x_t} \right) \leq \tilde{q} \leq \log_e \left(\frac{y_t}{(1-f)x_t} \right) \\ \frac{1}{2} \sqrt{\frac{x_t y_t}{1-f}} & \text{o.w.} \end{cases}, \quad (35c)$$

where $f \in (0, 1)$ represents the fee ratio. These three functions as well as the resulting curve are illustrated in Figure 5.

Remark 5. In Example 2, $c(x, y) = \log_e(x) + \log_e(y)$ is strictly concave and differentiable. Since the curves before and after each swap are the same, equations (18)–(19) hold.

In Example 3, equations (34a)–(34b) lead to a piecewise-defined resulting curve that is not differentiable at (x_t, y_t) , so equation (19) does not hold. See Remark 1.

One of the drawbacks of Uniswap v2 is the spread between the buy and sell marginal prices. In other words, selling a token incurs a different marginal price than buying that token. Without compromising LP profitability, a key feature of the NoFeeSwap protocol is to address that by avoiding any price range within which the liquidity distribution function is zero.

¹⁶The case of $\tilde{q} \geq \tilde{q}_t$ maps to $x_t(\tilde{q}) \leq x_t$ and $y_t(\tilde{q}) \geq y_t$. The case of $\tilde{q} \leq \tilde{q}_t$ maps to $x_t(\tilde{q}) \geq x_t$ and $y_t(\tilde{q}) \leq y_t$.

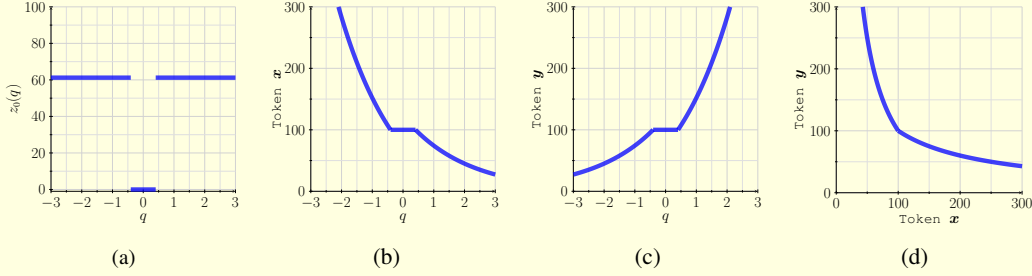


Figure 5: Illustrations corresponding to Example 3 with parameters $x_t = y_t = 100$, and $f = \frac{1}{3}$. Since there is no concentrated liquidity, $\check{q}_{\text{lower}} = -\infty$, $\check{q}_{\text{upper}} = +\infty$. The fee ratio is exaggerated for illustrative purposes. As demonstrated by the figures, no liquidity is devoted to the range $\left[\log_e\left(\frac{(1-f)y_t}{x_t}\right), \log_e\left(\frac{y_t}{(1-f)x_t}\right)\right]$, which results in a marginal spread. (a) Liquidity distribution function, $\check{z}_t$; (b) Parameterized reserve function, x_t ; (c) Parameterized reserve function, y_t ; (d) The resulting curve, c_t .

Remark 6. In order to avoid spread, the liquidity distribution function must not vanish on any nontrivial interval. Consider a price range $(\check{q}_1, \check{q}_2)$ where $\check{q}_1, \check{q}_2 \in [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}]$ and assume that $\check{z}_t(\check{q}) = 0$ for every $\check{q} \in (\check{q}_1, \check{q}_2)$. Hence:

$$x_t(\check{q}_2) - x_t(\check{q}_1) = \int_{\check{q}_2}^{\check{q}_1} e^{-\frac{h}{2}} \check{z}_t(h) dh = 0, \quad (36a)$$

$$y_t(\check{q}_2) - y_t(\check{q}_1) = \int_{\check{q}_1}^{\check{q}_2} e^{+\frac{h}{2}} \check{z}_t(h) dh = 0. \quad (36b)$$

As a consequence, if $\check{q}_t \in (\check{q}_1, \check{q}_2)$:

$$x_t(\check{q}_1) = x_t(\check{q}_t) = x_t(\check{q}_2), \quad (37a)$$

$$y_t(\check{q}_1) = y_t(\check{q}_t) = y_t(\check{q}_2). \quad (37b)$$

This means that for an infinitesimal trade, the trader needs to move the price to either $\check{q}_1 - \delta\check{q}$ or $\check{q}_2 + \delta\check{q}$, where $\delta\check{q} \rightarrow 0^+$. The marginal prices associated with each of these two trades are:

$$\frac{y_t(\check{q}_2 + \delta\check{q}) - y_t(\check{q}_t)}{x_t(\check{q}_t) - x_t(\check{q}_2 + \delta\check{q})} = \frac{y_t(\check{q}_2 + \delta\check{q}) - y_t(\check{q}_2)}{x_t(\check{q}_2) - x_t(\check{q}_2 + \delta\check{q})} \approx \frac{\check{z}_t(\check{q}_2) \times \delta\check{q} \times e^{+\frac{\check{q}_2}{2}}}{\check{z}_t(\check{q}_2) \times \delta\check{q} \times e^{-\frac{\check{q}_2}{2}}} = e^{\check{q}_2}, \quad (38a)$$

$$\frac{y_t(\check{q}_t) - y_t(\check{q}_1 - \delta\check{q})}{x_t(\check{q}_1 - \delta\check{q}) - x_t(\check{q}_t)} = \frac{y_t(\check{q}_1) - y_t(\check{q}_1 - \delta\check{q})}{x_t(\check{q}_1 - \delta\check{q}) - x_t(\check{q}_1)} \approx \frac{\check{z}_t(\check{q}_1) \times \delta\check{q} \times e^{+\frac{\check{q}_1}{2}}}{\check{z}_t(\check{q}_1) \times \delta\check{q} \times e^{-\frac{\check{q}_1}{2}}} = e^{\check{q}_1}. \quad (38b)$$

Hence, the marginal buy price of the pool is $e^{\check{q}_1}$ and the marginal sell price of the pool is $e^{\check{q}_2}$, which incurs spread.

Remark 7. In NoFeeSwap, in order to ensure a high-precision calculation of the integrals in equations (28a)–(28b), all price movements are restricted to $\check{p} \in (e^{-16}p_{\text{offset}}, e^{+16}p_{\text{offset}})$, and the log-price range $\check{q} \in (q_{\text{offset}} - 16, q_{\text{offset}} + 16)$, where $q_{\text{offset}} = \log_e(p_{\text{offset}})$ is the natural logarithm of the offset price established by the creator of the pool. Therefore, instead of the integrals in equations (28a)–(28b), the following are calculated and stored.^{17,18}

$$\begin{aligned} \sqrt{p_{\text{offset}}} x_t(\check{q}) &= \sqrt{p_{\text{offset}}} \int_{\check{q}}^{\check{q}_{\text{upper}}} e^{-\frac{h}{2}} \check{z}_t(h) dh \\ &= \int_{\check{q}-q_{\text{offset}}}^{\check{q}_{\text{upper}}-q_{\text{offset}}} e^{-\frac{h}{2}} \check{z}_t(h + q_{\text{offset}}) dh = \int_q^{q_{\text{upper}}} e^{-\frac{h}{2}} z_t(h) dh, \end{aligned} \quad (39a)$$

$$\begin{aligned} \frac{1}{\sqrt{p_{\text{offset}}}} y_t(\check{q}) &= \frac{1}{\sqrt{p_{\text{offset}}}} \int_{\check{q}_{\text{lower}}}^{\check{q}} e^{+\frac{h}{2}} \check{z}_t(h) dh \\ &= \int_{\check{q}_{\text{lower}}-q_{\text{offset}}}^{\check{q}-q_{\text{offset}}} e^{+\frac{h}{2}} \check{z}_t(h + q_{\text{offset}}) dh = \int_{q_{\text{lower}}}^q e^{+\frac{h}{2}} z_t(h) dh, \end{aligned} \quad (39b)$$

where

$$q_{\text{upper}} \triangleq \check{q}_{\text{upper}} - q_{\text{offset}}, \quad (40a)$$

$$q \triangleq \check{q} - q_{\text{offset}}, \quad (40b)$$

$$q_{\text{lower}} \triangleq \check{q}_{\text{lower}} - q_{\text{offset}}, \quad (40c)$$

$$z_t(\cdot) \triangleq \check{z}_t(\cdot + q_{\text{offset}}), \quad (40d)$$

are the standardized log-price values and the standardized liquidity distribution function.

Remark 8. The price p and log-price q refer to standardized values, and one needs to perform the following transformations to obtain the real-world values:

$$\check{p} = p_{\text{offset}} \times p, \quad (41a)$$

$$\check{q} = q_{\text{offset}} + q, \quad (41b)$$

$$\sqrt{\check{p}} = \sqrt{p_{\text{offset}}} \times \sqrt{p}. \quad (41c)$$

¹⁷It is straightforward to verify these equalities via change of variables.

¹⁸Similar arguments hold for other token reserve equations, e.g., the integrals in Remark 2.

Remark 9. *For numerical reasons, all integrals are stored with a multiplier of $\frac{e^{-8}}{2}$ in the blockchain storage. This guarantees that the stored values of the integrals remain below 1, as shown in equation (58). This multiplier is omitted from the formulations of this document to enhance readability.*

3 NoFeeSwap Protocol

In the NoFeeSwap protocol, there are several aspects to the liquidity distribution function and update algorithms. Section 3.1 explains the structure of the liquidity distribution. Section 3.1.3 explains an algorithm for curve updates after each swap. Sections 3.2 and 3.3 explain the main parameters of the protocol and each liquidity pool. Section 3.4 discusses the calculation of the price and swap amounts, and Section 3.5 provides the calculation of the consequent curve updates and the growth of the LPs. Section 3.6 provides explanations on the components involved in a swap call. Section 3.7 covers Solidity data types. Section 3.8 provides explanations on the protocol's external functions.

3.1 Liquidity Distribution Function

The management of the liquidity distribution is through the introduction and interactions of two functions: 1) **Kernel**, which is a static liquidity distribution applied to each liquidity interval; 2) **Curve Sequence**, which dynamically updates after each swap and is a mechanism for handling the liquidity distribution in the active liquidity interval.

3.1.1 Kernel

Consider a pool with *log-price spacing* equal to $q_{\text{spacing}} = q_{\text{upper}} - q_{\text{lower}} > 0$. Therefore, every concentrated liquidity interval boundary is of the form $q_{\text{lower}} + m q_{\text{spacing}}$, for some $m \in \mathbb{Z}$.

At the time of initialization of a pool, the pool owner provides a piecewise linear monotonically non-decreasing function $k : [0, q_{\text{spacing}}] \rightarrow [0, 1]$ which satisfies:

$$k(0) = 0, \quad k(q_{\text{spacing}}) = 1, \quad (42)$$

and is referred to as the kernel.¹⁹ The role of the kernel is to dictate the liquidity distribution function in all liquidity intervals, including the active liquidity interval

¹⁹Note that breakpoints with equal horizontal coordinates (i.e., $k(q_0) = h_1$ and $k(q_0) = h_2$)

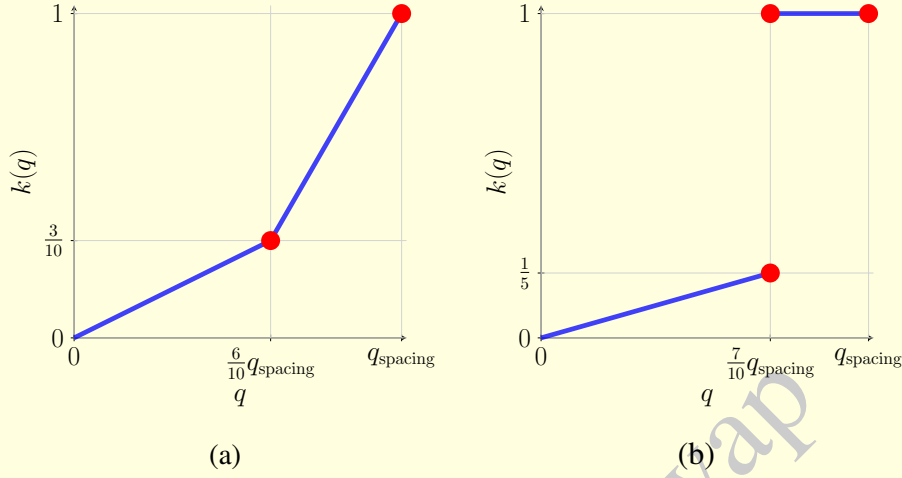


Figure 6: (a) A kernel defined by the set of break-points $\{(0, 0), (0.6q_{\text{spacing}}, 0.3), (q_{\text{spacing}}, 1)\}$; (b) A kernel defined by the set of break-points $\{(0, 0), (0.7q_{\text{spacing}}, 0.2), (0.7q_{\text{spacing}}, 1), (q_{\text{spacing}}, 1)\}$.

that contains the current price of the pool.²⁰ Figure 6 provides simple examples of the kernel function.

The pool owner selects the kernel. The kernel can also be modified by the pool owner if the `isMutableKernel` flag²¹ is set. See explanations of `staticParamsStoragePointer` in Section 3.2.2 for kernel modifications.

Remark 10. *A newly specified kernel stays pending and gets activated upon the earliest transition to a new liquidity interval in that pool. At the transition moment, the pending kernel becomes the pool's kernel, and `pendingKernelLength` becomes zero.*

Consider an arbitrary integer m and the liquidity interval $[q_{\text{lower}} + mq_{\text{spacing}}, q_{\text{upper}} +$

are permitted. In such cases, we have a kernel discontinuity, i.e., $k(q_0^-) = \min\{h_1, h_2\}$ and $k(q_0^+) = \max\{h_1, h_2\}$. The value of the function at a discontinuity is not relevant, and only the integral values in various ranges matter.

²⁰Inactive liquidity intervals include every interval except the active liquidity interval.

²¹Flags are discussed in Section 3.2.1.1.

$mq_{\text{spacing}}]$ ²². Define:

$$\begin{aligned}
\text{outgoingMax}_k &\triangleq e^{+\frac{q_{\text{lower}}+mq_{\text{spacing}}}{2}} \int_{q_{\text{lower}}+mq_{\text{spacing}}}^{q_{\text{upper}}+mq_{\text{spacing}}} e^{-\frac{h}{2}} k(h - q_{\text{lower}} - mq_{\text{spacing}}) dh \\
&= e^{-\frac{q_{\text{upper}}+mq_{\text{spacing}}}{2}} \int_{q_{\text{lower}}+mq_{\text{spacing}}}^{q_{\text{upper}}+mq_{\text{spacing}}} e^{+\frac{h}{2}} k(q_{\text{upper}} + mq_{\text{spacing}} - h) dh \\
&= \int_0^{q_{\text{spacing}}} e^{-\frac{h}{2}} k(h) dh.
\end{aligned} \tag{43}$$

The value of outgoingMax_k is a constant associated with the kernel and is independent of the integer m , i.e., the third integral has no m and the first two integrals are invariant to m .²³ This value serves as a normalizer for kernel functions.

3.1.1.1 Inactive liquidity intervals on the right side. The standardized liquidity distribution function over the liquidity interval $q \in [q_{\text{lower}} + mq_{\text{spacing}}, q_{\text{upper}} + mq_{\text{spacing}}]$, for any integer $m > 0$, is equal to:

$$z(q) = \frac{\text{sharesTotal} \times \text{growth}}{\text{outgoingMax}_k} k(q - q_{\text{lower}} - mq_{\text{spacing}}), \tag{44}$$

where sharesTotal is a non-negative integer denoting the total number of LP shares in the corresponding liquidity interval and $\text{growth} \geq 1$ denotes the corresponding liquidity interval's cumulative liquidity growth.²⁴

This liquidity interval is inactive, and the pool price is on its left side and hence

²²which is the same as $[q_{\text{lower}} + mq_{\text{spacing}}, q_{\text{lower}} + (m+1)q_{\text{spacing}}]$

²³There are exponential terms, $e^{+\frac{q_{\text{lower}}+mq_{\text{spacing}}}{2}}$ and $e^{-\frac{q_{\text{upper}}+mq_{\text{spacing}}}{2}}$, to make the first two integrals invariant for all liquidity intervals.

²⁴The protocol keeps track of the share count and the growth value for the active liquidity interval. See growth and sharesTotal in Section 3.2.2. To retrieve such values for other liquidity intervals, the mapping parameters are introduced. See Section 3.3.

the reserve of `token1` is zero²⁵²⁶, i.e.,

$$\text{token1reserve} = 0$$

$$\text{token0reserve}$$

$$\begin{aligned} &= \frac{1}{\sqrt{p_{\text{offset}}}} \int_{q_{\text{lower}} + mq_{\text{spacing}}}^{q_{\text{upper}} + mq_{\text{spacing}}} e^{-\frac{h}{2}} z(h) dh \\ &= \frac{1}{\sqrt{p_{\text{offset}}}} \int_{q_{\text{lower}} + mq_{\text{spacing}}}^{q_{\text{upper}} + mq_{\text{spacing}}} e^{-\frac{h}{2}} \frac{\text{sharesTotal} \times \text{growth}}{\text{outgoingMax}_k} k(h - q_{\text{lower}} - mq_{\text{spacing}}) dh \\ &= \frac{1}{\sqrt{p_{\text{offset}}}} \text{sharesTotal} \times \text{growth} \times e^{-\frac{q_{\text{lower}} + mq_{\text{spacing}}}{2}}. \end{aligned} \quad (45)$$

3.1.1.2 Inactive liquidity intervals on the left side. The standardized liquidity distribution function over the liquidity interval $q \in [q_{\text{lower}} - mq_{\text{spacing}}, q_{\text{upper}} - mq_{\text{spacing}}]$, for any integer $m > 0$, is equal to:

$$z(q) = \frac{\text{sharesTotal} \times \text{growth}}{\text{outgoingMax}_k} k(q_{\text{upper}} - mq_{\text{spacing}} - q), \quad (46)$$

whose shape is a mirror reflection of equation (44). This liquidity interval is inactive, and the pool price is on its right side, hence:

$$\text{token0reserve} = 0$$

$$\text{token1reserve}$$

$$\begin{aligned} &= \sqrt{p_{\text{offset}}} \int_{q_{\text{lower}} - mq_{\text{spacing}}}^{q_{\text{upper}} - mq_{\text{spacing}}} e^{+\frac{h}{2}} z(h) dh \\ &= \sqrt{p_{\text{offset}}} \int_{q_{\text{lower}} - mq_{\text{spacing}}}^{q_{\text{upper}} - mq_{\text{spacing}}} e^{+\frac{h}{2}} \frac{\text{sharesTotal} \times \text{growth}}{\text{outgoingMax}_k} k(q_{\text{upper}} - mq_{\text{spacing}} - h) dh \\ &= \sqrt{p_{\text{offset}}} \text{sharesTotal} \times \text{growth} \times e^{+\frac{q_{\text{upper}} - mq_{\text{spacing}}}{2}}. \end{aligned} \quad (47)$$

The liquidity distribution function and the token amounts for the *active* liquidity interval are discussed next.

²⁵In this document, occasionally, `token0` and `token x` are used interchangeably; `token1` and `token y` are used interchangeably.

²⁶See equation (39).

3.1.2 Curve Sequence

In order to keep track of the liquidity distribution function within the active liquidity interval, before the t^{th} swap, a finite sequence of log-price values is considered:²⁷

$$\bar{q}_t \triangleq q_t^0, q_t^1, q_t^2, \dots, q_t^{l-1}, \quad (48)$$

such that:

$$q_{\text{lower}} = \min\{q_t^0, q_t^1\}, \quad (49)$$

$$q_{\text{upper}} = \max\{q_t^0, q_t^1\}, \quad (50)$$

$$q_{\text{current}} = q_t^{l-1}, \quad (51)$$

and for each $i \in \{2, \dots, l-1\}$,

$$\min\{q_t^{i-1}, q_t^{i-2}\} < q_t^i < \max\{q_t^{i-1}, q_t^{i-2}\}. \quad (52)$$

This is referred to as the *curve sequence*.²⁸ The liquidity distribution function for the active liquidity interval is equal to:

$$z_t(q) \triangleq \frac{\text{sharesTotal} \times \text{growth}_t}{\text{outgoingMax}_k} k(w_{\bar{q}_t}(q)), \quad q_{\text{lower}} \leq q \leq q_{\text{upper}}, \quad (53)$$

where:

$$w_{\bar{q}_t}(q) \triangleq \sum_{i=1}^{l-1} (q - q_t^i) (\mathcal{I}\{q_t^{i+1} < q < q_t^{i-1}\} - \mathcal{I}\{q_t^{i-1} < q < q_t^{i+1}\}), \quad (54)$$

and the out-of-range value q_t^l is equal to q_t^{l-1} . The indicator function $\mathcal{I}$ returns 1 if its argument is true and returns 0, otherwise. See Figure 7.

Remark 11. *To facilitate coordinating integrations, each piece of the function $w(q)$ is characterized using three members of the curve sequence, the two ends and the intersection with the horizontal axis. $w(q)$ is defined in such a way that each of these points is a member of the curve sequence.*

²⁷The occasional use of time-indexing is to emphasize the evolution with swaps and to formulate the impact of swaps. For example, z_t , growth_t , and $\bar{q}_t$ change with each swap and are functions of swaps until before t^{th} . However, swaps do not necessarily change the kernel, the number of shares, and outgoingMax_k .

²⁸These equations are for instances when $q_{\text{lower}} < q_{\text{current}} < q_{\text{upper}}$. When $q_{\text{current}} = q_{\text{lower}}$, the curve sequence is $\bar{q}_t = q_{\text{upper}}, q_{\text{lower}}$, and when $q_{\text{current}} = q_{\text{upper}}$, the curve sequence is $\bar{q}_t = q_{\text{lower}}, q_{\text{upper}}$.

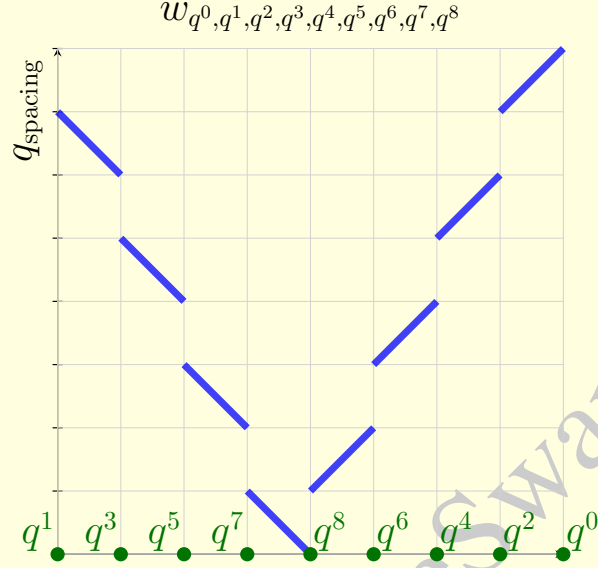


Figure 7: A sample $w_{\bar{q}_t}(q)$ with a curve sequence of size 9.

Figure 8(a) shows $w_{\bar{q}_t}$ for the default curve sequence when the pool price is on the left side of this liquidity interval. Figure 8(b) shows $w_{\bar{q}_t}$ for the example of $\bar{q}_t = q_{\text{lower}} + q_{\text{spacing}}, q_{\text{lower}}, q_{\text{lower}} + \frac{2}{3}q_{\text{spacing}}, q_{\text{lower}} + \frac{1}{3}q_{\text{spacing}}$. Updates of the curve sequence and $w_{\bar{q}_t}$ with swaps are discussed in Section 3.1.3.

After each swap, the protocol calculates a composition of $w_{\bar{q}_t}(\cdot)$ and the kernel function, $k(\cdot)$, to determine the liquidity distribution function of the active liquidity interval according to equation (53). The token exchange amounts corresponding to each price movement can then be determined by integrating over $z_t(q)$ throughout the appropriate liquidity intervals, as in Remark 2²⁹.

When a pool price moves inside a liquidity interval as a consequence of swaps, the curve sequence for that liquidity interval dynamically updates/changes. If the kernel is as in Figure 6(b) and the curve sequence is such that $w(q)$ is as in Figure 8(b), then, $k(w(q))$ can be calculated for every $h \in [q_{\text{lower}}, q_{\text{upper}}]$ as in Figure 9.³⁰

For inactive liquidity intervals, $w(q)$ is always in the form of a shifted identity function, or mirrored. Additionally, the liquidity distribution function in inactive liquidity intervals is proportional to a shifted kernel function, or mirrored. Note

²⁹with consideration of the offset as in Remark 7

³⁰ $z_t(q)$ can be derived using equation (53).

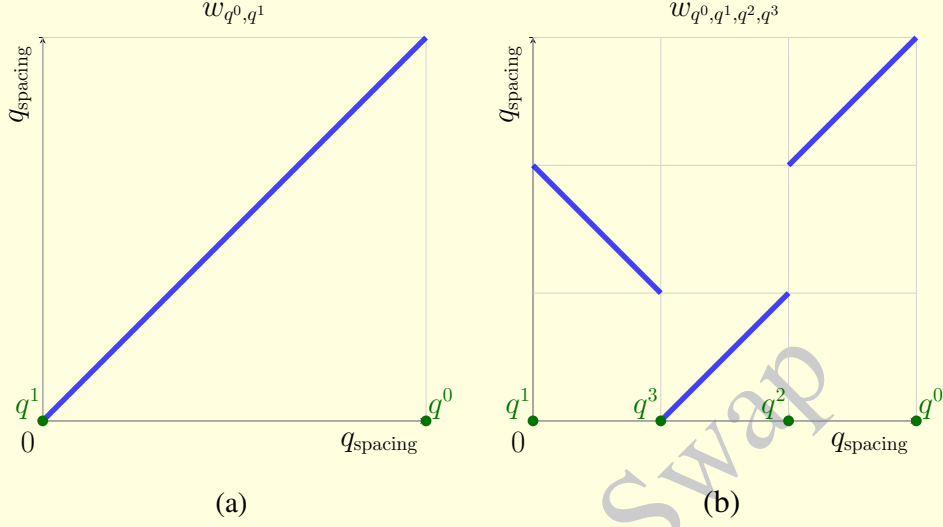


Figure 8: (a) $w(q)$ and the default curve sequence for an inactive liquidity interval when the pool price is on the left side of it; (b) A sample snapshot of $w(q)$ in the active liquidity interval after some swaps.

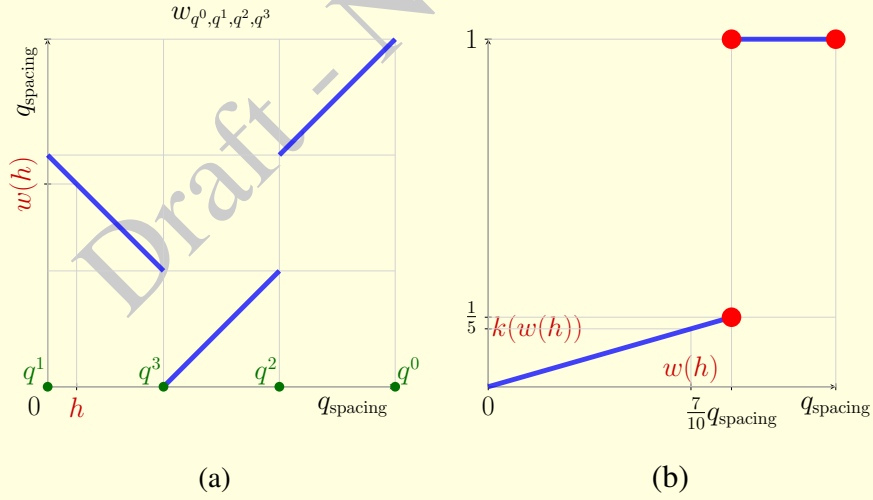


Figure 9: On the left side, $w(h)$ can be calculated for every h in the range $[q_{\text{lower}}, q_{\text{upper}}]$. Then, on the right side, $w(h)$ is given to the kernel function as input, and $k(w(h))$ is calculated. This illustrates the composition of the curve sequence in Figure 8(b) and the kernel in Figure 6(b), which determines $k(w(h))$.

that the scalar multiplier in the liquidity distribution function, $\frac{\text{sharesTotal} \times \text{growth}}{\text{outgoingMax}_k}$, is different for each liquidity interval (and is zero for intervals with zero liquidity). As depicted in Figure 10, for the liquidity intervals that are on the left side of the current pool price, the liquidity distribution is effectively mirrored.

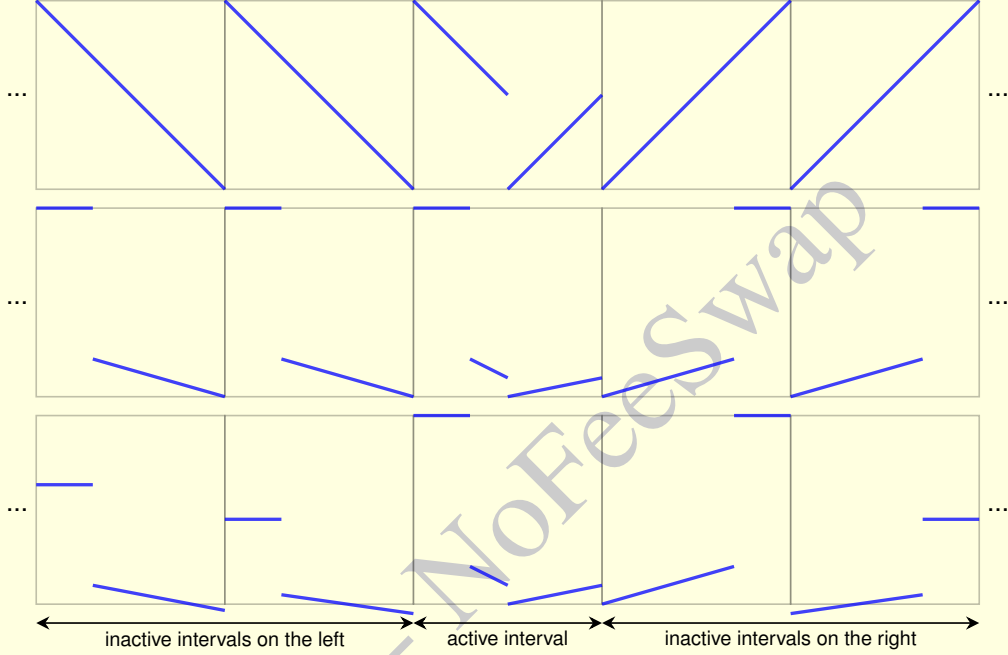


Figure 10: (a) $w(q)$ [top]; (b) $k(w(q))$ [middle]; (c) $z(q)$ [bottom].

A major component of the NoFeeSwap protocol is how the curve sequence is updated as a result of a swap. This is discussed next.

3.1.3 Curve Sequence Updates after Swaps

Figure 8(b) shows a sample snapshot of $w_{\bar{q}_t}(q)$ for the active liquidity interval. Observe that $q_t^0, q_t^1, q_t^2, \dots, q_t^{\ell-1}$ (a curve sequence of size ℓ) approaches the pool price, alternating from left and right, with decreasing heights as in equation (54) and Figure 7. The function $w_{\bar{q}_t}(q)$ is a rearrangement of the identity function in which every other piece is mirrored. See Figure 11.

The pool price changes through swaps. If the pool price enters a liquidity interval from the left (q_{lower}), the first two members of the curve sequence are

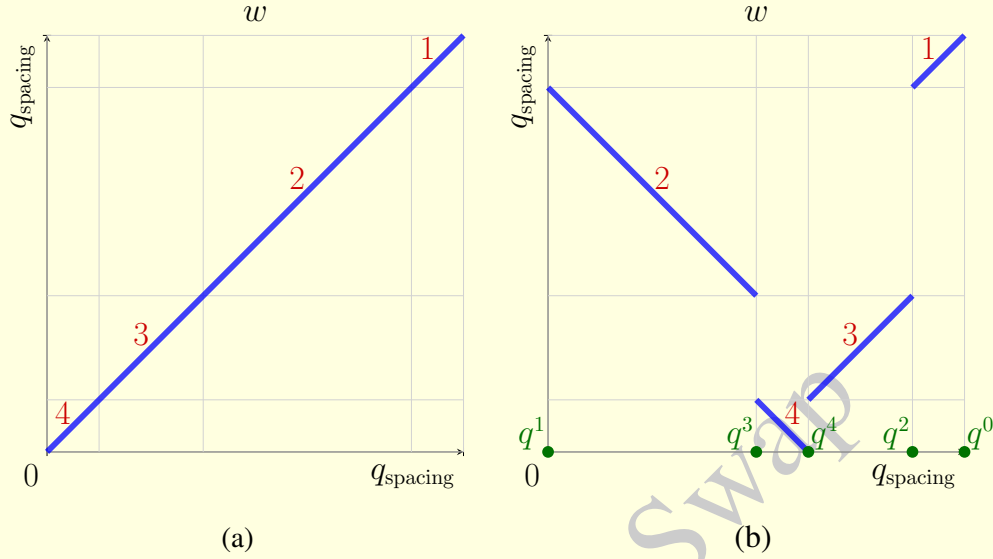


Figure 11: (a) An illustration of $w(q)$ for an inactive liquidity interval. This shape holds when the pool price is on the left side of this liquidity interval. The pieces to be moved (through swaps) are specified; (b) An updated $w(q)$ (shifted and every other one is mirrored). Note that all pieces preserve their height after updates.

$q^0 = q_{\text{upper}}$ and $q^1 = q_{\text{lower}}$. These two and their order remain unchanged as long as the pool price has not reached the other boundary (q_{upper}). Similarly, if the pool price enters a liquidity interval from the right (q_{upper}), the first two members of the curve sequence are $q^0 = q_{\text{lower}}$ and $q^1 = q_{\text{upper}}$.

After each swap, the curve sequence is updated/amended in preparation for the next swap. An update of the curve sequence can change the number of its members, e.g., it can convert Figure 8(b) to any instance shown in Figure 12. Consider a log-price $q_{\text{new}} \in (q_{\text{lower}}, q_{\text{upper}})$ to be added to a curve sequence of size ℓ . If q_{new} is between $q_t^{\ell-2}$ and $q_t^{\ell-1}$ (the last two members of the curve sequence), then q_{new} is appended to the curve sequence, as in the first case in Figure 12. Otherwise, let ℓ' denote the largest integer such that q_{new} is between $q_t^{\ell'-2}$ and $q_t^{\ell'-1}$ ($2 \leq \ell' \leq \ell - 1$). In this case, the curve sequence is transformed to $q_t^0, q_t^1, q_t^2, \dots, q_t^{\ell'-1}, q_{\text{new}}$ (a curve sequence of size $\ell' + 1$), as in the second and third cases in Figure 12. The fourth and fifth cases in Figure 12 show instances in which the pool price is moved to the left and right boundaries of the liquidity interval, respectively³¹.

³¹and possibly the pool price leaves the liquidity interval after reaching a boundary, through an

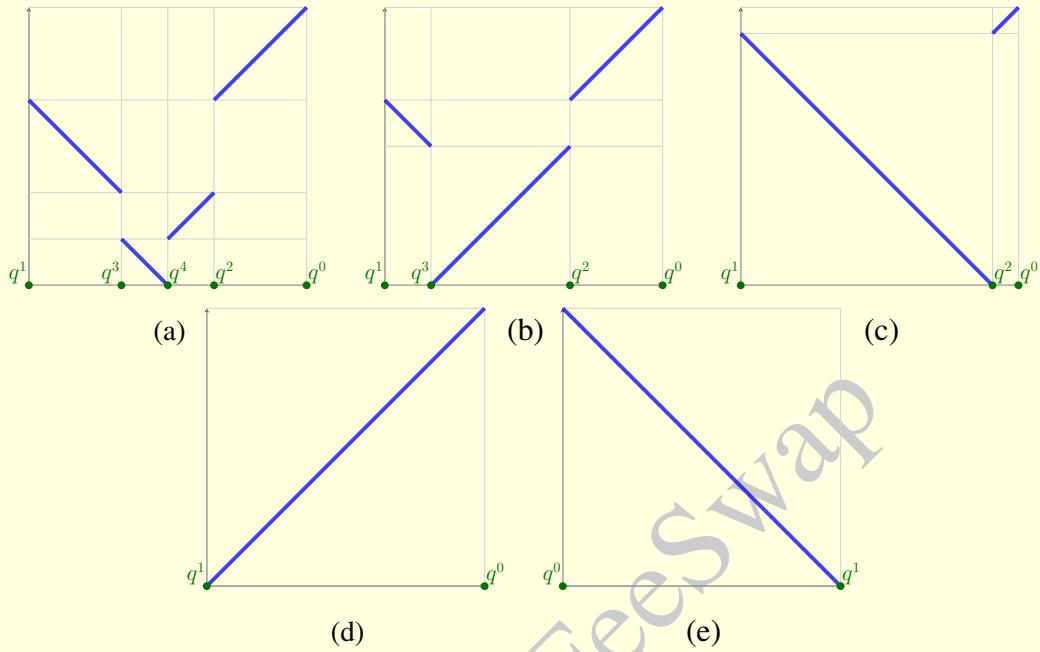


Figure 12: Possible updated $w(q)$ shapes from Figure 8(b) with an amendment.

Remark 12. Section 3.5 explains that after each swap, the curve sequence may undergo up to two updates and details how the amended log-price for each curve update is determined.³²

Remark 13. Members of the curve sequence can be viewed as historical peaks of the price, with the latest members representing the most recent ones, i.e., the curve sequence can be interpreted as the protocol's price memory in the active liquidity interval. By keeping track of the pool price fluctuations through the curve sequence updates, the protocol applies more slippage for moving towards the most recently observed pool prices. This can result in higher profit for LPs at the cost of lower profit for arbitrageurs, as arbitrageurs tend to move the pool price back towards the market price after other traders' transactions.

Furthermore, Section 3.5 demonstrates that these updates to the curve sequence result in liquidity growth and profit for LPs.

interval transition

³²Remark 46 discusses the possibility of having only one such update after certain swaps.

Remark 14. *The curve sequence enables efficient storage of historical prices on the blockchain that comes with various advantages.*

- A short stack: *The concept of a curve sequence is an innovative and efficient method of keeping track of historical price movements using a short stack. A curve sequence is amended/updated efficiently after each swap by trimming and appending new members to the end through the algorithm discussed in this section.*
- Pool prices in chronological order: *The curve sequence members are the recorded pool prices in chronological order. The value of every member of the curve sequence is in the range between the value of the previous two members. The first two members of the curve sequence are the boundaries of the active liquidity interval. The last member is the most recently appended value.*
- Similarity to order book charts: *The shape of $w(q)$ preserves the desired feature of increasing liquidity as the pool price moves away from the current price toward the boundaries of the liquidity interval.*
- Historical trading data: *It can facilitate:*
 - For LPs: *Higher profits by applying different slippage on the offered liquidity in two directions based on recent trading activity.*
 - For traders: *Maintaining recent trading activity can facilitate strategic trading decisions.*

3.2 Protocol Parameters [7]

There are various pool-level parameters that are accessed as part of every swap call. These parameters are divided into two categories: i) Dynamic parameters that are written in singleton's storage, ii) Static parameters that are stored as the bytecode of a so-called "storage contract", and can be accessed via the opcode `extcodecopy`. In the following, we explain the aforementioned ones, as well as a few protocol-level parameters:

Section 3.2.1 explains the identification parameters of each liquidity pool. Sections 3.2.2 and 3.2.3 provide the dynamic and static parameters, respectively.³³

³³These are variables that are tracked in storage and are required for every swap call.

Section 3.2.4 presents the parameters related to the splitting of liquidity growth for pools. Finally, Section 3.2.5 briefly discusses the global protocol slot.

3.2.1 poolId

`poolId` is the pool's identifier, a unique and *immutable* 256-bit identifier for each pool that is set at the time of pool initialization. Figure 13 shows its structure.

- *Hook Address:* The least significant 160 bits refer to the hook address.
 - If a hook is intended, these 160 bits should refer to the hook contract's address. In this case, at least one hook flag³⁴ must be set.
 - If no hook is intended, these 160 bits must be zero. All of the hook flags must be zero, as well.

Remark 15. *Each pool can select up to one hook contract at the initialization (and use zeros as hook address if no hook is desired). But, each hook address can be used by multiple pools.*

- *Flags:* The next 20 bits are referred to as flags. Each of these flags is discussed in this section.

Remark 16. *Of the 20 flags, the least significant 17 flags are for hooks. The 3 most significant ones are non-hook flags, i.e., `isMutableKernel`, `isMutablePoolGrowthPortion` and `isDonateAllowed`.*

- *Log of price offset:* The next 8 bits represent the natural logarithm of the price offset which must be an integer in the `int8` representation in the range $[-89, +89]$. This covers the range $[e^{-89}, e^{+89}]$ for the price offset. The real-world price of a pool can be within the range $(e^{-16}p_{\text{offset}}, e^{+16}p_{\text{offset}})$, as in Remark 7.

Remark 17. *A candidate for the logarithmic offset value is the natural logarithm of the intended real-world pool price³⁵ at initialization. This choice allows the pool to be able to cover real-world prices within $e^{\pm 16}$ of the initial pool price. Alternatively, one may simply input 0 as the logarithm of the offset, in which case, the range of the pool's real-world price is (e^{-16}, e^{+16}) .*

³⁴See Remark 16.

³⁵The real-world pool price is typically around the market price. See equation (11).

- *Salt*: The remaining (most significant) 68 bits can be chosen arbitrarily by the pool creator. Since the `saltedPoolId` is a deterministic function of these 68 bits, they can be brute-forced in order to find any desirable pattern for the most significant 68 bits of `poolId`. The precise formula for this will be given later in this document.

3.2.1.1 Flags. Every time a hook is invoked, a snapshot of memory is handed to the hook via `calldata`. This way, the hook contract can access the parameters in the protocol’s memory within its `calldata`. This includes any input to the call to the protocol.

In the following, each flag is explained. The least significant 17 flags are the permissions/accesses that the pool creator has determined³⁶ to provide to the hook. Flags, starting from the least significant bit, are as follows:

1. `isPreInitialize`. If this flag is active, the hook is called before the pool initialization, with access to input data.³⁷
2. `isPostInitialize`. If this flag is active, the hook is called after the pool initialization, with access to all of the parameters that are used to construct the pool.

Remark 18. *If there is a hook, at least one of the `isPreInitialize` or `isPostInitialize` flags is enforced to be active.*

3. `isPreMint`. If this flag is active, the hook is called before minting positions, with access to input data.
4. `isMidMint`. If this flag is active, the hook is called after the pre-hook and after the pool is locked, but before the mint state mutation, with access to all of the parameters. However, the hook cannot interact with the pool, due to a reentrancy lock (see Figure 14).
5. `isPostMint`. If this flag is active, the hook is called after minting positions, with access to all of the parameters.
6. `isPreBurn`. If this flag is active, the hook is called before burning positions, with access to input data.

³⁶during the pool initialization

³⁷However, for all “pre” flags, the pool parameters are not passed to the hook via `calldata` and can only be accessed externally.

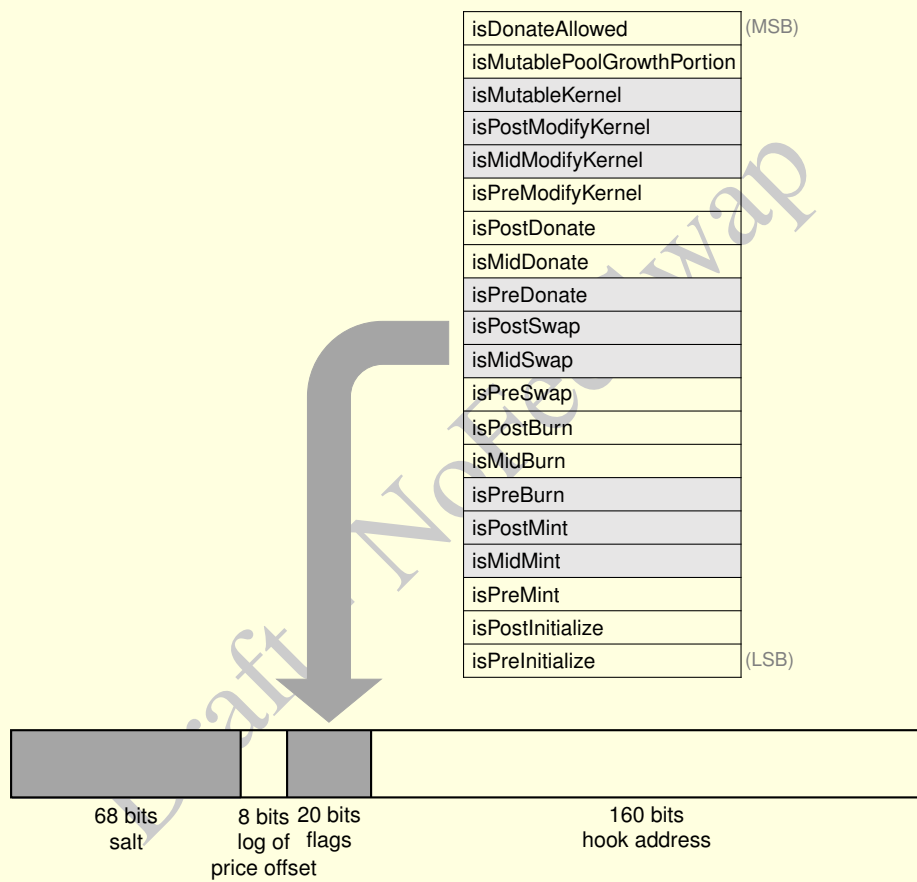


Figure 13: The 256-bit structure of poolId.

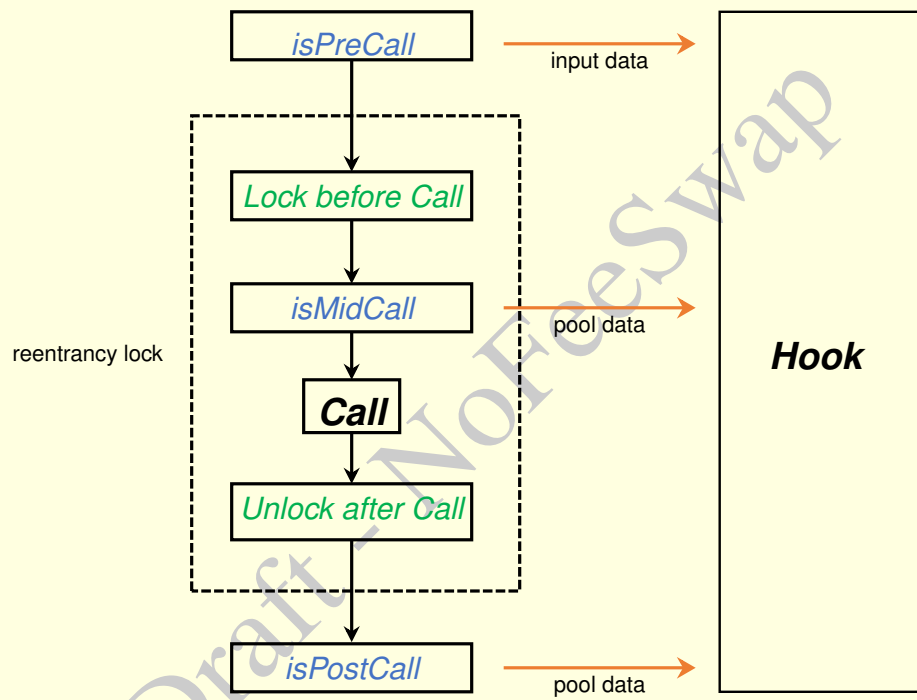


Figure 14: The placement of hooks for mint/burn/swap/donate/modifyKernel calls.

7. `isMidBurn`. If this flag is active, the hook is called after the pre-hook and after the pool is locked, but before the burn state mutation, with access to all of the parameters. However, the hook cannot interact with the pool, due to a reentrancy lock (see Figure 14).
8. `isPostBurn`. If this flag is active, the hook is called after burning positions, with access to all of the parameters.
9. `isPreSwap`. If this flag is active, the hook is called before swaps, with access to input data.
10. `isMidSwap`. If this flag is active, the hook is called after the pre-hook and after the pool is locked, but before the swap state mutation, with access to all of the parameters. However, the hook cannot interact with the pool, due to a reentrancy lock (see Figure 14).
11. `isPostSwap`. If this flag is active, the hook is called after swaps, with access to all of the parameters.
12. `isPreDonate`. If this flag is active, the hook is called before donating, with access to input data.
13. `isMidDonate`. If this flag is active, the hook is called after the pre-hook and after the pool is locked, but before the donate state mutation, with access to all of the parameters. However, the hook cannot interact with the pool, due to a reentrancy lock (see Figure 14).
14. `isPostDonate`. If this flag is active, the hook is called after donating, with access to all of the parameters.
15. `isPreModifyKernel`. If this flag is active, the hook is called before a pending kernel is selected, with access to input data.
16. `isMidModifyKernel`. If this flag is active, the hook is called after the pre-hook and after the pool is locked, but before the pending kernel state mutation, with access to all of the parameters. However, the hook cannot interact with the pool, due to a reentrancy lock (see Figure 14).
17. `isPostModifyKernel`. If this flag is active, the hook is called after a pending kernel is selected, with access to all of the parameters.

18. `isMutableKernel`. If this flag is active, pool's kernel is modifiable by the pool owner. If this flag is inactive, `isPreModifyKernel`, `isMidModifyKernel` and `isPostModifyKernel` flags should be inactive as well.
19. `isMutablePoolGrowthPortion`. If active, `poolGrowthPortion` is modifiable by the pool owner. Otherwise, it stays as the amount that is picked at the time of pool initialization.
20. `isDonateAllowed`. If this flag is active, donate to the active liquidity interval is permitted.

Example 4. Assume that `poolId` of a pool is selected as:

0x00000009000000000020A0005
2260FAC5E5542A773AA44FBCFEDF7C193BC2C599.

in hexadecimal representation.

- The least significant 40 hexadecimal digits³⁸ (blue) represent the address of the hook. So, the hook address is:

0x2260FAC5E5542A773AA44FBCFEDF7C193BC2C599.

- The green digits (0xA0005) indicate the flags with the binary form:

0b10100000000000000101.

In this case, `isPreInitialize`, `isPreMint`, `isMutableKernel` and `isDonateAllowed` flags are activated. `isPreInitialize` and `isPreMint` hook flags are activated, kernel can be modified by the pool owner in the future, and that any address is able to donate to the pool's active liquidity interval.

- The red digits (0x20) indicate an `int8` representation³⁹ of the offset's natural logarithm with the binary representation of:

0b00100000,

³⁸Each hexadecimal digit is 4 bits.

³⁹The natural logarithm of price offset must be an integer in the range $[-89, +89]$.

which correspond to $q_{\text{offset}} = 32$, and, therefore, $p_{\text{offset}} = e^{32} \sim 7.9 \times 10^{13}$. Alternatively, if the `int8` representation value is `0xF2`, the binary form becomes:

0b11110010,

and the corresponding values are $q_{\text{offset}} = -14$ and $p_{\text{offset}} = e^{-14} \sim 8.3 \times 10^{-7}$.

- The salt bits are `0x000000090000000000`. This is explained further in the next section.

3.2.1.2 Salt. The most significant 68 bits of the `unsaltedPoolId` do not impact any of the pool’s attributes. `unsaltedPoolId` is an `uint256` input parameter of the initialization method of `NoFeeswapDelegatee.sol`. The `poolId` is derived from `unsaltedPoolId` and `msg.sender` via the following deterministic formula:

$$\text{poolId} = \text{unsaltedPoolId} + (\text{keccak256}(\text{abi.encodePacked}(\text{msg.sender}, \text{unsaltedPoolId}))) \ll 188, \quad (55)$$

where ‘+’ denotes addition modulo 2^{256} and the ‘ $\ll$ ’ is equivalent to the EVM opcode `shl`. This way, `poolId` can be determined/mined a priori.

After initialization, `poolId` is used as the sole identifying parameter of every pool for all purposes.

3.2.2 Dynamic Parameters

These parameters are subject to frequent updates with `swap` calls. In the following, the usage and update process of each is explained.

- `growth`. (128-bit, type `X111`)⁴⁰ If a swap call touches an interval, then the amount of liquidity in that interval grows. In other words, a single share in that interval becomes more valuable after the swap. The variable `growth` is the cumulative growth for the active liquidity interval and its value is greater than or equal to 1 (corresponding to 2^{111} in storage). The highest possible

⁴⁰As in Section 3.7, `X#` types are defined as 256-bit values, however, the parameters here are maintained in the most packed form for efficiency purposes. Only the lowest number of bits required to maintain the values is recorded in storage.

growth value is 2^{16} (corresponding to 2^{11+16} in storage). At the time of pool initialization, this value is set to 1 and is non-decreasing over time. The steps of updating `growth` with each swap are discussed in Section 3.5.

Remark 19. *The type $X\#$ means that the original number is multiplied by $2^\#$ in storage and stored as a fixed-point value with $\#$ binary fractional places, i.e., the binary representation is shifted to the left by $\#$ digits. See Section 3.7 for more information on data types.*

Remark 20. *Section 3.3.2 and Figure 18 illustrate how the growth for the liquidity intervals adjacent to the active interval is retrieved, which is used during interval transitions.*

Remark 21. *The mechanism of `donate` gives the deposited amount to the LPs in the active liquidity interval and increases the growth parameter accordingly, such that the deposited amount is effectively divided between the LPs of the active interval proportional to their share counts, without changing the number of shares that each LP owns.*

Remark 22. *The growth in the active liquidity interval increases by `donate` and `swap` calls. A portion of this growth goes to the protocol, and a portion of the remaining growth goes to the pool owner. The parameters in Section 3.2.4 are used to track the growth portions of the protocol and the pool owners.*

- `integral0/1`. (216-bit each, type $X216$) These are used for efficient token amount calculations. These values are updated in the storage once after each swap. These integrals are defined as:

$$\text{integral0} \triangleq \int_{q_{\text{current}}}^{q_{\text{upper}}} e^{-\frac{h}{2}} k(w_{\bar{q}_t}(h)) dh, \quad (56a)$$

$$\text{integral1} \triangleq \int_{q_{\text{lower}}}^{q_{\text{current}}} e^{+\frac{h}{2}} k(w_{\bar{q}_t}(h)) dh. \quad (56b)$$

Considering equations (28) and (53) and Remark 7, the LPs' total reserve amounts in the active liquidity interval can be calculated for both tokens as:

$$\text{interval reserve 0} = \frac{1}{\sqrt{p_{\text{offset}}}} \times \frac{\text{sharesTotal} \times \text{growth}_t}{\text{outgoingMax}_k} \times \text{integral0}, \quad (57a)$$

$$\text{interval reserve 1} = \sqrt{p_{\text{offset}}} \times \frac{\text{sharesTotal} \times \text{growth}_t}{\text{outgoingMax}_k} \times \text{integral1}. \quad (57b)$$

Remark 23. A multiplier of $\frac{e^{-8}}{2}$ scales the integrals in equations (56a)–(56b) to stay below 1.

$$\frac{1}{2}e^{-8} \int_{q_{\text{lower}}}^{q_{\text{upper}}} e^{\pm \frac{h}{2}} k(w_{\tilde{q}_t}(h)) dh < \frac{1}{2}e^{-8} \int_{-16}^{16} e^{\frac{h}{2}} dh < 1. \quad (58)$$

Remark 9 explains that this document omits such scaling.

Keeping track of these integrals and updating them in the contract's storage with each swap is more gas-efficient than a full calculation of them with each swap. The stored, scaled versions of these integrals are of type `X216` and since they are between 0 and 1, they are represented with exactly 216 bits.

- `sharesTotal`. (128-bit, type `uint256`) It is the total number of shares in the active liquidity interval. It can change only with `modifyPosition` (as shares are added or removed) and `transition` to an adjacent liquidity interval (as the number of shares in another liquidity interval needs to be specified). Section 3.3.1 and Figure 17 explain how the number of shares is retrieved or updated for inactive liquidity intervals.

Equation (57) shows the relations between `sharesTotal` and the reserve amounts in the active liquidity interval.

- `staticParamsStoragePointer`. (16-bit, type `uint16`) It is a pointer to the static parameters of the pool.⁴¹ Each time the static parameters of a pool change, the protocol stores those parameters using `SSTORE`⁴². This pointer is automatically updated to keep track of the latest version of the data.

⁴¹See Section 3.2.3.

⁴²Inspired by “Philology” and “Solady”

Every time the `modifyKernel` function introduces a pending kernel, or an interval transition causes a pending kernel to be activated, or when any of the growth parameters `poolGrowthPortion`, `maxPoolGrowthPortion` or `protocolGrowthPortion` are updated, this reference pointer is updated as well.⁴³

To access the latest version of the static parameters, a 160-bit address of the storage contract is needed. The address is calculated using this pointer, which is only 16 bits (2 bytes) and fits in among the dynamic parameters.

Remark 24. *The protocol’s mechanism for this update is internal to the system and can be skipped for most readers: Each time the static parameters are updated, a proxy contract is deployed with `create2`. The proxy’s address, `proxyAddress`, is calculated by concatenating and hashing the following values, into 20 bytes (160 bits):*

```
0xff (1 byte)
NofeeswapAddress (20 bytes)
salt (32 bytes)
PROXY_CREATION_HASH (32 bytes)
```

where `salt` is calculated by concatenating and hashing:

```
staticParamsStoragePointer (2 bytes)
poolId (32 bytes)
```

The proxy’s sole purpose is a one-time deployment of a second contract in which static parameters are encoded. The address of this contract is calculated by concatenating and hashing the following values, into 20 bytes (160 bits):

```
0xd69401 (6 bytes)
proxyAddress (20 bytes)
```

⁴³See the list of static parameters in Section 3.2.3. `maxPoolGrowthPortion` and `protocolGrowthPortion` are the pool’s local versions that can be updated by anyone to sync with the Sentinel contract (which is modifiable through governance). `poolGrowthPortion` is modifiable only by the pool owner. Each time an interval transition occurs and a pending kernel is activated, `kernel`, `pendingKernelLength`, `outgoingMax`, `outgoingMaxModularInverse` and `incomingMax` are updated. Tags, price offset, and price spacing are immutable and cannot be modified after pool initialization.

Given `staticParamsStoragePointer`, which is only 16 bits, one can deterministically calculate the storage contract's address and retrieve static parameters via `extcodecopy`.

- `logPriceCurrent`. (64-bit, type X59) It is the latest log-price of the pool, and therefore, the last member of the curve sequence.

Remark 25. *`sharesTotal`, `staticParamsStoragePointer` and `logPriceCurrent` are placed next to each other in storage, within the third slot of dynamic parameters. Therefore, they can be read on-chain efficiently by reading a single slot of storage.*

- `staticParamsStoragePointerExtension`. (256-bit, type `uint256`) It is a pointer to the pool's static parameters. This is an internal component to facilitate updating the static parameters in the uncommon cases when `staticParamsStoragePointer` updates reaches its maximum value $2^{16} - 1$. More precisely:
 - If `staticParamsStoragePointer` $< 2^{16}$, this pointer's value effectively is `staticParamsStoragePointer`.
 - If `staticParamsStoragePointer` $= 2^{16}$, the pointer occupies more than 16 bits, for which the singleton's storage slot

$$\text{getDynamicParamsSlot}(\text{poolId}) - 1 \quad (59)$$

is used.

- `curve`. It is a sequence of 64-bit values of type X59. Each 256-bit slot fits 4 members of the curve sequence that are log-prices of type X59 in offset-binary representation, i.e., 0 corresponds to the real-world price of $p_{\text{offset}} \times e^{-16}$ and `0x8000000000000000` corresponds to the real-world price of p_{offset} . See Table 1 for more details. `curve` sequence should have at least two log-price members. The first and second 64-bit members are the boundaries of the active liquidity interval. The last 64-bit member of the `curve` sequence is the current log-price of the pool, i.e., `logPriceCurrent`. The ordering rules discussed in Section 3.1.2 are verified upon the initialization of a pool and are preserved by each update of the `curve` sequence.

3.2.3 Static Parameters

These are pool parameters that typically do not change with frequent operations such as `swap` and `modifyPosition`. Some are immutable, some are modifiable only by the pool owner, some are syncable with the protocol by any address, and some are updated when a new kernel is provided or activated. `SSTORE3` is used to read `kernel` and other static parameters efficiently.

In this document, occasionally, *tags* and *tokens* are used interchangeably. Tag emphasizes the fact that trading can be beyond ERC-20 tokens and can include various other tradables, a.k.a., multi-tokens.

- `tag0/1`. (256-bit each, type `Tag`) These refer to the two tokens in a pool, and traders can swap from one to the other.⁴⁴ There is a lot of flexibility on what can be traded in the `NoFeeSwap` protocol.⁴⁵ Type of each tag may refer to any of the following:
 - **Native token.** For example, ETH is Ethereum’s native token. The native token’s tag value is zero.
 - **ERC-20.** The 160-bit token address is upcast to a 256-bit tag.
 - **ERC-1155.** The value of the tag is determined by hashing the 160 bits of `tokenAddress` and `tokenId` (256 bits):
`keccak256(encodePacked(tokenId, tokenAddress))`.
 - **ERC-6909.** The value of the tag is determined by hashing the 160 bits of `tokenAddress` and `tokenId` (256 bits):
`keccak256(encodePacked(tokenId, tokenAddress))`.
 - **NoFeeSwap LP token.** The tag value is determined by hashing `poolId` (256 bits) and min/max log-prices of the position (each 256 bits):
`keccak256(encodePacked(poolId, qmin, qmax))`. The bounds `qmin` and `qmax`, here, are in the X59 two’s complement representation, as in Table 2. See X59 in Section 3.7 and Remark 50.

⁴⁴In this document, occasionally, *tags* and *tokens* are used interchangeably. Tag emphasizes the fact that trading can be beyond ERC-20 tokens and can include various other tradables, aka, multi-tokens.

⁴⁵The accounting of tags in `NoFeeSwap`’s singleton is compliant with ERC-6909 multi-token specifications.

Remark 26. *LP tokens from pools with no hook can be traded as above. However, this is not the case in some situations, depending on the hook. For example, the incentive hook discussed in Section 5.1 holds on to this LP token and, in return, gives out an NFT that cannot be traded as a tag.*

The hashing here allows `tag` to fit in a 256-bit as an identifier. The tags of a pool are immutable and cannot be modified after the pool initialization.

Remark 27. *In every pool, the tag with an arithmetically smaller 256-bit is `tag0` and the other one is `tag1`.*

Remark 28. *The first two tag types from the list above are available in other DEXs. NoFeeSwap provides more flexibility and facilitates trading beyond ERC-20 tokens.*

Remark 29. *The concept of tag enables trading beyond ERC-20 in a DEX, i.e., it facilitates trading the protocol's LP tokens as well as ERC-1155 and ERC-6909 multi-tokens. This feature enables the extension of the DEX capability to manage combinations of NFTs and semi-fungible tokens.*

- `sqrOffset/sqrtInverseOffset`. (256-bit each, type X127) These are immutable and cannot be modified after the initialization. See Remark 17 for relevant explanations on picking offset. The justification for adding these parameters is discussed in Remark 7.

Remark 30. *Section 3.2.1 explains that the price offset's natural logarithm is encoded in `poolId`. See Example 4. The purpose of including these two additional values in the static parameters is to optimize the calculations.*

- `spacing`. (496-bit) Define $q_{\text{spacing}} = q_{\text{upper}} - q_{\text{lower}}$ as the width of each liquidity interval, which is immutable and cannot be modified after initialization. This 496-bit memory space occupies:
 - `logSpacing` (64-bit)
 - `sqrSpacing` (216-bit, type X216)
 - `sqrInverseSpacing` (216-bit, type X216)

which can be accessed via `PriceLibrary`. These, respectively, represent:

$$q_{\text{spacing}} = q_{\text{upper}} - q_{\text{lower}}, \quad (60a)$$

$$\frac{1}{\sqrt{p_{\text{spacing}}}} = e^{-\frac{q_{\text{spacing}}}{2}} = e^{-\frac{q_{\text{upper}} - q_{\text{lower}}}{2}}, \quad (60b)$$

$$\frac{\sqrt{p_{\text{spacing}}}}{e^{+16}} = e^{-16 + \frac{q_{\text{spacing}}}{2}} = e^{-16 + \frac{q_{\text{upper}} - q_{\text{lower}}}{2}}. \quad (60c)$$

It is straightforward to verify that $-16 < q_{\text{lower}} < q_{\text{upper}} < 16^{46}$ assures that the last two parameters stay between 0 and 1.

Remark 31. q_{upper} and q_{lower} refer to the boundaries of the active liquidity interval and should not be confused with q_{min} and q_{max} , which are used to refer to the boundaries of a liquidity position in this document.

- `outgoingMax`. (216-bit, type `X216`) It is calculated as in equation (43). It is updated when a new `kernel` is activated. There are two usages for this parameter: 1) When a pending `kernel` is activated, this is recalculated, as a normalizer, to keep the rest of the calculations intact and invariant with respect to updates of `kernel`; 2) For large trades, the liquidity intervals can be efficiently crossed by using this value as opposed to calculating the integrals.

Since the protocol performs division by `outgoingMax` frequently, its modular inverse is included as well.

- `outgoingMaxModularInverse`. (256-bit, type `uint256`) It is the modular inverse of `outgoingMax/2k` modulo 2^{256} , where k is the largest power of 2 that divides `outgoingMax`.
- `incomingMax`. (216-bit, type `X216`) It is similar to `outgoingMax` but for the incoming token (the one that the pool receives from a swapper) as

⁴⁶as in Remark 7

opposed to the outgoing token. This parameter is defined as follows:

$$\begin{aligned}
\text{incomingMax}_k &\triangleq e^{-\frac{q_{\text{upper}}}{2}} \int_{q_{\text{lower}}}^{q_{\text{upper}}} e^{+\frac{h}{2}} k(h - q_{\text{lower}}) dh \\
&= e^{+\frac{q_{\text{lower}}}{2}} \int_{q_{\text{lower}}}^{q_{\text{upper}}} e^{-\frac{h}{2}} k(q_{\text{upper}} - h) dh \\
&= e^{-\frac{q_{\text{spacing}}}{2}} \int_0^{q_{\text{spacing}}} e^{+\frac{h}{2}} k(h) dh,
\end{aligned} \tag{61}$$

which is a constant associated with the kernel. Similarly to `outgoingMax`, in (43), the value of this parameter is equivalent for all liquidity intervals. Here, for simplicity, it is written only for the active interval.

Similarly to `outgoingMax`, this is updated when a new kernel is activated. The usage of this is that for large trades, the liquidity intervals can be efficiently crossed by using this as opposed to calculating the integrals.

- `poolGrowthPortion`. (48-bit, type X47) It is the portion of liquidity growth which the pool owner takes. This portion is capped by the value `maxPoolGrowthPortion` (discussed next), determined by the protocol owner.

If the `isMutablePoolGrowthPortion` flag is on during the pool initialization, the pool owner can change this pool parameter through the `modifyPoolGrowthPortion` method of `NofeeswapDelegatee`.

- `maxPoolGrowthPortion`. (48-bit, type X47) It is an upper limit on the portion that the pool owner can take from the liquidity growth. It is the pool's local copy of this parameter.

Remark 32. *Professional pool owners can regularly design and update kernels, benefiting other LPs in the pool, and receive a portion of the pool's growth in return. Note that the protocol governance can set any cap on the pool owner's portion.*

- `protocolGrowthPortion`. (48-bit, type X47) It is the portion of liquidity growth that goes to the protocol. It is the pool's local copy of this parameter.

Remark 33. *The global parameters `maxPoolGrowthPortion` and also `protocolGrowthPortion`, for every pool, are under the control of the protocol and are modifiable by the protocol's governance.*

After a change of those global parameters, any address can run the function `updateGrowthPortions` of `NoFeeSwapDelegatee` for any pool to update growth portion parameters in that pool's storage and make it in sync with the protocol's global version. See Section 3.2.5.

Remark 34. Currently, `protocolGrowthPortion` is set to zero for all pools. Protocol governance can increase this to a non-zero amount.

Remark 35. The growth portions of the protocol and the pool owner are kept track of and can be withdrawn as mentioned in Section 3.2.4. The protocol first applies the protocol's growth portion, and then, the pool's growth portion, i.e., the growth is split into three components as below:

LPs' Portion

$$\begin{aligned} \text{growth}_{\text{post}} - \text{growth}_{\text{pre}} &= (\text{growth}_{\text{postNoSplit}} - \text{growth}_{\text{pre}}) \\ &\times (1 - \text{protocolGrowthPortion}) \\ &\times (1 - \text{poolGrowthPortion}[\text{capped}]), \end{aligned} \quad (62a)$$

Protocol's Portion

$$\begin{aligned} \text{marginalGrowth}_{\text{protocol}} &= (\text{growth}_{\text{postNoSplit}} - \text{growth}_{\text{pre}}) \\ &\times \text{protocolGrowthPortion}, \end{aligned} \quad (62b)$$

Pool Owner's Portion

$$\begin{aligned} \text{marginalGrowth}_{\text{pool}} &= (\text{growth}_{\text{postNoSplit}} - \text{growth}_{\text{pre}}) \\ &\times (1 - \text{protocolGrowthPortion}) \\ &\times \text{poolGrowthPortion}[\text{capped}], \end{aligned} \quad (62c)$$

where

- $\text{growth}_{\text{pre}}$ represents the accumulated growth for LPs before the current swap;
- $\text{growth}_{\text{postNoSplit}}$ represents the accumulated growth for LPs after the current swap if no growth splitting is applied, i.e., it is the LPs' growth under the assumptions of $\text{protocolGrowthPortion} = 0$ and $\text{poolGrowthPortion}[\text{capped}] = 0$, which is the growth derived in Section 3.5;
- $\text{growth}_{\text{post}}$ represents the accumulated growth for LPs after the current swap (where growth splitting is taken into account);

- $\text{poolGrowthPortion}[\text{capped}]$ is the effective growth portion for the pool owner, i.e., $\min\{\text{maxPoolGrowthPortion}, \text{poolGrowthPortion}\}$;
- $\text{marginalGrowth}_{\text{protocol}}$ represents the marginal growth for the protocol from the current swap;
- $\text{marginalGrowth}_{\text{pool}}$ represents the marginal growth for the pool owner from the current swap.

It is straightforward to verify that the amount of growth from the current swap which LPs do not receive due to splitting, $\text{growth}_{\text{postNoSplit}} - \text{growth}_{\text{post}}$, is equal to the combined growth of the protocol and the pool owner from the current swap, $\text{marginalGrowth}_{\text{protocol}} + \text{marginalGrowth}_{\text{pool}}$.

- `pendingKernelLength`. (16-bit, type `Index`) Keeps track of the number of break-points of the pending kernel. If it is 0, it means that there are no pending kernels.
- `kernel`. It is defined in Section 3.1.1. It governs the distribution of liquidity across the price horizon. We store kernels by listing their break-points *except for the origin*⁴⁷. For example, a kernel stored with a single break-point, $\{(q_{\text{spacing}}, 1)\}$, represents a simple ramp that connects the origin, $(0, 0)$, to the point $(q_{\text{spacing}}, 1)$. In addition, $\{(\frac{q_{\text{spacing}}}{2}, 0), (\frac{q_{\text{spacing}}}{2}, 1), (q_{\text{spacing}}, 1)\}$ represents a step function with a jump in $\frac{q_{\text{spacing}}}{2}$.

Remark 36. A wide variety of kernel functions can be constructed. The break-points are provided through a `kernelCompactArray` on initialization of a pool or when selecting a new pending kernel. A pool owner can provide a new kernel in compact form through the `modifyKernel` function. Figure 15 shows the structure for one break-point. In the compact form, which is how a pool owner provides the input, for each break-point, a 64-bit horizontal coordinate (log-price, q) is given in `X59` representation, and a 16-bit vertical coordinate, corresponding to kernel's intensity, $k(q)$, is provided in `X15` representation. Hence, each break-point takes 80 bits in the compact form. Break-points should be tightly packed within a `uint256[]` array with their height appearing first. Examples of kernels with a few break-points are provided in Figure 6.

⁴⁷The exclusion is only for efficiency purposes as the origin is always the first member.

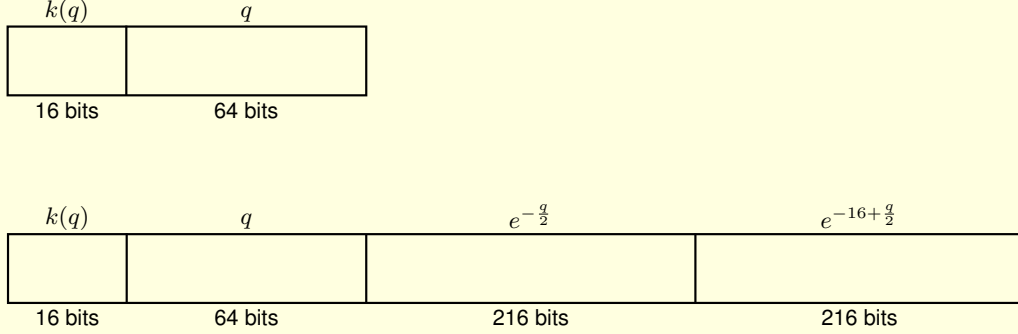


Figure 15: (a) Structure of each break-point in kernel's compact form, `kernelCompactArray`, 80 bits (top); (b) Structure of each break-point in kernel's expanded form, `kernel`, 512 bits (bottom).

The expanded version of the kernel includes two additional terms for each break-point (for computational efficiency), i.e., $e^{-\frac{q}{2}}$ and $e^{-16+\frac{q}{2}}$. These are always less than 1 as $0 < q < \frac{32}{3}$.⁴⁸ Since they are always less than 1, they are stored in X216 representation with 216 bits. Hence, each `kernel` break-point takes 512 bits (16 + 64 + 216 + 216).

Remark 37. Assume that the pool owner inputs $k(q_1), q_1, k(q_2), q_2, k(q_3), q_3, \dots, k(q_N), q_N$ for `kernelCompactArray`. Note that $(k(q_0), q_0) = (0, 0)$ is always considered the first member of `kernel` and is not entered. Below are the input requirements:⁴⁹

- $q_{i-1} = q_i = q_{i+1}$ must be avoided for any $1 \leq i \leq N - 1$.
- $k(q_{i-1}) = k(q_i) = k(q_{i+1})$ must be avoided for any $1 \leq i \leq N - 1$.
- $(k(q_i), q_i) = (k(q_{i+1}), q_{i+1})$ must be avoided for any $0 \leq i \leq N - 1$.
- $q_{i+1} < q_i$ must be avoided for any $0 \leq i \leq N - 1$.
- $k(q_{i+1}) < k(q_i)$ must be avoided for any $0 \leq i \leq N - 1$.

⁴⁸As in Section 3.1.1, all q 's of the kernel are between 0 and q_{spacing} . Also, as in Remark 55, $q_{\text{spacing}} < \frac{32}{3}$.

⁴⁹Otherwise, the initialization or the kernel update call is **reverted**.

- $q_1 = 0$ must be avoided, i.e., a vertical jump from the origin is not permitted.
- $q_{N-1} = q_N$ must be avoided, i.e., a vertical jump to the last point is not permitted.
- $k(q_N) = 1^{50}$ and $q_N = q_{\text{spacing}}$ must hold.
- $q_{i+1} - q_i \geq 2^{-27}$ should hold⁵¹ for each $0 \leq i \leq N - 1$ where $q_{i+1} \neq q_i$, to ensure the robustness of the numerical calculation and to avoid catastrophic cancellation.

3.2.4 Growth Sharing Parameters

Growth in the active liquidity interval increases by `donate` and `swap` calls, and the below parameters are used for keeping track of the portions for the protocol and the pool owners.

- `accrued0/1`. (104-bit each) These values account for the total accrued amounts of growth portions with respect to `tag0/1`, owed to the pool owner and the protocol, collectively.

The equations in Remark 35 are in terms of growth. They should be multiplied by the number of shares and other parameters for determining the token amounts, similar to equations (45), (47), (57).

- `poolRatio0/1`. (24-bit each, type X23) These are the ratios of accumulated values belonging to the pool (and the rest belonging to the protocol).

These 4 parameters are packed in one 256-bit slot, as $(104 + 24) \times 2 = 256$.

Remark 38. To avoid overflows, `sharesGross` is a storage parameter that represents the sum of share counts across all liquidity intervals of a pool, with the maximum permitted amount of $2^{127} - 1$ for each pool.

⁵⁰X15 representation for this value is 2^{15} .

⁵¹The X59 representation for this difference value is $2^{-27+59} = 2^{32}$. This limit corresponds to $\frac{p_{i+1}}{p_i} \geq e^{(2^{-27})} \sim 1.00000000745$.

3.2.5 Protocol Slot

Protocol slot is a 256-bit space in storage that stores the global growth portion parameters that are subject to change through protocol governance, i.e., the default values for `maxPoolGrowthPortion` and `protocolGrowthPortion`. These can also be overridden by the `Sentinel` contract to allow customized growth portions for each pool or token pair. The protocol slot also stores the address of the protocol owner, which is currently `Timelock`. See Figure 16.

This is not read in every call. There is a local version of the growth portion parameters kept for each pool, as in Section 3.2.3.

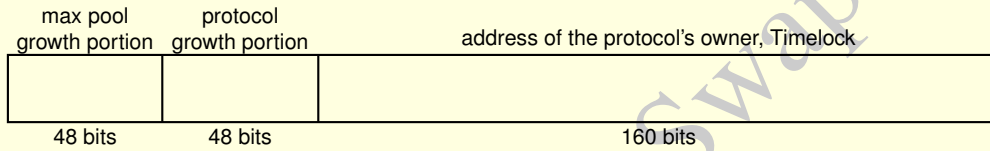


Figure 16: Protocol slot (48+48+160=256 bits).

3.3 Mappings

There are two mappings on the boundaries of the liquidity interval, `sharesDelta` and `growthMultiplier`, enabling efficient updates of the share and growth amounts when there is a transition to an adjacent liquidity interval. They are also used for handling `modifyPosition` calls.

3.3.1 `sharesDelta`

`sharesDelta` is defined on the boundaries of adjacent liquidity intervals and is the net difference of the number of shares in the two intervals. It is of type `int256`. See Figure 17.

3.3.1.1 `sharesTotal` updates with interval transitions. As the active liquidity interval transitions, `sharesDelta` is used to update `sharesTotal`.

- With an interval transition to the left (through q_{lower}), `sharesDelta[ $q_{\text{lower}}$ ]` should be subtracted from the number of shares in the active interval to derive the number of shares in the new active interval.

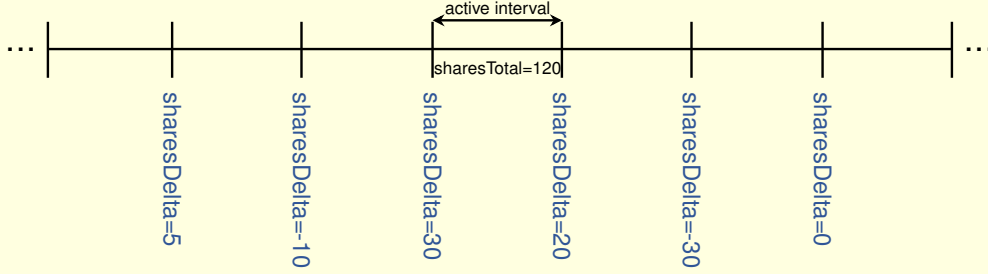


Figure 17: Depicting sharesTotal and sharesDelta where share counts for these 7 liquidity intervals are (95, 100, 90, 120, 140, 110, 110).

- With an interval transition to the right (through q_{upper}), $\text{sharesDelta}[q_{\text{upper}}]$ should be added to the number of shares in the active interval to derive the number of shares in the new active interval.

3.3.1.2 sharesDelta updates with modifyPosition. The mappings at q_{min} and q_{max} (the two boundaries of the position) are updated.

3.3.2 growthMultiplier

Each `growthMultiplier` is a slot and is stored in the X208 representation. For each integer m , let $g(m) \geq 1$ denote the growth value for the liquidity interval.⁵²

$$[q_{\text{lower}} + mq_{\text{spacing}}, q_{\text{upper}} + mq_{\text{spacing}}]. \quad (63)$$

The mapping can be calculated as:

$$\text{growthMultiplier}[q_{\text{upper}} + |m|q_{\text{spacing}}] = \sum_{n=0}^{+\infty} g(+n + |m| + 1) e^{-\frac{q_{\text{upper}} + (n+|m|)q_{\text{spacing}}}{2}}, \quad (64a)$$

$$\text{growthMultiplier}[q_{\text{lower}} - |m|q_{\text{spacing}}] = \sum_{n=0}^{+\infty} g(-n - |m| - 1) e^{+\frac{q_{\text{lower}} - (n+|m|)q_{\text{spacing}}}{2}}. \quad (64b)$$

⁵²It is $\text{growth}_{\text{post}}$ in Remark 35. The growth updates through swaps are shown in equation (74).

It can be observed that this mapping is defined on every spaced log-price. Note that:

- The mapping terms on the right side of the active liquidity interval record a weighted sum of the growth values for all the liquidity intervals on their right side;
- The mapping terms on the left side of the active liquidity interval record a weighted sum of the growth values for all the liquidity intervals on their left side.

3.3.2.1 Facilitating `modifyPosition`. These values make calculations of the token amounts straightforward. If an LP wants to withdraw a single share from every liquidity interval within spaced log-price $q_{\text{upper}} + mq_{\text{spacing}}$ ($m \geq 0$) to $+\infty$, the number of `token0` owed is equal to:⁵³

$$\frac{1}{\sqrt{p_{\text{offset}}}} \times \text{growthMultiplier}[q_{\text{upper}} + mq_{\text{spacing}}]. \quad (65)$$

Similarly, the amount of `token1` corresponding to a single share from every liquidity interval within $-\infty$ to spaced log-price $q_{\text{lower}} - mq_{\text{spacing}}$ ($m \geq 0$) is equal to:

$$\sqrt{p_{\text{offset}}} \times \text{growthMultiplier}[q_{\text{lower}} - mq_{\text{spacing}}]. \quad (66)$$

Remark 60 shows how the token amounts for `modifyPosition` are derived from this mapping using equations (65)–(66).

3.3.2.2 Keeping track of the growth values. The `growthMultiplier` mapping enables keeping track of the growth values for all liquidity intervals. An interval transition requires calculation for the new active interval's growth and updating the transitioned boundary's mapping. Figure 18 shows an interval transition towards the left, including:

1. The new active liquidity interval's growth value is derived as:

$$g(-1) = e^{-\frac{q_{\text{lower}}}{2}} (\text{growthMultiplier}[q_{\text{lower}}] - \text{growthMultiplier}[q_{\text{lower}} - q_{\text{spacing}}]). \quad (67)$$

⁵³This can be shown using equations (64a) and (45).

2. The transitioned boundary's mapping, $\text{growthMultiplier}[q_{\text{lower}}]$, is updated to track the cumulative growth on the right side of the new active liquidity interval:

$$\text{growthMultiplier}[q_{\text{lower}}] = \text{growthMultiplier}[q_{\text{upper}}] + e^{-\frac{q_{\text{lower}}}{2}} g(0). \quad (68)$$

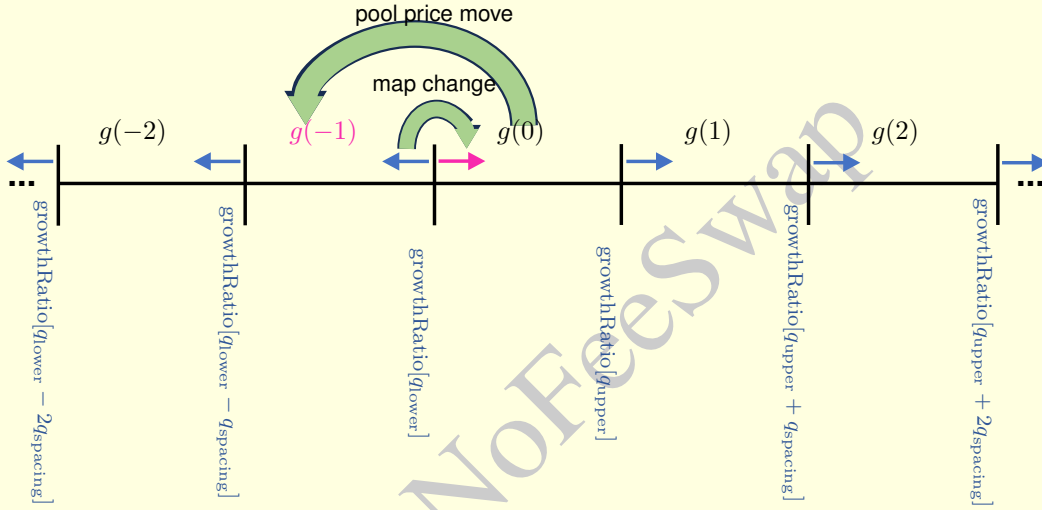


Figure 18: Interaction of growth and growthMultiplier with an interval transition towards left.

Similarly, after an interval transition towards the right:

1. The new active liquidity interval's growth value is derived as:

$$g(1) = e^{+\frac{q_{\text{upper}}}{2}} (\text{growthMultiplier}[q_{\text{upper}}] - \text{growthMultiplier}[q_{\text{upper}} + q_{\text{spacing}}]), \quad (69)$$

2. The transitioned boundary's mapping, $\text{growthMultiplier}[q_{\text{upper}}]$, is updated to track the cumulative growth on the left side of the new active liquidity interval:

$$\text{growthMultiplier}[q_{\text{upper}}] = \text{growthMultiplier}[q_{\text{lower}}] + e^{+\frac{q_{\text{upper}}}{2}} g(0). \quad (70)$$

3.4 Calculation of q_{target} and Swap Amounts

The main inputs for a swap call are `logPriceLimit`, `amountSpecified`, and `zeroForOne`.

1. `logPriceLimit`: q_{limit} is a limit that the swap does not move the pool's price beyond. If `zeroForOne[LSB8]` $\notin \{0, 1\}$, the swap's direction is implicitly determined from q_{limit} .⁵⁴ `logPriceLimit` is in X59 representation. See Table 2.
2. `amountSpecified`: This specifies a token-amount limit for the swap. Its sign determines whether the amount refers to the incoming token or the outgoing token.⁵⁵

`exactInput` is a parameter, internal to the protocol, which specifies if `amountSpecified` determines the value of the incoming token or the outgoing token. Its determination is as follows. Note that the incoming token is received by the pool, and therefore, its amount is positive. Hence, if `amountSpecified` > 0 , it refers to the incoming token's amount, and if `amountSpecified` < 0 , it refers to the outgoing token's amount.

- (a) `exactInput` = 0 $\iff$ `amountSpecified` < 0 ,
- (b) `exactInput` = 1 $\iff$ `amountSpecified` > 0 .

3. `zeroForOne`: It is a 256-bit value in which the most significant 248 bits and the least significant 8 bits are reserved for separate purposes.

- (a) `transitionThreshold`:⁵⁶ It is a threshold on the amount of liquidity that should be available in a liquidity interval for the swap function to transact in it. For example, if `transitionThreshold` = 0, which is the default, no minimum liquidity is imposed.

For example, if this value is 5 in a swap call, it means that at least 5 units of liquidity should exist in an interval for the swapper to be willing to trade in that interval.

⁵⁴If `zeroForOne[LSB8]` $\in \{0, 1\}$, it determines the swap's direction.

⁵⁵The incoming token refers to the token that the swapper gives to the pool, and the outgoing token refers to the token that the swapper receives from the pool.

⁵⁶`transitionThreshold` is defined as `zeroForOne` $>> 8$.

Remark 39. A novel feature is the existence of an optional threshold, picked by swappers, which allows them to avoid trading in low liquidity intervals and to avoid the consequent undesired high slippage and gas costs.

(b) zeroForOne[LSB8]:⁵⁷

- i. $\text{zeroForOne}[\text{LSB8}] = 0$ means swapper gives token1 and receives token0. Therefore, the pool gives token0 and receives token1.
- ii. $\text{zeroForOne}[\text{LSB8}] = 1$ means swapper gives token0 and receives token1. Therefore, the pool gives token1 and receives token0.
- iii. $\text{zeroForOne}[\text{LSB8}] \notin \{0, 1\}$ means that the trade is “zero for one” if $q_{\text{limit}} < q_{\text{current}}$ and it is “one for zero” if $q_{\text{current}} < q_{\text{limit}}$, i.e., swaps are in the direction of reaching q_{limit} .

Remark 40. For $\text{zeroForOne}[\text{LSB8}]$, the values of 0 and 1 are reserved for specifying the swap’s direction. In case $\text{zeroForOne}[\text{LSB8}] \in \{0, 1\}$, then logPriceLimit should be picked in the direction that $\text{zeroForOne}[\text{LSB8}]$ implies, and the swap gets reverted otherwise. Having only these 2 choices can be inconvenient for arbitrageurs. A novelty here is to allow swappers to pick $\text{zeroForOne}[\text{LSB8}] \notin \{0, 1\}$, which signals the swapper’s lack of intention to specify a direction. In this case, the swap happens such that the pool price moves towards logPriceLimit , with no specific direction enforced. This is particularly attractive for arbitrageurs who look for mispricing in various pools, as opposed to swapping in a specific direction.

Remark 41. Swaps in arbitrary directions are enabled, which facilitates arbitrage actions that intend to sync the pool price with a target price, e.g., an outside source, with no regard for the direction of the swap.

Remark 42. The swap calls can be categorized as follows for their directions:

- **Price-increasing trades.** (tag1 is received by the pool)

– $\text{zeroForOne}[\text{LSB8}] = 0$: If $q_{\text{limit}} > q_{\text{current}}$, the swap proceeds and otherwise it gets reverted.

⁵⁷ $\text{zeroForOne}[\text{LSB8}]$ is defined as: $((1 \ll 8) - 1) \wedge \text{zeroForOne}$.

- zeroForOne[LSB8] $\notin \{0, 1\}$ & $q_{\text{limit}} > q_{\text{current}}$.
- **Price-decreasing trades.** (*tag0 is received by the pool*)
 - zeroForOne[LSB8] = 1: If $q_{\text{limit}} < q_{\text{current}}$, the swap proceeds and otherwise it gets reverted.
 - zeroForOne[LSB8] $\notin \{0, 1\}$ & $q_{\text{limit}} < q_{\text{current}}$.

Numerical Formulation. Depending on the swapper's inputs, one of the following equations is solved for each swap. This determines the size of the swap. The corresponding functions are `searchIncoming`⁵⁸ and `searchOutgoing`⁵⁹. The derivation of q_{target} for various sets of parameters is as follows:

1. If price-increasing trade & exactInput = 0, a specified amount of `tag0` is taken from the protocol in swap. Derive $q_{\text{target}} = \min\{q_{\text{search}}, q_{\text{limit}}\}$, where q_{search} is calculated through numerical methods from:

$$\frac{1}{\sqrt{p_{\text{offset}}}} \times \int_{q_{\text{current}}}^{q_{\text{search}}} e^{-\frac{h}{2}} z_t(h) dh = |\text{amountSpecified}|. \quad (71a)$$

2. If price-increasing trade & exactInput = 1, a specified amount of `tag1` is given to the protocol in swap. Derive $q_{\text{target}} = \min\{q_{\text{search}}, q_{\text{limit}}\}$, where q_{search} is calculated through numerical methods from:

$$\sqrt{p_{\text{offset}}} \times \int_{q_{\text{current}}}^{q_{\text{search}}} e^{+\frac{h}{2}} z_t(h) dh = |\text{amountSpecified}|. \quad (71b)$$

3. If price-decreasing trade & exactInput = 1, a specified amount of `tag0` is given to the protocol in swap. Derive $q_{\text{target}} = \max\{q_{\text{search}}, q_{\text{limit}}\}$, where q_{search} is calculated through numerical methods from:

$$\frac{1}{\sqrt{p_{\text{offset}}}} \times \int_{q_{\text{search}}}^{q_{\text{current}}} e^{-\frac{h}{2}} z_t(h) dh = |\text{amountSpecified}|. \quad (71c)$$

4. If price-decreasing trade & exactInput = 0, a specified amount of `tag1` is taken from the protocol in swap. Derive $q_{\text{target}} = \max\{q_{\text{search}}, q_{\text{limit}}\}$, where q_{search} is calculated through numerical methods from:

$$\sqrt{p_{\text{offset}}} \times \int_{q_{\text{search}}}^{q_{\text{current}}} e^{+\frac{h}{2}} z_t(h) dh = |\text{amountSpecified}|. \quad (71d)$$

⁵⁸when exactInput = 1

⁵⁹when exactInput = 0

Remark 43. *swapWithin and cross are discussed in Section 3.6. Each cross reduces the absolute value of amountSpecified to the remaining specified amount. The above equations for finding q_{search} are part of swapWithin.*

Remark 44. *The above steps determine q_{target} . The corresponding swap token amounts are derived using Remarks 2 and 7.*

3.5 Calculation of Overshoot & Growth

As demonstrated in Figure 20, there are typically two updates for curve sequence after each swapWithin. The curve sequence amends an overshoot from the target log-price to $q_{\text{overshoot}}$, and then comes back and amends q_{target} .⁶⁰ Note that these are parts of the process for updating the curve sequence and growth after each swapWithin, as explained in the following.⁶¹

Let $\bar{q}_t$ represent the curve sequence before the pool's t^{th} swap, and $\bar{q}_{t+1}$ represent the curve sequence after this swap, i.e., after amending it with t^{th} swap's $q_{\text{overshoot}}$ and q_{target} .

Remark 45. *When the pool's price is on q_{target} and curve sequence is changed from $\bar{q}_t$ to $\bar{q}_{t+1}$ (by appending $q_{\text{overshoot}}$ and q_{target}), the number of shares and tokens does not change in the transition from the old curve to the amended one. Therefore, $\text{growth} \times \text{integral0}$ and $\text{growth} \times \text{integral1}$ must stay unchanged, according to equation (57), i.e.,*

$$\frac{\text{growth}_t}{\text{growth}_{t+1}} = \frac{\text{integral0}_{t+1}}{\text{integral0}_t} = \frac{\text{integral1}_{t+1}}{\text{integral1}_t}. \quad (72)$$

Define:

$$f_{\text{left}}(q_{\text{overshoot}}) \triangleq \frac{\int_{q_{\text{lower}}}^{q_{\text{target}}} e^{+\frac{h}{2}} k(w_{\bar{q}_{t+1}}(h)) dh}{\int_{q_{\text{lower}}}^{q_{\text{target}}} e^{+\frac{h}{2}} k(w_{\bar{q}_t}(h)) dh}, \quad (73a)$$

$$f_{\text{right}}(q_{\text{overshoot}}) \triangleq \frac{\int_{q_{\text{target}}}^{q_{\text{upper}}} e^{-\frac{h}{2}} k(w_{\bar{q}_{t+1}}(h)) dh}{\int_{q_{\text{target}}}^{q_{\text{upper}}} e^{-\frac{h}{2}} k(w_{\bar{q}_t}(h)) dh}. \quad (73b)$$

⁶⁰Therefore, either $q_{\text{current}} \leq q_{\text{target}} \leq q_{\text{overshoot}} \leq q_{\text{upper}}$ or $q_{\text{lower}} \leq q_{\text{overshoot}} \leq q_{\text{target}} \leq q_{\text{current}}$, depending on the swap's direction.

⁶¹The exchange amounts in the swap are already calculated before such updates, in the first step of Figure 20.

f_{left} and f_{right} represent the inverse of marginal growth of two sides, as in equation (72). The equation $f_{\text{left}} = f_{\text{right}}$ should be solved with respect to $q_{\text{overshoot}}$ to equate the growth values implied from these two fractions.⁶²

Assume that the trade is such that the pool price increases, i.e., $q_{\text{current}} \leq q_{\text{target}}$.⁶³ Then, $q_{\text{overshoot}} \in [q_{\text{target}}, q_{\text{upper}}]$. The following hold due to the specific structure of curve sequence updates described in Section 3.1.3.

- The extreme end where $q_{\text{overshoot}}$ is equal to q_{target} :
 - $f_{\text{left}}(q_{\text{target}}) \leq 1$, due to the Hardy–Littlewood inequality, as $w_{\bar{q}_{t+1}}$ is a decreasing rearrangement of $w_{\bar{q}_t}$ on the left side of q_{target} , and the exponential term in the integral is monotonically increasing.
 - $f_{\text{right}}(q_{\text{target}}) = 1$, as $w_{\bar{q}_{t+1}}$ matches $w_{\bar{q}_t}$ on the right side of q_{target} . In this case, there is no overshoot and the two liquidity distribution functions are identical within the liquidity interval $[q_{\text{target}}, q_{\text{upper}}]$.
- The other extreme end where $q_{\text{overshoot}}$ is equal to q_{upper} :
 - $f_{\text{left}}(q_{\text{upper}}) \geq f_{\text{right}}(q_{\text{upper}})$, because $w_{\bar{q}_{t+1}}$ takes its lowest amounts between q_{target} and q_{upper} , and the kernel is a non-decreasing function.
 - $f_{\text{right}}(q_{\text{upper}}) \leq 1$, as $w_{\bar{q}_{t+1}}$ falls below $w_{\bar{q}_t}$ on the right side of q_{target} . In this case, curve sequence is transformed to $[q_{\text{lower}}, q_{\text{upper}}, q_{\text{target}}]$, and therefore, $w_{\bar{q}_{t+1}}$ on the right side of q_{target} is on the lowest levels after the swap.

$q_{\text{overshoot}}$ is calculated such that $f_{\text{left}} = f_{\text{right}}$ holds for its corresponding $\bar{q}_{t+1}$. The above bullet points along with the Intermediate Value Theorem prove that there is a $q_{\text{overshoot}}^*$ value, $q_{\text{target}} \leq q_{\text{overshoot}}^* \leq q_{\text{upper}}$ such that $f_{\text{left}}(q_{\text{overshoot}}^*) = f_{\text{right}}(q_{\text{overshoot}}^*)$.^{64,65} This equation is solved iteratively by minimizing a mismatch term between f_{left} and f_{right} .

⁶²Note that q_{target} is already determined from Section 3.4 and the only remaining variable in curve sequence is $q_{\text{overshoot}}$.

⁶³The other case can be argued similarly.

⁶⁴ f_{right} goes from 1 to a value ≤ 1 over range $[q_{\text{target}}, q_{\text{upper}}]$. And, f_{left} goes from a value ≤ 1 to a value $\geq f_{\text{right}}$ over this range. Even when k has jump discontinuities, these integral functions are continuous in $q_{\text{overshoot}}$ because the discontinuities occur at finitely many points of measure zero. Therefore, the two functions intersect somewhere in this range.

⁶⁵ f_{right} is a non-increasing function of $q_{\text{overshoot}}$ and f_{left} is a non-decreasing function of $q_{\text{overshoot}}$, over range $[q_{\text{target}}, q_{\text{upper}}]$.

If $\bar{q}_{t+1}$ is derived by amending $q_{\text{overshoot}}^*$ and q_{target} , the below equation holds according to Remark 45.

$$\frac{\text{growth}_t}{\text{growth}_{t+1}} = \frac{\int_{q_{\text{lower}}}^{q_{\text{target}}} e^{+\frac{h}{2}} k(w_{\bar{q}_{t+1}}(h)) dh}{\int_{q_{\text{lower}}}^{q_{\text{target}}} e^{+\frac{h}{2}} k(w_{\bar{q}_t}(h)) dh} = \frac{\int_{q_{\text{target}}}^{q_{\text{upper}}} e^{-\frac{h}{2}} k(w_{\bar{q}_{t+1}}(h)) dh}{\int_{q_{\text{target}}}^{q_{\text{upper}}} e^{-\frac{h}{2}} k(w_{\bar{q}_t}(h)) dh} \leq 1, \quad (74)$$

and, therefore, the shrinkage of integrals caused by updates of curve sequence leads to growth.

Remark 46. *There are typically two updates of the curve sequence after each swap, one to overshoot and one to target. However, if the overshoot and target match, there will be only one update. For example, they match when the swap hits one of the two boundaries of a liquidity interval.*

Please refer to the section `_overshoot_` in `Memory.sol` for further details.

3.6 Swap Flowcharts

There are three types of operations that can be executed as part of a swap. Figure 19 shows the sequence for any swap.

1. **SwapWithin.** Swaps inside a liquidity interval. The previous sections focused on the algorithms involved in the execution of this component.

Figure 20 is a high-level demonstration of a swapWithin operation. Of the three steps in this flowchart, Section 3.4 discusses the first step. Section 3.5 discusses the calculation in the middle step. The last step is straightforward using the method discussed in Section 3.1.3.

2. **Cross a liquidity interval.** Cross an entire liquidity interval from one boundary to the next. This is carried out efficiently using the values `outgoingMax` and `incomingMax`.
3. **Transition to another liquidity interval.** Transition through a boundary from one liquidity interval to an adjacent one, which requires updating dynamic parameters (Section 3.2.2), shares/growth mappings (Section 3.3), and, if there is a pending `kernel`, static parameters (Section 3.2.3).

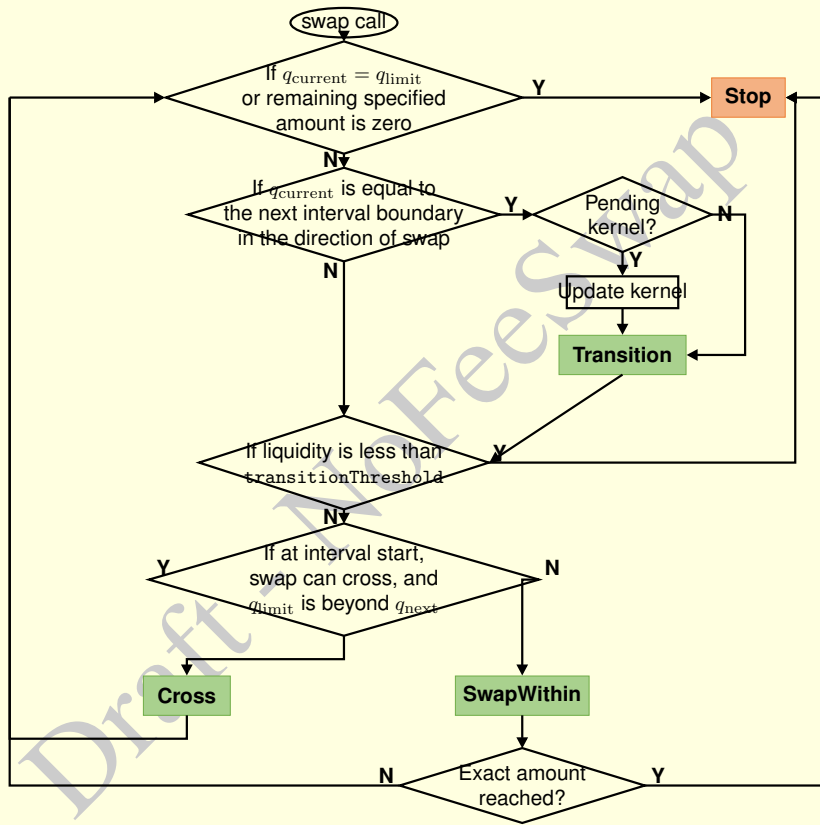


Figure 19: Swap flowchart determining how swapWithin, transition and cross operations are called.

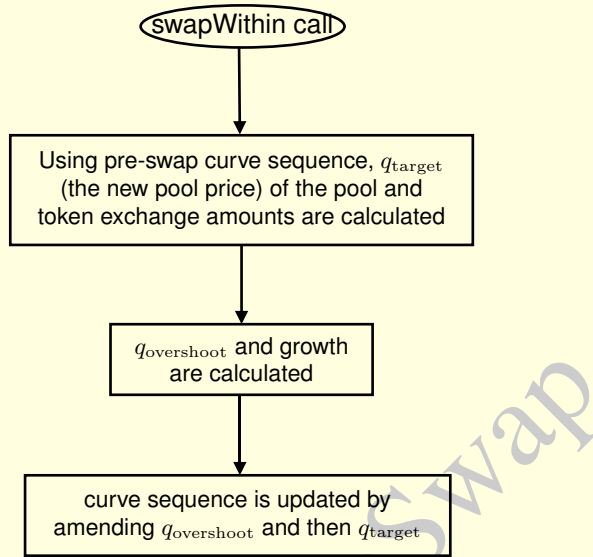


Figure 20: swapWithin flowchart.

3.7 Data Types

Solidity's custom types are widely used by the protocol. The types $X\#$ are created to specify fixed-point values with $\#$ binary fractional places.

- $X15$. (uint256) This type is dedicated to the liquidity distribution. The distribution of liquidity within the interval is governed by a piecewise-linear `kernel` function. The `kernel` function is characterized by a list of break-points provided by the pool owner. The vertical coordinate of each break-point is a number within the range $[0, 1]$ which is stored as $X15$, with 2^{15} representing 1. See $k(q)$ in Figure 15.
- $X23$. (uint256) This type is dedicated to the growth ratio parameters between the protocol and the pool owner. As in Section 3.2.4, the pool ratio parameters are stored as $X23$ with 2^{23} representing 100%.
- $X47$. (uint256) This type is dedicated to the growth portion parameters. As in Section 3.2.3, the growth portion parameters are stored as $X47$ with 2^{47} representing 100%.

- X59. (int256) This type is dedicated to the natural logarithm of prices. There are two distinct representation formats:

- **Offset-binary representation (offset-adjusted):**⁶⁶ It is used for members of `curve` sequence. The red terms in Table 1 cause the offset-binary format. As in Remark 7, the log-prices of a `curve` sequence are normalized to stay within ± 16 . As in Table 1, q_{offset} is included to convert it to the real-world price.

In this format, 0 refers to $q = -16$ and $p = e^{-16}$. The value 2^{64} would refer to $q = +16$ and $p = e^{+16}$, but it is not representable in 64 bits; the largest representable value is $2^{64} - 1$.

Offset-binary representation is selected for several reasons, including its properties due to symmetry over the range $q \in [-16, 16]$ and compatibility with exponential.

- **Two's complement representation (real-world):** For other log-prices, this representation is used.⁶⁷ See Table 2. Also, note that these are real-world values and not offset-adjusted ones.

* *Real-world prices for the purpose of simplicity for the users.*

- For `logPriceMin` and `logPriceMax` (in `modifyPosition`) and `logPriceLimit` (in `swap`), any signed value can be selected in a slot, i.e., $-2^{255} \leq h \leq 2^{255} - 1$.
- For the purpose of tag creation for LP tokens, 256-bit X59 representation is used for specifying the position's boundaries. See tags for `NoFeeSwap`'s LP tokens in Section 3.2.3.

* *Relative and non-negative measures where offset is not relevant.*

- q_{spacing} fits in 64 bits, as $0 < h < 2^{63}$. Note that $q_{\text{spacing}} = q_{\text{upper}} - q_{\text{lower}} > 0$ and $q_{\text{spacing}} < 16$ (2^{63-59}) according to Remark 55.
- For `kernel`, each q fits in 64 bits, as $0 \leq h < 2^{63}$. For every break-point of the kernel, $q \in [0, q_{\text{spacing}}]$. See q in Figure 15.
- q_{offset} is in X59 representation in various calculations. However, the one included in `poolId` is not in X59 representation.

⁶⁶“Offset-binary” refers to the method for signed number representation and “offset-adjusted” refers to applying q_{offset} .

⁶⁷There are various reasons, including simplicity and the fact that summing one offset-binary with two's complement numbers preserves the offset-binary representation.

Table 1: The relationship between the real-world prices and the corresponding offset-binary representation.

X59 for curve sequence members	standardized log-price, within $[-16, 16)$	real-world log-price, see equation (41b)	standardized price, within $[e^{-16}, e^{16})$	real-world price, see equation (41a)
$0 \leq h < 2^{64}$	$\frac{h-2^{63}}{2^{59}}$	$\frac{h-2^{63}}{2^{59}} + q_{\text{offset}}$	$e^{\frac{h-2^{63}}{2^{59}}}$	$e^{\frac{h-2^{63}}{2^{59}} + q_{\text{offset}}}$

Table 2: The relationship between the real-world prices and the corresponding two's complement representation.

X59	log-price	price	sqrt-price
h	$\frac{h}{2^{59}}$	$e^{\frac{h}{2^{59}}}$	$e^{\frac{h}{2^{60}}}$

Remark 47. q_{offset} , in storage, is not in X59 representation. It is maintained in `int8` representation in `poolId`, as in Section 3.2.1. In addition, `sqrtOffset` is maintained in static parameters in X127. However, they are converted to the X59 representation in various internal calculations.

Remark 48. The storage of the curve sequence using X59 representation in offset-binary is innovative.

- Due to offset-binary representation, the exponential of each member of the curve sequence can be calculated efficiently, i.e., the corresponding two's complement representation would have required conditioning on the sign and extra addition and inversion operations, which increases the gas cost and decreases the accuracy.
- A benefit of $e^{\frac{-2^{63}}{2^{59}}}$ in Table 1, resulting from the offset-binary representation, is that for the sqrt-price it becomes a e^{-8} multiplier which is efficiently used for another purpose discussed in Remark 9.
- The offset-binary's addition to non-negative two's complement log-prices preserves offset-binary.

Remark 49. For exponential calculations using the Padé approximant, usually the multipliers are too large for storage in EVM and a large number of terms are needed for sufficient accuracy. Here, the inputs are shrunk by a certain amount before the exponent calculation, to make sure that a small

number of terms with parameters storable with only a few digits are sufficient. Then, after the exponent calculation, the output is squared a sufficient number of times to net the effect of shrinkage and provide the expected value with full precision.

- X74. (int256) It is used as an intermediary type when searching for precise log-price movements corresponding to the specified outgoing/incoming token amounts.
- X111. (int256) It is dedicated to growth and liquidity values.
- X127. (int256) It is used for token amounts and `sqrtOffset`.
- X208. (uint256) It is used for growth multipliers.
- X216. (int256) It is dedicated to integrals and the sqrt-prices.

There are additional data types:

- Type `Index` (uint16) is used for enumerations, e.g., for members of the curve sequence and kernel.
- Type `Tag` (uint256) is used for referring to items in Section 3.2.3's tag discussion.
- Types `Curve`/`Kernel`/`KernelCompact` (uint256) are memory pointers for handling curve sequence and kernel.
- There are two data types for price, one includes three members `logPrice`, `sqrtPrice`, `sqrtInversePrice` and the other one includes four members `heightPrice`, `logPrice`, `sqrtPrice`, `sqrtInversePrice`.

3.8 External Functions

Here, a description of the external functions in the protocol and the corresponding inputs/outputs is provided.⁶⁸

⁶⁸The requirement in Remark 51 is a safeguard against reentrancy.

3.8.1 NoFeeSwap Contract

This contract is the core protocol and contains all swap-related functions.

- `modifyBalance`. Mints/burns a singleton balance of `tag`. This results in a transient storage balance that must be settled. This is more gas-efficient than external transfers from/to the sender's wallet.⁶⁹ See Figure 21.
 - `owner`: Balance owner.
 - `tag`: The tag whose balance is to be modified.
 - `amount`: The amount of `tag` to be added (positive) or removed (negative) to/from the owner's singleton balances.
- `modifyBalance`. Mints/burns a singleton multi balance for a list of spenders and tags. This results in transient storage balances that must be settled. See Figure 21. Can be used for gas-efficient transactions by eliminating the need to update multiple storage slots.⁷⁰
 - `spenderIndex`: The index of `msg.sender` among the members of `spenders`.
 - `spenders`: The strictly monotonically increasing packed list of accounts with access to the target singleton multi balance.
 - `tags`: The strictly monotonically increasing packed list of tags associated with the target singleton multi balance.
 - `balances`: The packed list of current balances associated with each tag, respectively.
 - `indices`: The packed list of tag indices whose balances are to be modified.
 - `amounts`: The packed list of amounts to be added (positive) or removed (negative) to/from the singleton multi balance.

⁶⁹Frequent users benefit from lower gas costs by depositing a certain amount of `tag` to NoFeeSwap, for future activities, as opposed to using an external wallet for every transaction.

⁷⁰A trader who exchanges or provides liquidity for pools of related token pairs frequently benefits from lower gas costs by depositing balances associated with several tags in a singleton multi balance for future activities.

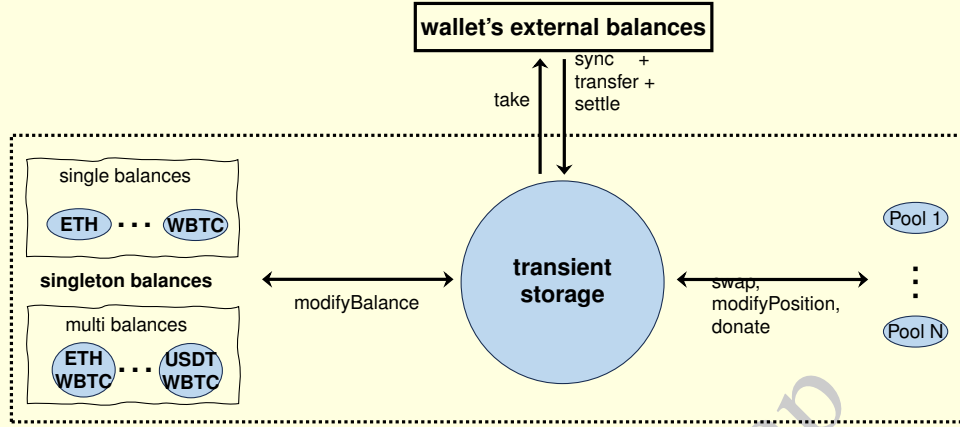


Figure 21: A visualization of the fund transfer structure.

Remark 50. In order to eliminate the need for expensive token and multi-token transfers, NoFeeSwap's core singleton offers an ERC-6909 compliant accounting structure. Users have the option to deposit their native, ERC-20, ERC-1155, and ERC-6909 tokens in return for the singleton's ERC-6909 balance. As explained in Section 3.2.3, the corresponding singleton multi-token identifiers are regarded as *tag* and are calculated as follows:

$$\begin{aligned}
 \text{Native} &\rightarrow \text{tag} = \text{Tag.wrap}(0) \\
 \text{ERC} - 20 &\rightarrow \text{tag} = \text{Tag.wrap}(\text{uint256}(\text{uint160}(\text{underlyingERC20Address})))
 \end{aligned} \tag{75}$$

$$\begin{aligned}
 \text{ERC} - 1155 &\rightarrow \text{tag} = \text{Tag.wrap}(\text{uint256}(\text{keccak256}(\text{encodePacked}(\text{underlyingERC1155TokenId}, \text{underlyingERC1155Address}))))
 \end{aligned} \tag{76}$$

```

ERC - 6909 → tag = Tag.wrap(uint256(keccak256(
    encodePacked(
        underlyingERC6909TokenId,
        underlyingERC6909Address
    )
)))
(77)

```

```

LP - Position → tag = Tag.wrap(uint256(keccak256(
    encodePacked(
        poolId,
        qMin,
        qMax
    )
)))
(78)

```

The ownership of these tags is accounted for via the singleton's internal ERC-6909 mechanism.

Note that one may also use the NoFeeSwap singleton address and any of the aforementioned tags as a pair of underlyingERC6909Address and underlyingERC6909TokenId in equation (77), in order to derive a different Tag for the same asset. However, users are discouraged from doing this in order to avoid unnecessary fragmentation.

In addition to the standard ERC-6909 interface, NoFeeSwap offers a multi-balance feature which allows users to track the balances associated with a list of tags and authorized spenders through a single storage slot.

- **unlock.** This function gives access to NoFeeSwap functions that are safeguarded against reentrancy (the calls that require a change in transient balances).
 - **unlockTarget:** The target contract address which must implement IUnlockCallback, i.e., the callback address. See Remark 51.
 - **data:** The data/instructions to be passed to the target contract.
 - **[output] result:** The output from the target contract.

Remark 51. *unlockCallback*: In order to interact with any of the following functions of *No feeswap* and *No feeswapDelegatee*, an account must first invoke *INo feeswap.unlock (unlockTarget, data)* to unlock the protocol.

```

INo feeswap.clear(Tag, uint256)
INo feeswap.take(address, address, uint256)
INo feeswap.take(address, uint256, address, uint256)
INo feeswap.take(
    address, uint256, address, uint256, bytes
)
INo feeswap.settle()
INo feeswap.transferTransientBalanceFrom(
    address, address, Tag, uint256
)
INo feeswap.modifyBalance(address, Tag, int256)
INo feeswap.modifyBalance(
    Index, bytes, bytes, bytes, bytes, bytes
)
INo feeswap.swap(uint256, int256, X59, uint256, bytes)
INo feeswapDelegatee.modifyPosition(
    uint256, X59, X59, int256, bytes
)
INo feeswapDelegatee.donate(uint256, uint256, bytes)

```

The function *INo feeswap.unlock* then invokes

IUnlockCallback (target) .unlockCallback (caller, data) through which any account may interact with the above methods.

The default target which implements *unlockCallback* is the contract *Operator.sol*, for which a sequence of instructions should be encoded as data. See Section 4.

All transient balances must be settled by the *unlockCallback* function to prevent the unlock call from reverting. In other words, all of the balances

in transient storage for all accounts should net to zero for this call to go through.

- *caller*: The caller of the protocol's `unlock` method.
- *data*: The input data that was passed to the `unlock` method.
- *[output] returnData*: The output that should be returned to the caller.

- `clear`. Clears positive transient balance.

- *tag*: The tag whose transient balance is to be cleared.
- *amount*: The amount to be cleared, which must be equal to the current transient balance of the caller.

Remark 52. *Some notes before discussing the next functions:*

- *take* function is used for withdrawals from the transient storage to an external wallet. (There is no need to use a `transfer` function with it as it is implicitly done in it.)
- *sync/transfer/settle* functions are used (in order) for deposits to the transient storage from an external wallet.
- *transfer* function for NoFeeSwap LP tokens is included here. For other tokens, this function from its corresponding contract is needed.

- `take`. Pays native or ERC-20 assets from NoFeeSwap to `to` (transient storage balance to a target wallet address). It can be used for flash loans as well. See Figure 21.

- *token*: ERC-20 or native tokens to be taken from the protocol's wallet. `address(0)` refers to the native token.
- *to*: The target address.
- *amount*: The token amount to be taken from the protocol.

- `take`. Pays ERC-6909 assets from NoFeeSwap to `to` (transient storage balance to a target wallet address). See Figure 21.

- *token*: ERC-6909 address.

- `tokenId`: Multi-token id to be taken from the protocol's wallet.
 - `to`: The target address.
 - `amount`: The token amount to be taken from the protocol.
- `take`. Pays ERC-1155 assets from NoFeeSwap to `to` (transient storage balance to a target wallet address). See Figure 21.
 - `token`: ERC-1155 address.
 - `tokenId`: Multi-token id to be taken from the protocol's wallet.
 - `to`: The target address.
 - `amount`: The token amount to be taken from the protocol.
 - `transferData`: Data for `IERC1155.safeTransferFrom`.
- `sync`. Synchronizes NoFeeSwap ERC-20. Should be called prior to ERC-20 transfers to NoFeeSwap.
 - `token`: ERC-20 tokens to be synced.
- `sync`. Synchronizes NoFeeSwap ERC-1155 or ERC-6909 balance. Should be called prior to ERC-1155 or ERC-6909 transfers to NoFeeSwap.
 - `token`: ERC-1155 or ERC-6909 address to be synced.
 - `tokenId`: Multi-token id to be synced.
- `settle`. Should be called after native, ERC-20, ERC-1155 or ERC-6909 transfers to NoFeeSwap to update transient balances. Can also be used to send native tokens to the protocol to settle native transient balance of `msg.sender`.
 - `[output] paid`: The amount paid.
- `transferTransientBalanceFrom`. Transfers transient balance from `sender` to `receiver`. Can also be used to settle on behalf of other accounts.
 - `sender`: The sender's address.
 - `receiver`: The recipient's address.
 - `tag`: The tag whose balance is to be transferred.

- `amount`: The amount to be transferred.
- `dispatch`. Executes delegate calls to the `NoFeeSwapDelegatee` contract. See Section 3.8.2.
 - `input`: Encoded input via `abi.encodeWithSelector` to be passed as a delegate call to `NoFeeSwapDelegatee` contract.
 - `[output] output0/1`: Two output slots.
- `swap`. Performs a swap. See Section 3.4 for the explanations of the input parameters.
 - `poolId`: The identifier of the target pool. See Section 3.2.1.
 - `amountSpecified`: The amount to be given to (positive) or taken from (negative) the pool.
 - `logPriceLimit`: See Section 3.4 and Table 2 for explanations.
 - `zeroForOne`: See Section 3.4 for explanations.
 - `hookData`: The data to be passed to the hook.
 - `[output] amount0/1`: The amount of `tag0/1` swapped. Positive amounts are incoming to the protocol, and negative amounts are outgoing from the protocol.
- `storageAccess`. Provides access to `NoFeeSwap` storage (one slot)
 - `slot`: The `bytes32` slot to be read.
 - `[output]` The `bytes32` value read from storage.
- `storageAccess`. Provides access to `NoFeeSwap` storage (a consecutive sequence of slots)
 - `startSlot`: The `bytes32` starting slot to be read.
 - `nSlots`: Number of slots to be read.
 - `[output]` The `bytes32[]` values read from storage.
- `storageAccess`. Provides access to `NoFeeSwap` storage (a sparse sequence of slots)
 - `slots`: A `bytes32[]` array of storage slots to be read.

- [output] The `bytes32 []` values read from storage.
- `transientAccess`. Provides access to NoFeeSwap transient storage (one slot)
 - `slot`: The `bytes32` slot to be read.
 - [output] The `bytes32` value read from transient storage.
- `transientAccess`. Provides access to NoFeeSwap transient storage (a consecutive sequence of slots)
 - `startSlot`: The `bytes32` starting slot to be read.
 - `nSlots`: Number of slots to be read.
 - [output] The `bytes32 []` values read from transient storage.
- `transientAccess`. Provides access to NoFeeSwap transient storage (a sparse sequence of slots)
 - `slots`: A `bytes32 []` array of transient storage slots to be read.
 - [output] The `bytes32 []` values read from transient storage.

See ERC-6909 specifications for the description of the remaining functions from the ERC-6909 standard, i.e.,

`supportsInterface`
`balanceOf`
`allowance`
`isOperator`
`transfer`
`transferFrom`
`approve`
`setOperator`.

3.8.2 NoFeeSwapDelegatee Contract

This contract is the second part of the protocol's core, and most operations except for swap are included here. Every function of this contract is executed through the `dispatch` function of NoFeeSwap contract, as mentioned in Section 3.8.1.

- `initialize`. Initializes a new NoFeeSwap pool.

- `unsaltedPoolId`: The unique pool identifier to be initialized. It encapsulates the hook address, the flags, and the price offset, as explained in Section 3.2.1. Please see Section 3.2.1.2 for explanations on `unsaltedPoolId`.
- `tag0/1`: The two tags for which this pool is created. The arithmetically smaller one is `tag0` and the other one is `tag1`.

[illegible]

Example 6. If $q_{\text{spacing}} = 2^{71}$, the kernel can be as in Table 3, which is depicted in Figure 22. Note that the first 0 entry is always omitted, as mentioned in Section 3.2.3. According to Table 2, the corresponding value of $q_{\text{spacing}} = 2$ in X59 is 2^{60} .

Kernel Break-point q	q in X59	Corresponding height, $k(q)$	$k(q)$ in X15
0	0	0	0
0.5	2^{58}	0.125	2^{12}
1	2^{59}	0.5	2^{14}
1	2^{59}	1	2^{15}
2	2^{60}	1	2^{15}

⁷¹i.e., $q_{\text{upper}} - q_{\text{lower}} = 2$ and $\frac{p_{\text{upper}}}{p_{\text{lower}}} = e^2$.

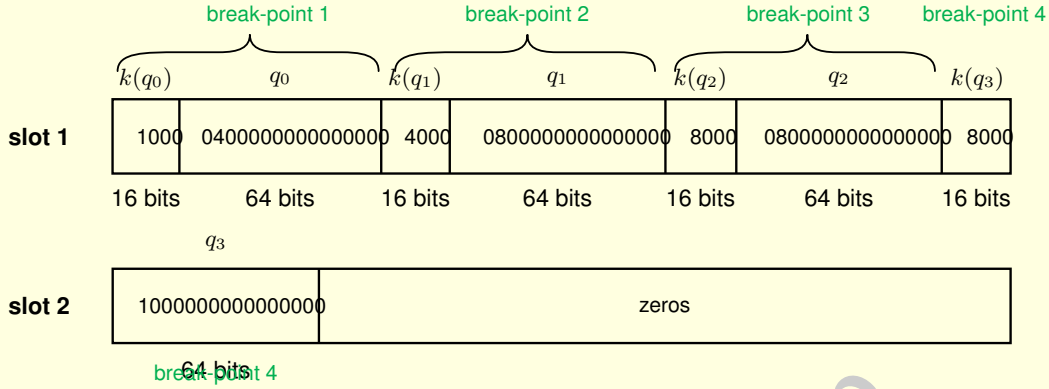


Figure 22: compactKernel of Example 6.

the initial curve sequence. The current price of the pool is the last member of the curve sequence. q_{spacing} can be determined from the first two members of the curve sequence, as $q_{\text{spacing}} = q_{\text{upper}} - q_{\text{lower}} = |q^0 - q^1|$. See Sections 3.1.2 and 3.1.3 for details on the curve sequence, and Table 1 for the X59 representation of it.

Remark 53. For a valid initialization, the q_{spacing} from curveArray must match the one implied from kernelCompactArray.

- hookData: Data to be passed to the initialize hooks.

Remark 54. The initial curve sequence is supplied by the pool creator and determines q_{spacing} which should not be less than 2^{40} in the X59 representation.⁷²

Remark 55. When a curve sequence is used to initialize a pool, $-16 < q^0, q^1 < 16$ holds as in Remark 7 and Table 1. The rightmost and leftmost liquidity intervals of this entire horizon are regarded as blank liquidity intervals and should be avoided, i.e., there should be at least 3 intervals. Therefore, $q_{\text{spacing}} = |q^0 - q^1| < \frac{32}{3}$.

Remark 56. One candidate for the initial curve sequence (consistent with Remark 13) is q^0, q^1, q^2 , where:

⁷²The corresponding log-price value is $q_{\text{spacing}} = \frac{2^{40}}{2^{59}}$. The corresponding price value is $p_{\text{spacing}} = \exp\left(\frac{2^{40}}{2^{59}}\right) \approx 1.0000019$, which is equal to $\frac{p_{\text{upper}}}{p_{\text{lower}}}$ for each liquidity interval.

- **Pool price:** q^2 is the market log-price at the time of initialization.
- q_{spacing} **selection (permanent impactful influence):** $|q^1 - q^0|$, which determines q_{spacing} , is selected based on the economic characteristics of the tokens, such as price volatility, as well as the view and strategy of the creator of the pool.
- **Curve shape (temporary influence):**
 - * Select $q^0 < q^2 < q^1$ if the historical price has reached q^2 from above
 - * Select $q^1 < q^2 < q^0$ if the historical price has reached q^2 from below

Remark 57. The selection of q_{spacing} is generally more material than the initial curve sequence because the initial curve sequence is not permanent. However,

- For pools where q_{spacing} is relatively small and it is expected that the liquidity intervals are transitioned frequently, some $q^0, q^1, q^2 (= q_{\text{current}})$ can be simply selected as the curve sequence, at initialization, where $|q^0 - q^1|$ is selected equal to the desired q_{spacing} . The curve sequence is expected to reset shortly after an interval transition.
- For pools where q_{spacing} is relatively large, the choice of initial curve sequence is more material, as the curve sequence resets infrequently and the implications of its initial value last longer. For example, if $q_{\text{spacing}} = 5$, assuming that the historical market log-prices over the last few weeks peaked at 7 (up), 2 (down), and 4 (up), in chronological order, with the current log-price of 3, the curve sequence can be chosen as 7, 2, 4, 3.

Example 7. Assume that it is intended to create a new pool for ETH-WBTC, when the market price of WBTC is exactly 20 times that of ETH. Note that WBTC has 8 decimals and ETH has 18 decimals. Hence, the smallest unit of WBTC is 10^{-8} and the smallest unit for ETH is 10^{-18} . Therefore, the price in the protocol should be around:

$$\frac{\text{price of each unit of WBTC (tag1)}}{\text{price of each unit of ETH (tag0)}} = 20 \times 10^{18-8} = 2 \times 10^{11}. \quad (79)$$

If the pool creator intends to make p_{offset} around 2×10^{11} , then the corresponding `int8` representation of `logOffset` can be calculated using

SymPy with command `floor(log(2*(10**11)))` which is equal to 26, i.e., $q_{\text{offset}} = 26$ and $p_{\text{offset}} = e^{26}$.⁷³

Therefore, $q_{\text{current}} = \log_e(2 \times 10^{11}) - q_{\text{offset}} \sim 0.02$, with X59 offset-binary representation of (according to Table 1):

`hex(floor((2**63)+(2**59)*(log(2*(10**11))-26)))`, which is equal to `0x802c33d08936686c`. This is selected as the last member of the curve sequence.

According to Remark 7, price of the real-world pool can move to any value within $e^{-16}p_{\text{offset}}$ and $e^{+16}p_{\text{offset}}$, where in this example, p_{offset} is a number close to the market price at initialization.

Example 8. Each member of the curve sequence is a 64-bit representing a log-price value in offset-binary X59 representation. Four members are packed in each slot of the input array. In Example 7, if there are 5 members,

$$\begin{aligned} - q^0 &= q_{\text{current}} + 1 \implies 0x882c33d08936686c \implies p = e^{+1}p_{\text{current}} \\ - q^1 &= q_{\text{current}} - 1 \implies 0x782c33d08936686c \implies p = e^{-1}p_{\text{current}} \\ - q^2 &= q_{\text{current}} + \frac{1}{2} \implies 0x842c33d08936686c \implies p = e^{+\frac{1}{2}}p_{\text{current}} \\ - q^3 &= q_{\text{current}} - \frac{1}{2} \implies 0x7c2c33d08936686c \implies p = e^{-\frac{1}{2}}p_{\text{current}} \\ - q^4 &= q_{\text{current}} \implies 0x802c33d08936686c \implies p = p_{\text{current}} \end{aligned}$$

These can be derived using price relationships in Table 1. Note that this example also has an offset multiplier of $p_{\text{offset}} = e^{26}$. The corresponding slots of this curve sequence are illustrated in Figure 23.

Observe that $q_{\text{spacing}} = |q_{\text{upper}} - q_{\text{lower}}| = |q^0 - q^1| = 2$, and, the kernel's last log-price must be consistent.

Example 9. q_{spacing} in Example 8 is too large for most practical cases. Its value should be derived based on the historical fluctuations of that token pair's market price, and the pool creator's preference on how often it is desired to have transitions between liquidity intervals and reset curve sequence.

Assume a case quite similar to Example 8, but where the pool creator intends to make $\frac{p_{\text{upper}}}{p_{\text{lower}}} \sim 1.004$, which means that a price move from one

⁷³See Remark 47.

	q^0	q^1	q^2	q^3
slot 1	882c33d08936686c	782c33d08936686c	842c33d08936686c	7c2c33d08936686c
	64 bits	64 bits	64 bits	64 bits
	q^4			
slot 2	802c33d08936686c	0000000000000000	0000000000000000	0000000000000000
	64 bits	64 bits	64 bits	64 bits

Figure 23: The curve sequence of Example 8 in initialization.

interval boundary to the other leads to a 0.4% price change. Then, $q_{\text{spacing}} = |q^0 - q^1| = 2^{-8}$ can be selected⁷⁴, and therefore, $2^{-8+59} = 2^{51}$ in X59 representation, as in Table 2. If there are 5 members in curve sequence as before,

$$\begin{aligned}
- q^0 &= q_{\text{current}} + 2^{-9} \implies p = e^{+2^{-9}} p_{\text{current}} \\
- q^1 &= q_{\text{current}} - 2^{-9} \implies p = e^{-2^{-9}} p_{\text{current}} \\
- q^2 &= q_{\text{current}} + 2^{-10} \implies p = e^{+2^{-10}} p_{\text{current}} \\
- q^3 &= q_{\text{current}} - 2^{-10} \implies p = e^{-2^{-10}} p_{\text{current}} \\
- q^4 &= q_{\text{current}} \implies p = p_{\text{current}}
\end{aligned}$$

The price ratio of the two boundaries of a liquidity interval is $e^{(2^{-8})} \sim 1.004$, leading to $\sim 0.4\%$ price difference between the two.

- `modifyPosition`. Mints/burns within a concentrated liquidity interval.
 - `poolId`: Identifier of the target pool.
 - `logPriceMin`: The left (smaller) position boundary which must be equal to the boundaries of the active liquidity interval modulo q_{spacing} .
 - `logPriceMax`: The right (larger) position boundary which must be equal to the boundaries of the active liquidity interval modulo q_{spacing} .
 - `shares`: The number of shares to be minted (positive)/burned (negative) per interval. The call applies `shares` count to **each** of $k = \frac{q_{\text{max}} - q_{\text{min}}}{q_{\text{spacing}}}$ intervals.

⁷⁴since $2^{-8} \sim \log_e(1.004)$

- For a burn, the caller’s transient balance for the LP position tag may not become positive. The burn must therefore be backed by a corresponding LP balance debit or transient-balance transfer within the unlock.
- `hookData`: Data to be passed to hooks.
- `[output] amount0/1`: The amount of `tag0/1`. Positive means they are deposited into the pool and negative means they are withdrawn from the pool.

Remark 58. *To create a new position, the selected left and right log-price boundaries, $q_{\min}$ and $q_{\max}$, must be in the form of $q_{\text{lower}} + kq_{\text{spacing}}$, each for an integer k . Additionally, $-16 + q_{\text{spacing}} < q_{\min} < q_{\max} < 16 - q_{\text{spacing}}$ must hold, since the rightmost and leftmost liquidity intervals of this entire horizon are regarded as blank liquidity intervals and should be avoided, as in Remark 55.*

Tables 1 and 2 show the differences of different representations. Table 1 shows how to convert the offset-adjusted offset-binary representation of curve sequence log-prices to the real-world log-prices.

Remark 59. *If the first two members of curve sequence are q^0 and q^1 , then, $q_{\text{lower}} = \min\{q^0, q^1\}$, $q_{\text{upper}} = \max\{q^0, q^1\}$ and $q_{\text{spacing}} = |q^0 - q^1|$. The first two members of the curve sequence can be retrieved for this purpose using `readBoundaries` in `Storage.sol`.*

Remark 60. *The incoming and outgoing token `modifyPosition` amounts in any arbitrary range can be calculated as follows. Note that q_{lower} and q_{upper} represent the boundaries of the active liquidity interval.*

- Assuming $q_{\text{upper}} \leq q_{\min}$, for a single share in every interval from $q_{\min}$ to $q_{\max}$, the corresponding amount of `token0` is equal to:⁷⁵

$$\text{amount0} = \frac{1}{\sqrt{p_{\text{offset}}}} (\text{growthMultiplier}[q_{\min}] - \text{growthMultiplier}[q_{\max}]).$$

- Similarly, assuming $q_{\max} \leq q_{\text{lower}}$, the amount of `token1` correspond-

⁷⁵See equation (65).

ing to a single share in every interval from $q_{\min}$ to $q_{\max}$ is equal to:⁷⁶

$$\text{amount1} = \sqrt{p_{\text{offset}}}(\text{growthMultiplier}[q_{\max}] - \text{growthMultiplier}[q_{\min}]).$$

- Lastly, the token amount in the active liquidity interval corresponding to any specific number of shares is derived through equation (57).
- `donate`. Donates the token amounts equivalent to a number of shares to be distributed proportionally among all the LPs in the current active liquidity interval. See Remark 21.
 - `poolId`: Target pool’s identifier.
 - `shares`: The number of shares to be donated.
 - `hookData`: Data to be passed to donate hooks.
 - `[output] amount0/1`: These are the donated amounts of `tag0/1`.
- `modifyProtocol`. Sets the values `maxPoolGrowthPortion`, as well as `protocolGrowthPortion`, and the address of the protocol owner which are all in one slot. See Section 3.2.5. Must be called by the current protocol owner, which is `timelock` contract, i.e., through the protocol’s governance.
 - `protocol`: The new value for the protocol slot.
- `modifySentinel`. Sets `Sentinel` contract. Must be called by the current protocol owner..
 - `sentinel`: New sentinel contract.
- `modifyPoolOwner`. Sets a new pool owner for the given `poolId`. It must be called by the current pool owner.
 - `poolId`: The identifier of the target pool.
 - `newOwner`: The new owner’s address.

⁷⁶See equation (66).

- `modifyKernel`. Provides a new pending kernel for a given pool, to be activated when the next liquidity interval transition happens. If there is already a pending kernel in that pool, it replaces it. It must be called by the pool owner.
 - `poolId`: The identifier of the target pool.
 - `kernelCompactArray`: The new kernel array in its compact form.
 - `hookData`: Data to be passed to the hook.
- `modifyPoolGrowthPortion`. Changes `poolGrowthPortion` to a proposed amount. This is governed by the `Sentinel` contract, i.e., the protocol can receive some tokens to permit increasing this. It must be called by the pool owner.⁷⁷
 - `poolId`: The identifier of the target pool.
 - `poolGrowthPortion`: The new pool growth portion.
- `updateGrowthPortions`. Syncs `protocolGrowthPortion` and `maxPoolGrowthPortion` for a pool to the latest `Sentinel` value or protocol owner. It can be called by any address.
 - `poolId`: The identifier of the target pool.
- `collectPool`. Collects pool owner's accrued growth portions. The collected amounts are transferred to the pool owner's singleton single balances. It can be called by any address.
 - `poolId`: The target pool's identifier.
 - `[output] amount0/1`: The total amount of `tag0/1` collected.
- `collectProtocol`. Collects protocol owner's accrued growth portions. The collected amounts are transferred to the protocol owner's singleton single balances. It can be called by any address.
 - `poolId`: The target pool's identifier.
 - `[output] amount0/1`: The total amount of `tag0/1` collected.

⁷⁷Note that the effect of `poolGrowthPortion` is capped by `maxPoolGrowthPortion`.

4 Operator (optional tool)

An operator is provided to make a sequence of calls to the protocol. Any alternative operator can be used for the calls as well.

A sequence of compactly encoded instructions from the following list can be given to the operator contract to be executed in order. The first 4 bytes of data are a `uint32` deadline, followed by the encoded instructions. Actions that refer to the payer use the `unlockCallback` caller as the payer. It is designed to give high flexibility to the users.

0. **PUSH0:** Places `0x0` in the transient storage slot `valueSlot` (clears it).

```
abi.encodePacked(uint8 action, uint8 valueSlot)
```

1. **PUSH10:** Places `signextend(9, value)` in the transient storage slot `valueSlot`.

```
abi.encodePacked(uint8 action, int80 value, uint8 valueSlot)
```

2. **PUSH16:** Places `signextend(15, value)` in the transient storage slot `valueSlot`.

```
abi.encodePacked(uint8 action, int128 value, uint8 valueSlot)
```

3. **PUSH32:** Places `value` in the transient storage slot `valueSlot`.

```
abi.encodePacked(uint8 action, int256 value, uint8 valueSlot)
```

4. **NEG:** Negates the content of the given transient storage slot, `valueSlot`, and inserts the outcome in `resultSlot`.

```
abi.encodePacked(  
    uint8 action, uint8 valueSlot, uint8 resultSlot  
)
```

5. **ADD:** Calculates the addition of `value0Slot` and `value1Slot` contents, and inserts the outcome in `resultSlot`. Throws in case of overflow or

underflow.

```
abi.encodePacked(  
    uint8 action,  
    uint8 value0Slot,  
    uint8 value1Slot,  
    uint8 resultSlot  
)
```

6. SUB: Calculates the subtraction value0Slot and value1Slot contents, and inserts the outcome in resultSlot. Throws in case of overflow or underflow.

```
abi.encodePacked(  
    uint8 action,  
    uint8 value0Slot,  
    uint8 value1Slot,  
    uint8 resultSlot  
)
```

7. MIN: Calculates the minimum of two signed integer values, value0Slot and value1Slot, and inserts the outcome in resultSlot.

```
abi.encodePacked(  
    uint8 action,  
    uint8 value0Slot,  
    uint8 value1Slot,  
    uint8 resultSlot  
)
```

8. MAX: Calculates the maximum of two signed integers, value0Slot and

value1Slot, and inserts the outcome in resultSlot.

```
abi.encodePacked(
    uint8 action,
    uint8 value0Slot,
    uint8 value1Slot,
    uint8 resultSlot
)
```

9. **MUL:** Calculates the multiplication value0Slot and value1Slot contents, and inserts the outcome in resultSlot. Throws in case of overflow or underflow.

```
abi.encodePacked(
    uint8 action,
    uint8 value0Slot,
    uint8 value1Slot,
    uint8 resultSlot
)
```

10. **DIV:** Calculates the division of value0Slot by value1Slot contents, and inserts the outcome in resultSlot. It is rounded towards the origin. It throws if the denominator is zero or if the operation is the division of type (int256).min by -1.

```
abi.encodePacked(
    uint8 action,
    uint8 value0Slot,
    uint8 value1Slot,
    uint8 resultSlot
)
```

11. **DIV_ROUND_DOWN:** Calculates the division of value0Slot by value1Slot contents, rounds the result down, and inserts the outcome in resultSlot.

It throws if the denominator is zero or if the operation is the division of `type(int256).min` by `-1`.

```
abi.encodePacked(
    uint8 action,
    uint8 value0Slot,
    uint8 value1Slot,
    uint8 resultSlot
)
```

12. **DIV_ROUND_UP**: Calculates the division of `value0Slot` by `value1Slot` contents, rounds the result up, and inserts the outcome in `resultSlot`. It throws if the denominator is zero or if the operation is the division of `type(int256).min` by `-1`.

```
abi.encodePacked(
    uint8 action,
    uint8 value0Slot,
    uint8 value1Slot,
    uint8 resultSlot
)
```

13. **LT**: Checks if `value0Slot` is less than `value1Slot` using signed comparison, and inserts the boolean outcome in `resultSlot`.

```
abi.encodePacked(
    uint8 action,
    uint8 value0Slot,
    uint8 value1Slot,
    uint8 resultSlot
)
```

14. **EQ**: Checks if `value0Slot` is equal to `value1Slot`, and inserts the

boolean outcome in resultSlot.

```
abi.encodePacked(  
    uint8 action,  
    uint8 value0Slot,  
    uint8 value1Slot,  
    uint8 resultSlot  
)
```

15. **LTEQ:** Checks if value0Slot is less than or equal to value1Slot using signed comparison, and inserts the boolean outcome in resultSlot.

```
abi.encodePacked(  
    uint8 action,  
    uint8 value0Slot,  
    uint8 value1Slot,  
    uint8 resultSlot  
)
```

16. **ISZERO:** Checks if valueSlot is equal to zero, and inserts the boolean outcome in resultSlot.

```
abi.encodePacked(  
    uint8 action, uint8 valueSlot, uint8 resultSlot  
)
```

17. **AND:** Calculates AND of two booleans, value0Slot and value1Slot, and inserts the resulting bits in resultSlot.

```
abi.encodePacked(  
    uint8 action,  
    uint8 value0Slot,  
    uint8 value1Slot,  
    uint8 resultSlot  
)
```

18. OR: Calculates OR of two booleans, `value0Slot` and `value1Slot`, and inserts the resulting bits in `resultSlot`.

```
abi.encodePacked(
    uint8 action,
    uint8 value0Slot,
    uint8 value1Slot,
    uint8 resultSlot
)
```

19. XOR: Calculates XOR of two booleans, `value0Slot` and `value1Slot`, and inserts the resulting bits in `resultSlot`.

```
abi.encodePacked(
    uint8 action,
    uint8 value0Slot,
    uint8 value1Slot,
    uint8 resultSlot
)
```

20. JUMPDEST: Marks a valid destination for JUMP and has no other effect.

```
abi.encodePacked(uint8 action)
```

21. JUMP: If the content of `conditionSlot` is non-zero, it jumps to the operation at byte destination. Throws if destination refers to anything but a JUMPDEST.

```
abi.encodePacked(
    uint8 action, uint16 destination, uint8 conditionSlot
)
```

22. READ_TRANSIENT_BALANCE: Reads a tag's transient balance of an address in the protocol, and inserts the outcome in `resultSlot`.

```
abi.encodePacked(
    uint8 action, Tag tag, address owner, uint8 resultSlot
)
```

23. **READ_BALANCE_OF_NATIVE:** Reads the native balance of an address and inserts the outcome in `resultSlot`.

```
abi.encodePacked(  
    uint8 action, address owner, uint8 resultSlot  
)
```

24. **READ_BALANCE_OF_ERC20:** Reads an ERC-20 balance of an address and inserts the outcome in `resultSlot` (together with the `successSlot` flag that shows if the action succeeded).⁷⁸

```
abi.encodePacked(  
    uint8 action,  
    address token,  
    address owner,  
    uint8 successSlot,  
    uint8 resultSlot  
)
```

25. **READ_BALANCE_OF_MULTITOKEN:** Reads a multi-token balance of an address and inserts the outcome in `resultSlot` (together with the flag that shows if the action succeeded in `successSlot`).

```
abi.encodePacked(  
    uint8 action,  
    address token,  
    uint256 id,  
    address owner,  
    uint8 successSlot,  
    uint8 resultSlot  
)
```

⁷⁸The `resultSlot` stores the value returned by the contract, and the `successSlot` stores the low-level call success flag.

26. `READ_ALLOWANCE_ERC20`: Reads the ERC-20 allowance granted by owner to spender.

```
abi.encodePacked(
    uint8 action,
    address token,
    address owner,
    address spender,
    uint8 successSlot,
    uint8 resultSlot
)
```

27. `READ_ALLOWANCE_PERMIT2`: Reads the ERC-20 allowance granted by owner to spender via permit2.

```
abi.encodePacked(
    uint8 action,
    address token,
    address owner,
    address spender,
    uint8 resultSlot
)
```

28. `READ_ALLOWANCE_ERC6909`: Reads the ERC-6909 allowance granted by owner to spender for id.

```
abi.encodePacked(
    uint8 action,
    address token,
    uint256 id,
    address owner,
    address spender,
    uint8 successSlot,
    uint8 resultSlot
)
```


29. `READ_IS_OPERATOR_ERC6909`: Reads a spender's operator status for an ERC-6909 token of the owner.

```
abi.encodePacked(
    uint8 action,
    address token,
    address owner,
    address spender,
    uint8 successSlot,
    uint8 resultSlot
)
```

30. `READ_IS_APPROVED_FOR_ALL_ERC1155`: Reads whether spender is approved for all ERC-1155 tokens of owner.

```
abi.encodePacked(
    uint8 action,
    address token,
    address owner,
    address spender,
    uint8 successSlot,
    uint8 resultSlot
)
```

31. `WRAP_NATIVE`: Wraps ETH to WETH.

```
abi.encodePacked(
    uint8 action,
    uint8 valueSlot,
    uint8 successSlot
)
```

32. UNWRAP_NATIVE: Unwraps WETH to ETH.

```
abi.encodePacked(
    uint8 action,
    uint8 amountSlot,
    uint8 successSlot
)
```

33. PERMIT_PERMIT2: Grants permit2 allowance to spender for a given amount of the owner's token via the owner's EIP-712 signature.

```
abi.encodePacked(
    uint8 action,
    address owner,
    uint48 nonce,
    uint8 amountSlot,
    address token,
    uint48 expiration,
    uint48 signatureDeadline,
    address spender,
    uint8 successSlot,
    uint8 signatureByteCount,
    bytes signature
)
```

34. PERMIT_BATCH_PERMIT2: Grants permit2 allowance for a batch of

tokens owned by owner to spender via the owner's EIP-712 signature.

```
abi.encodePacked(
    uint8 action,
    address owner,
    uint48 signatureDeadline,
    uint8 signatureByteCount,
    address spender,
    uint8 successSlot,
    uint8 numberOfPermissions,
    bytes permissions,
    bytes signature
)
```

where permissions is the packed concatenation of (address token, uint8 amountSlot, uint40 expiration, uint48 nonce) for each token.

35. `TRANSFER_NATIVE`: Transfers an amount of the native token to an address.

```
abi.encodePacked(
    uint8 action,
    address to,
    uint8 amountSlot,
    uint8 successSlot
)
```

36. `TRANSFER_FROM_PAYER_ERC20`: Transfers an amount of an ERC-20

token to an address, together with a flag that shows if the action succeeded.

```
abi.encodePacked(
    uint8 action,
    address token,
    uint8 amountSlot,
    address to,
    uint8 successSlot,
    uint8 resultSlot
)
```

37. `TRANSFER_FROM_PAYER_PERMIT2`: Transfers an amount of an ERC-20 token to an address through Permit2.

```
abi.encodePacked(
    uint8 action,
    address to,
    uint8 amountSlot,
    address token,
    uint8 successSlot
)
```

38. `TRANSFER_FROM_PAYER_ERC6909`: Transfers an amount of an ERC-6909 token to an address.

```
abi.encodePacked(
    uint8 action,
    address token,
    uint256 id,
    address to,
    uint8 amountSlot,
    uint8 successSlot,
    uint8 resultSlot
)
```

39. `SAFE_TRANSFER_FROM_PAYER_ERC1155`: Transfers an amount of an ERC-1155 token to an address.

```
abi.encodePacked(
    uint8 action,
    address token,
    uint256 id,
    address to,
    uint8 amountSlot,
    uint8 successSlot,
    uint24 dataByteCount,
    bytes data
)
```

40. `CLEAR`: Clears an amount of the operator's positive transient balance in tag, i.e., gives up that amount without taking it back.

```
abi.encodePacked(
    uint8 action,
    Tag tag,
    uint8 amountSlot,
    uint8 successSlot
)
```

41. `TAKE_TOKEN`: Takes an amount of a native or ERC-20 token from the protocol (transient balance) and gives it to a specified address. The native token is represented by `address(0)`.

```
abi.encodePacked(
    uint8 action,
    address token,
    address to,
    uint8 amountSlot,
    uint8 successSlot
)
```

42. TAKE_ERC6909: Takes an amount of an ERC-6909 token from the protocol (transient balance) and gives it to a specified address.

```
abi.encodePacked(
    uint8 action,
    address token,
    uint256 id,
    address to,
    uint8 amountSlot,
    uint8 successSlot
)
```

43. TAKE_ERC1155: Takes an amount of an ERC-1155 token from the protocol (transient balance) and gives it to a specified address.

```
abi.encodePacked(
    uint8 action,
    address token,
    uint256 id,
    address to,
    uint8 amountSlot,
    uint8 successSlot,
    uint24 dataByteCount,
    bytes data
)
```

44. SYNC_TOKEN: Syncs an ERC-20 token.

```
abi.encodePacked(
    uint8 action,
    address token
)
```

45. SYNC_MULTITOKEN: Syncs a multi-token.

```
abi.encodePacked(uint8 action, address token, uint256 id)
```

46. **SETTLE:** Should be used after ERC-20, ERC-6909, or ERC-1155 transfers to Nofeeswap in order to update the operator's transient balances. Can be used to send a value of native tokens to the protocol to settle the native transient balance of the operator. It stores the amount paid in `resultSlot`.

```
abi.encodePacked(
    uint8 action,
    uint8 valueSlot,
    uint8 successSlot,
    uint8 resultSlot
)
```

47. **TRANSFER_TRANSIENT_BALANCE:** Transfers a positive amount of transient balances of the operator to a receiver.

```
abi.encodePacked(
    uint8 action,
    Tag tag,
    address receiver,
    uint8 amountSlot,
    uint8 successSlot
)
```

48. **TRANSFER_TRANSIENT_BALANCE_FROM_PAYER:** Transfers a positive amount of transient balances of the payer to a receiver. Can be used to settle on behalf of other accounts.

```
abi.encodePacked(
    uint8 action,
    Tag tag,
    address receiver,
    uint8 amountSlot,
    uint8 successSlot
)
```

49. **MODIFY_BALANCE:** Modifies (in either direction) the corresponding singleton balance of a tag by a specified amount.

```
abi.encodePacked(
    uint8 action,
    Tag tag,
    uint8 amountSlot,
    uint8 successSlot
)
```

50. **MODIFY_MULTI_BALANCE:** Modifies (in either direction) the corresponding singleton multi balance by specified amounts. The payer and the operator must both be listed among the spenders.

```
abi.encodePacked(
    uint8 action,
    uint8 payerIndex,
    uint8 operatorIndex,
    uint8 successSlot,
    uint8 numberOfSpenders,
    uint8 numberOfTags,
    uint8 numberOfAmounts,
    abi.encodePacked(address[] spenders),
    abi.encodePacked(Tag[] tags),
    abi.encodePacked(uint8[] balanceSlots),
    abi.encodePacked(uint8[] indices),
    abi.encodePacked(uint8[] amountSlots)
)
```


51. **SWAP**: Performs a `swap` by taking the inputs.

```
abi.encodePacked(  
    uint8 action,  
    uint256 poolId,  
    uint8 amountSpecifiedSlot,  
    uint64 limitOffsetted,  
    uint8 zeroForOne,  
    uint8 transitionThresholdSlot,  
    uint8 successSlot,  
    uint8 amount0Slot,  
    uint8 amount1Slot,  
    uint16 hookDataBytesCount,  
    bytes hookData  
)
```

52. **MODIFY_POSITION**: Performs a `modifyPosition` by taking the inputs.

```
abi.encodePacked(  
    uint8 action,  
    uint256 poolId,  
    uint64 qMinOffsetted,  
    uint64 qMaxOffsetted,  
    uint8 sharesSlot,  
    uint8 successSlot,  
    uint8 amount0Slot,  
    uint8 amount1Slot,  
    uint16 hookDataBytesCount,  
    bytes hookData  
)
```

If the value in `sharesSlot` is negative, prior actions must make the operator's transient balance for the LP position tag cover the burn.

53. DONATE: Performs a donate by taking the inputs.

```
abi.encodePacked(
    uint8 action,
    uint256 poolId,
    uint8 sharesSlot,
    uint8 successSlot,
    uint8 amount0Slot,
    uint8 amount1Slot,
    uint16 hookDataBytesCount,
    bytes hookData
)
```

54. QUOTE_SWAP: Similar to swap, but it only quotes.

55. QUOTE_MODIFY_POSITION: Similar to modifyPosition, but it only quotes.

56. QUOTE_DONATE: Similar to donate, but it only quotes.

57. QUOTER_TRANSIENT_ACCESS: Reads a transient storage slot of the quoter.

```
abi.encodePacked(
    uint8 action,
    bytes32 transientSlot,
    uint8 resultSlot
)
```

58. REVERT: Reverts with the current content of returndata as the reason.

Remark 61. When `INofeeswap.unlock` is called with `Operator.sol` as `unlockTarget`, the operator receives data through its `unlockCallback` implementation. The data payload starts with a `uint32` deadline and is followed by the encoded instruction sequence. See Remark 51.

5 Pre-built Hooks

The pre-built hook described here is `PointIncentive`. It measures LP participation with points and represents each incentivized position with an ERC-721 token. The hook does not distribute ERC-20 tokens, does not contain a reward token, does not call a paymaster, and does not deploy per-token-pair hook contracts. Reward distribution is separated from point accounting and can be performed later by an external Merkle-tree distributor.

5.1 PointIncentive Hook Contract

`PointIncentive` is a single reusable hook. A pool subscribes to it at initialization by using the hook address in its `poolId`. Eligibility is not based on the token tags of the pool. Therefore, the same hook contract can be used by arbitrary token pairs, provided that the pool's hook flags are compatible with the hook callbacks needed by `PointIncentive`.

The hook holds the `NoFeeSwap` LP tag created by `modifyPosition` and mints an ERC-721 token to represent the corresponding incentivized position. The LP's core position remains in the hook, while the LP owns an NFT whose `tokenId` identifies the incentivized position. The NFT itself is not a `NoFeeSwap` tag. The hook supports ERC-721 permit and multicall through its inherited interfaces.

5.1.1 Point Accounting

Points are the criterion for measuring each NFT position's participation in the point system. The point accounting is independent of the reward asset and independent of the later allocation policy.

For each block and for each subscribing pool, LPs of the active liquidity interval earn points proportional to the following value per share:⁷⁹

$$\frac{\text{growth} \times \sqrt{\text{integral0} \times \text{integral1}}}{\text{outgoingMax}}. \quad (80)$$

If the price is on the boundary of the active interval, then one of `integral0` or `integral1` is zero and no points are earned for that block.

⁷⁹See pool's dynamic parameters in Section 3.2.2 and static parameters in Section 3.2.3.

Remark 62. *For each LP, the parameters in equation (80) are from their corresponding pool. For example, if at some block, the value of this parameter for the pool X is twice that of the pool Y , the LPs in the active interval of the pool X earn twice as many points per share per block compared to the LPs in the active interval of pool Y .*

The hook maintains cumulative points-per-share values for each subscribing pool and each accounting period. When the active interval changes, the cumulative values at the crossed interval boundaries are updated so that a position can later recover the points earned across the intervals it covered. This mapping is analogous to the growth multiplier described in Section 3.3.2, but it accumulates point weight instead of liquidity growth.

5.1.2 Accounting Periods

Point accounting is divided into block-based periods. The period containing a block number b is the period-start block:

$$\text{period}(b) = b - (b \bmod \text{periodLength}). \quad (81)$$

The deployment parameter `periodLength` is nonzero and fits in 32 bits. The deployment parameter `endPeriod` also fits in 32 bits and must itself be a period-start block, i.e., `endPeriod` is divisible by `periodLength`. No points are accounted for the period whose start is `endPeriod`, nor for any later period.

The current, still-live period is stored in `poolData`. A finalized period is stored in `periodData`. The hook only writes `periodData` when a period has ended and the pool is touched again by a mint, burn, swap, or donation. Therefore, the current live period is not final and should not be used for final reward allocation.

Remark 63. *The hook does not copy finalized data into every untouched intermediate period. If a pool is touched in period p and next touched in period n , the hook finalizes period p and stores `nextPeriod = n`. Every period start r with $p \leq r < n$ follows the same finalized period data. If a pool is first touched again at or after `endPeriod`, the previous touched period points to `endPeriod`.*

5.1.3 PointIncentive Contract's Parameters

5.1.3.1 Immutable Parameters. They are set when the `PointIncentive` contract is deployed.

- `nofeeswap`. NoFeeSwap's contract address.
- `periodLength`. Length of each point accounting period in blocks.
- `endPeriod`. First period for which points are not accounted. Periods are expressed as period-start block numbers.

5.1.3.2 Pool-level Parameters. They are stored in `poolData` and updated when a subscribing pool is touched by `swap`, `modifyPosition`, or `donate`. This data describes the live period.

- `blockNumber`. Last block number where the pool is accounted.
- `qLower`. Most recent lower boundary of the active liquidity interval.
- `qUpper`. Most recent upper boundary of the active liquidity interval.
- `activePointsPerShare`. Points per share for the active liquidity interval.

5.1.3.3 Period-level Parameters. They are stored in `periodData` after a period has been finalized.

- `qLower`. Final lower boundary used for this finalized period's point calculation.
- `qUpper`. Final upper boundary used for this finalized period's point calculation.
- `activePointsPerShare`. Final points per share for the active liquidity interval.
- `nextPeriod`. The next initialized period. Intermediate periods before `nextPeriod` use the same finalized data.

5.1.3.4 Position-level Parameters.

They are stored for each ERC-721 `tokenId` and updated when the position is minted, increased, or decreased.

- `poolId`. The pool identifier for the corresponding position.

- `qMin`. Left boundary of the corresponding position, stored in offsetted log-price coordinates.
- `qMax`. Right boundary of the corresponding position, stored in offsetted log-price coordinates.
- `shares`. The number of shares per liquidity interval for the corresponding position.

5.1.3.5 Points per share.

The value of the `pointsPerShareMapping(poolId, period, q)` mapping is equal to:⁸⁰

- If a spaced log-price q is less than or equal to q_{lower} : The total points per single share for a position from $-\infty$ to q .
- If a spaced log-price q is more than or equal to q_{upper} : The total points per single share for a position from q to $+\infty$.

For readability, write $M_p(q)$ for this mapping value. For a position with range $[q_{\min}, q_{\max}]$, the points per share in a finalized period are computed from the finalized period data and the two boundary mapping values at $q_{\min}$ and $q_{\max}$. If the finalized active interval is behind the position, then:

$$\text{pointsPerShare} = M_p(q_{\min}) - M_p(q_{\max}). \quad (82)$$

If the finalized active interval is ahead of the position, then:

$$\text{pointsPerShare} = M_p(q_{\max}) - M_p(q_{\min}). \quad (83)$$

If the active interval intersects the position, then:

$$\begin{aligned} \text{pointsPerShare} = & \text{activePointsPerShare} \\ & + M_p(q_{\text{upper}}) - M_p(q_{\max}) \\ & + M_p(q_{\text{lower}}) - M_p(q_{\min}). \end{aligned} \quad (84)$$

⁸⁰This mapping is similar to the growth multiplier in Section 3.3.2.

5.1.4 NFT Position Flow

The hook expects the `hookData` for `modifyPosition` to contain two words:

```
abi.encode(uint256 tokenId, address recipient). (85)
```

For a mint, if `tokenId` is zero, the hook creates a new ERC-721 token for `recipient`. The newly minted NFT stores the pool identifier, the position's offsetted log-price range, and the number of shares. The constructor initializes the token counter so that the first minted NFT has `tokenId = 1`.

If `tokenId` is nonzero, the hook requires `recipient` to be the owner of that NFT. It then verifies that the stored `poolId`, `qMin`, and `qMax` match the current `modifyPosition` calldata and adjusts the stored shares. The adjustment cannot reduce the shares below zero.

On mint, after accounting points and updating the NFT position, the hook calls `modifyBalance` so that the hook receives the NoFeeSwap LP tag corresponding to the minted shares. On burn, after accounting points and reducing the NFT shares, the hook releases the LP tag through transient balance transfer so that the owner can burn the corresponding NoFeeSwap position through the core flow. The NFT itself is not burned when its stored share count reaches zero.

Each change to the number of shares emits:

```
PointIncentiveSharesModified(tokenId, sharesBefore, sharesAfter). (86)
```

This event is the position-level signal used by off-chain accounting to determine the share history of each NFT.

5.1.5 External and Hook Functions

For ordinary pool actions, the external functions in the core⁸¹ are still used. The `PointIncentive` hook observes those actions through the hook callbacks and updates point accounting.

- `preInitialize`. Initializes pool-level point accounting for a subscribing pool. The hook records the first accounted block and the initial active interval boundaries.

⁸¹i.e., `NoFeeSwap.sol` and `NoFeeSwapDelegatee.sol`

- `midMint`. For a mint, the hook accounts points for the pool, creates or updates the point-incentive NFT position, and receives the NoFeeSwap LP tag for the minted shares. The caller still settles the corresponding token debits through the usual protocol flow or through the operator discussed in Section 4.
- `midBurn`. For a burn, the hook accounts points for the pool, reduces the shares of the corresponding point-incentive NFT position, and releases the NoFeeSwap LP tag needed for the burn. The NFT is not burned by this action.
- `midSwap`. Updates point accounting after a swap changes the pool state.
- `midDonate`. Updates point accounting after a donation changes the pool state.

Remark 64. *For `modifyPosition`, the `PointIncentive` contract performs its NFT update in `midMint` and `midBurn`. The corresponding debit or credit goes to singleton accounting for the caller to settle on the hook's behalf. The caller can settle those mint and burn actions together with other actions using tools like the operator discussed in Section 4.*

5.1.6 Merkle-Tree Reward Allocation

The current `PointIncentive` hook does not contain a `collect` function and does not store a per-NFT `pointsOwed` value. Instead, it stores enough pool-level and NFT-level data for point balances to be reconstructed for finalized periods. A reward program can later compute the points earned by each NFT `tokenId` off-chain and publish a Merkle root in a separate distributor contract.

For a target period r , the off-chain accounting process first locates the finalized source period $p \leq r$ whose `nextPeriod` is greater than r . If period r itself was finalized when the pool was touched later, then $p = r$. If period r was an untouched intermediate period, then p is the most recent touched period before r . The accounting process can then use:

- the finalized `periodData(poolId, p)` record;
- the $M_p(q)$ values needed for each NFT's `qMin` and `qMax`;
- the NFT's stored `poolId`, `qMin`, `qMax`, and share history;

- the `PointIncentiveSharesModified` events, together with archive-node state at the relevant transactions, to determine when each NFT's share count changed.

Because `PointIncentive` accounts pool points before changing an NFT's shares in `midMint` and `midBurn`, the transaction order determines which share count applies to the just-accounted range and which share count applies afterward.

Points are first allocated to NFT `tokenIds`, not directly to addresses. A Merkle leaf may therefore identify the chain, the `PointIncentive` contract, the NFT `tokenId`, the accounting period or campaign, the point amount, and any reward-specific fields such as `rewardToken` and `rewardAmount`. Abstractly, a leaf can be viewed as:

$$\text{leaf} = \text{hash}(\text{chainId}, \text{pointIncentive}, \text{tokenId}, \text{period}, \text{points}, \text{rewardToken}, \text{rewardAmount}). \quad (87)$$

The exact leaf format is part of the external distributor, not the hook.

The distributor can choose whether rewards are claimable by the owner of the NFT at a chosen snapshot block, by the current owner of the NFT at claim time, or by another rule announced for the reward program. This separation keeps swaps, donations, and position modifications free from reward-transfer logic while preserving an auditable on-chain source of point data.

Remark 65. *Since live-period data remains in `poolData`, Merkle allocations should normally use finalized periods only. A period becomes finalized only after the pool is touched in a later period or after a later touch finalizes accounting up to `endPeriod`.*

To initialize pools with the `PointIncentive` hook, the guidelines in Table 4 must be followed.

References

- [1] “NoFeeSwap Litepaper,” 2025, [Access].
- [2] R. Hanson, “Logarithmic market scoring rules for modular combinatorial information aggregation,” *The Journal of Prediction Markets*, vol. 1, no. 1, pp. 3–15, 2007.

Table 4: Permitted flag values for initialization of pools with a PointIncentive hook.

Flag	Value
isPreInitialize	1
isPostInitialize	0
isPreMint	0
isMidMint	1
isPostMint	0
isPreBurn	0
isMidBurn	1
isPostBurn	0
isPreSwap	0
isMidSwap	1
isPostSwap	0
isPreDonate	0
isMidDonate	1 (iff donate allowed)
isPostDonate	0
isPreModifyKernel	0
isMidModifyKernel	0
isPostModifyKernel	0
isMutableKernel	0 or 1
isMutablePoolGrowthPortion	0 or 1
isDonateAllowed	0 or 1

- [3] “Uniswap v3 Whitepaper,” 2021, [Access].
- [4] “Curve v1 Whitepaper,” 2021, [Access].
- [5] “Balancer v1 Whitepaper,” 2019, [Access].
- [6] “Uniswap v2 Whitepaper,” 2020, [Access].
- [7] “NoFeeSwap GitHub,” 2025, [Access].

Draft - NoFeeSwap